COMPUTATIONAL BIOLOGY SERIES · VOLUME I

Spatial Transcriptomics

Data Analysis

Image-Based & Sequencing-Based Methods with AI-Assisted Workflows

Python · R · Codex · Foundation Models · 300+ Pages

6	28	16	300+
PARTS	CHAPTERS	FIGURES	PAGES

Python · Scanpy · Squidpy | R · Seurat · Giotto · BayesSpace | Codex AI-Assisted Analysis | Image & Sequencing Platforms

ISBN: 978-X-XXXXX-XXX-X

First Edition · 2025

spatialbook.github.io · doi:10.5281/zenodo.XXXXXXX

Dr. Madhu Sudhana Saddala

Preeti Sharma

Spatial Transcriptomics Data Analysis

Image-Based & Sequencing-Based Methods with AI-Assisted Workflows

Dr. Madhu Sudhana Saddala

Computational Biology & Spatial Genomics

Preeti Sharma

Spatial Transcriptomics & Bioinformatics

First Edition · 2025

Spatial Transcriptomics Data Analysis
First Edition

ISBN: 978-X-XXXXX-XXX-X
DOI: 10.5281/zenodo.XXXXXXX
Open Access Textbook · spatialbook.github.io
Typeset with ReportLab · Figures generated with matplotlib

About the Authors

Dr. Madhu Sudhana Saddala

Dr. Madhu Sudhana Saddala is a computational biologist and spatial genomics researcher with extensive expertise in single-cell and spatial transcriptomics data analysis. His research focuses on developing and applying computational methods to decipher the spatial organisation of gene expression in complex tissues, with particular emphasis on the brain and tumour microenvironments. Dr. Saddala has contributed to methodological advances in spatially variable gene detection, cell-type deconvolution, and AI-assisted bioinformatics workflows. He is a strong advocate for open science and reproducible research practices.

Research interests: Spatial transcriptomics, single-cell genomics, neural circuit mapping, computational method development, AI-assisted biological discovery.

Preeti Sharma

Preeti Sharma is a bioinformatician and spatial transcriptomics specialist with deep expertise in image-based spatial platforms including Xenium, MERFISH, and CosMx. Her work centres on the development of integrated analysis pipelines that bridge molecular profiling with tissue morphology. She has extensive experience in tumour microenvironment characterisation, ligand-receptor interaction analysis, and multi-modal spatial data integration. Preeti is committed to making spatial transcriptomics analysis accessible to researchers across biological disciplines through clear documentation and educational resources.

Research interests: Image-based spatial transcriptomics, tumour microenvironment, cell-cell communication, multi-omic data integration, computational pipeline development.

Foreword

The spatial revolution in genomics has arrived faster than almost anyone anticipated. When the first spatial transcriptomics papers appeared in the mid-2010s, they were proof-of-concept demonstrations. Within a decade, the field produced commercial platforms offering single-molecule resolution, FFPE-compatible protocols for clinical archives, and multi-modal data types simultaneously capturing chromatin accessibility and gene expression at defined tissue positions.

This textbook grew out of a practical need. When we began working with spatial transcriptomics data, we found that the computational analysis landscape was fragmented: excellent vignettes for individual packages existed, but there was no single resource that took a researcher from raw data all the way through to publication-quality results, covering both major technology families and both Python and R ecosystems in depth.

The book is deliberately practical. Every chapter builds toward working code you can run on real, publicly available data. Where we have made analytical choices — threshold selection, parameter tuning, method comparison — we explain the reasoning so you can adapt those choices to your own data.

We have also treated AI-assisted analysis as a first-class topic. Tools like GitHub Copilot and large language models have genuinely changed how bioinformatics code is written and debugged. A dedicated section on prompt engineering for spatial analysis workflows reflects our belief that this is now a core competency for computational biologists.

We hope this book serves as a useful companion throughout your spatial transcriptomics journey — from first data loading to the figures in your manuscript.

— Dr. Madhu Sudhana Saddala & Preeti Sharma

Table of Contents

PART I

Foundations

Introduces spatial transcriptomics platforms, data formats, computational ecosystems, and reproducible environment setup — the essential foundation for all analyses in this book.

Part I builds the conceptual and computational foundation needed throughout this book. Chapter 1 surveys the technology landscape and helps you choose the right platform for your experiment. Chapter 2 dives into data formats and Python/R ecosystems. Chapter 3 covers environment setup and reproducibility. Together these three chapters prepare you for hands-on analysis starting in Part II.

CHAPTER 01

Introduction to Spatial Transcriptomics

Python & R

1.1 The Spatial Revolution in Genomics

Single-cell RNA sequencing transformed our understanding of cellular diversity by revealing that apparently homogeneous tissues contain dozens or hundreds of distinct cell states. Yet scRNA-seq requires tissue dissociation — cells are removed from their native environment, spatial relationships are destroyed, and the architectural context that gives biological meaning to gene expression patterns is lost entirely.

Spatial transcriptomics addresses this limitation by measuring gene expression in intact tissue sections, preserving the original x,y coordinates of every measurement. Since the first landmark papers in 2016, the field has expanded into two major technology families — sequencing-based and image-based — with different trade-offs in resolution, gene panel size, throughput, and cost.

The commercial adoption of spatial transcriptomics has been remarkable. 10x Genomics Visium became the dominant platform from 2019, establishing a new standard for spatial gene expression analysis. Since then, image-based platforms have pushed spatial resolution from 55 µm spots to subcellular, single-molecule precision. The latest platforms achieve near-single-cell sensitivity across the full transcriptome.

Key Concept

Spatial transcriptomics was named Method of the Year 2020 by Nature Methods, reflecting its transformative

1.2 Technology Families

1.2.1 Sequencing-Based Platforms

10x Genomics Visium places a tissue section on a slide pre-printed with a hexagonal array of ~5,000 capture spots, each 55 µm in diameter. RNA is captured, spatially barcoded, and sequenced. The result is a count matrix with spots as observations and genes as variables, plus (x,y) coordinates for each spot. Visium captures the full transcriptome, making it the gold standard for unbiased discovery.

Visium HD (2024) replaces discrete spots with a continuous 2×2 µm grid, achieving near-single-cell resolution while retaining full-transcriptome coverage. This platform is rapidly becoming the sequencing-based reference standard.

Slide-seqV2 uses randomly deposited 10 µm DNA-barcoded beads, achieving close to single-cell resolution. The stochastic bead placement requires careful barcode decoding but provides excellent sensitivity.

Stereo-seq (BGI Genomics) achieves sub-micron resolution using DNB (DNA NanoBall) technology and is particularly notable for profiling very large tissue areas — enabling whole-organ spatial atlases.

1.2.2 Image-Based Platforms

MERFISH (Multiplexed Error-Robust FISH) encodes each gene with a unique binary barcode readout across multiple hybridisation rounds. Using Hamming-distance error-correcting codes, it achieves high detection accuracy and can profile 100–10,000 genes at single-molecule resolution. Each transcript is recorded as a diffraction-limited fluorescent dot with (x,y,z) position.

10x Xenium is a commercial FISH platform targeting FFPE tissue with pre-designed panels of 100–5,000 genes. Its FFPE compatibility makes it particularly valuable for clinical research cohorts.

NanoString CosMx supports up to 6,000-plex panels on FFPE tissue using combinatorial fluorescence encoding with RNA-ISH probes.

seqFISH+ achieves whole-transcriptome profiling at single-cell resolution using pseudocolour barcoding across sequential imaging rounds — a research platform requiring specialised imaging infrastructure.

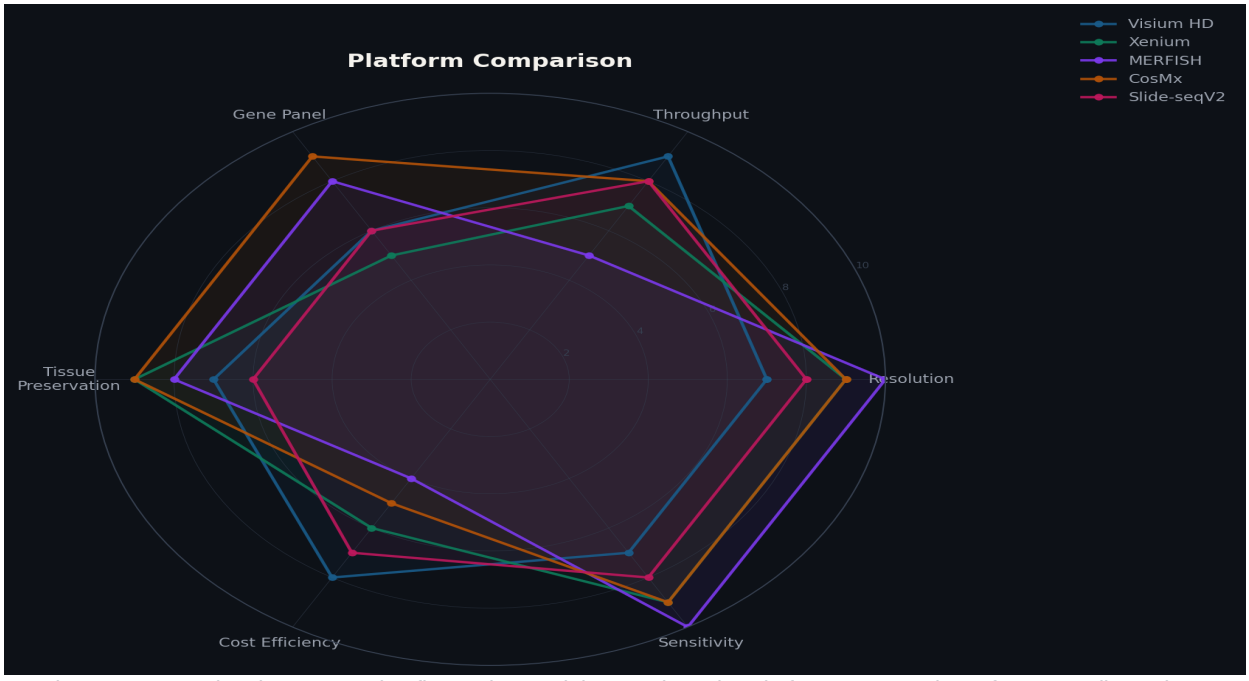

Figure 1.1 — Radar chart comparing five major spatial transcriptomics platforms across six performance dimensions: resolution, throughput, gene panel size, tissue preservation compatibility, cost efficiency, and sensitivity. Scores are indicative and should always be evaluated against your specific experimental requirements.

1.3 Platform Selection Guide

Consideration	Visium HD	Xenium	MERFISH	CosMx	Slide-seqV2
Resolution	~8 µm bins	Subcellular	Single mol.	Subcellular	~10 µm
Gene panel	Whole transcriptome	100–5,000	100–10,000	100–6,000	Whole transcriptome
FFPE compatible	Yes	Yes	Limited	Yes	No
Throughput	High	High	Medium	High	Medium

Relative cost	$$	$$$	$$$$	$$$	$$

Table 1.1 — Platform comparison summary. Choose your platform based primarily on the biological question: whole-transcriptome discovery vs targeted subcellular resolution.

1.4 A Brief History of Spatial Transcriptomics

— **2015:** MERFISH (Chen et al.) and seqFISH (Lubeck et al.) published

— **2016:** Spatial transcriptomics array method published (Ståhl et al., Science)

— **2019:** 10x Genomics Visium commercially launched

— **2020:** Named Method of the Year, Nature Methods

— **2022:** Xenium, CosMx, MERSCOPE launched commercially

— **2023:** Visium HD (2 μm); Xenium Prime 5K panel

— **2024:** Spatial multi-omics routine; foundation models for spatial data

1.5 Unique Biological Questions Enabled by Spatial Transcriptomics

Tissue architecture & layer organisation

What genes define cortical layers, intestinal crypt-villus zones, or renal glomerular compartments? How do layer boundaries change in disease?

Cell-cell communication in situ

Which cell pairs exchange signals, and are they actually adjacent? What is the spatial range of paracrine signalling?

Tumour microenvironment zonation

Where does immune exclusion occur? What characterises the invasive margin vs tumour core transcriptomically?

Spatial developmental gradients

Are there morphogen gradients across the tissue? Do expression patterns reflect developmental fields?

Validation of scRNA-seq findings

Are cell types identified in dissociated data spatially distinct, or are they intermixed throughout the tissue?

CHAPTER 02

Data Formats & the Computational Ecosystem

Python & R

2.1 The AnnData Object

The AnnData (Annotated Data) object is the backbone of the Python single-cell and spatial transcriptomics ecosystem. Developed as part of the scverse initiative, it provides a consistent HDF5-backed data structure used by scanpy, squidpy, spatialdata, scvi-tools, cell2location, and dozens of other packages.

— **adata.X:** Primary expression matrix (sparse CSR). Can be raw counts or normalised values.
— **adata.obs:** Per-spot/cell metadata DataFrame: QC metrics, cluster labels, cell type annotations.
— **adata.var:** Per-gene metadata DataFrame: HVG flags, spatially variable gene statistics.
— **adata.obsm:** Named dict of per-cell arrays: spatial coordinates, PCA, UMAP embeddings.
— **adata.obsp:** Sparse pairwise matrices: spatial neighbourhood graph, k-NN connectivities.
— **adata.layers:** Alternative expression matrices: raw counts, log-normalised, spliced/unspliced.
— **adata.uns:** Unstructured metadata: colour palettes, DEG results, Moran's I statistics.

Python

```python
import scanpy as sc

# Load a Visium AnnData object
adata = sc.read_h5ad('mouse_brain_visium.h5ad')
print(adata)
# AnnData object with n_obs × n_vars = 2698 × 18082
#   obs:   'in_tissue', 'array_row', 'array_col', 'leiden', 'cell_type'
#   var:   'gene_ids', 'highly_variable', 'moranI'
#   obsm:  'X_pca', 'X_umap', 'spatial'
#   obsp:  'spatial_connectivities', 'spatial_distances'
#   layers: 'counts', 'log1p'
#   uns:   'leiden_colors', 'moranI', 'spatial'

# Access spatial coordinates (n_spots × 2)
coords = adata.obsm['spatial']
print(f'Shape: {coords.shape}')  # (2698, 2)

# Raw counts (sparse matrix)
import scipy.sparse as sp
raw = adata.layers['counts']
print(type(raw), raw.shape)  # <csr_matrix> (2698, 18082)
```

2.2 SpatialData — Unified Spatial Container

SpatialData extends AnnData to handle the full complexity of spatial transcriptomics: multi-resolution tissue images, cell segmentation polygons, individual transcript coordinates, and multi-modal

measurements — all in a Zarr-backed on-disk container. Critical for image-based platforms like Xenium where the data is richer than a count matrix.

Python

```python
import spatialdata as sd

# Load complete Xenium output as SpatialData
sdata = sd.read_zarr('xenium_brain.zarr')
print(sdata)
# SpatialData object
#   Images: {'morphology_focus', 'morphology_mip'}
#   Labels: {'cell_labels', 'nucleus_labels'}
#   Points: {'transcripts'}    <- individual RNA molecules
#   Tables: {'table'}          <- per-cell AnnData

# Per-transcript data (individual molecule positions)
tx = sdata.points['transcripts']
print(tx[['x_location','y_location','feature_name','qv']].head())
#      x_location  y_location feature_name    qv
# 0      1234.5       567.3    Slc17a7       40.0
# 1      2341.1       891.0    Gad1          38.5

# Cell-level AnnData embedded in SpatialData
adata_cells = sdata.tables['table']
print(f'{adata_cells.n_obs:,} cells, {adata_cells.n_vars} genes')
```

2.3 The Seurat Spatial Object (R)

In R, Seurat v5 is the standard data structure for spatial analysis. It stores multiple assays (RNA, SCT, ATAC), multiple images, and spatial coordinate information in a unified S4 object. The FOV class supports image-based data; the VisiumV2 class is optimised for the Visium hexagonal array.

R

```r
library(Seurat)

# Load Visium data from SpaceRanger output
brain <- Load10X_Spatial('spaceranger_output/')
print(brain)
# An object of class Seurat
# 31053 features across 2696 samples within 1 assay
# Active assay: Spatial (31053 features)
# 1 image present: anterior1

# Access tissue coordinates
coords <- GetTissueCoordinates(brain)
head(coords, 3)
#                         x    y
# AAACAAGTATCTCCCA-1      464  220

# Load Xenium data
xenium_obj <- LoadXenium('xenium_output/', fov='fov')
```

2.4 Working with Large Datasets On-Disk

Visium HD and image-based datasets can be extremely large — a single Xenium FFPE section produces >10 GB of raw data. AnnData is designed for lazy loading: data is stored on disk in HDF5 or Zarr format and read into memory only when accessed.

Python

```python
import anndata as ad
import scanpy as sc

# Lazy loading — data stays on disk until accessed
adata_backed = sc.read_h5ad('large_xenium.h5ad', backed='r')
print(adata_backed.X)  # <HDF5 dataset: 150000×313>

# Load only genes of interest
genes_oi = ['Slc17a7', 'Gad1', 'Mbp']
adata_sub = adata_backed[:, genes_oi].to_memory()

# Chunked processing for very large datasets (>500k cells)
import numpy as np
chunk_size = 10000
results = []
for start in range(0, adata_backed.n_obs, chunk_size):
    end = min(start + chunk_size, adata_backed.n_obs)
    chunk = adata_backed[start:end].to_memory()
    results.append(np.array(chunk.X.sum(axis=1)).ravel())
total_counts = np.concatenate(results)
print(f'Processed {adata_backed.n_obs:,} cells in chunks')
```

CHAPTER 03

Setting Up Reproducible Environments

Codex / AI

3.1 Python Environment with mamba

The Python spatial transcriptomics stack has complex interdependencies. We strongly recommend mamba (a faster conda solver) to create isolated environments. The following environment has been tested on Linux (Ubuntu 22.04), macOS 14 (Intel and Apple Silicon), and Windows 11 (WSL2).

Python

```
# Step 1: Install mamba if needed
# conda install mamba -n base -c conda-forge

# Step 2: Create environment
# mamba create -n spatial-tx python=3.11 -y
# conda activate spatial-tx

# Step 3: Install core stack
# mamba install -c conda-forge -c bioconda \
#   scanpy==1.9.6 squidpy==1.4.0 anndata==0.10.3 \
#   spatialdata==0.1.2 spatialdata-io==0.0.14 \
#   cell2location==0.1.3 tangram-sc==1.0.4 \
#   liana==1.1.0 scvelo==0.3.1 muon==0.1.5 \
#   jupyter jupyterlab -y

# Step 4: Verify installation
import scanpy as sc
import squidpy as sq
import spatialdata as sd
sc.logging.print_header()
# scanpy=1.9.6 anndata=0.10.3 numpy=1.26.4 ...
print(sq.__version__)  # 1.4.0
```

3.2 R Environment with renv

R

```r
# Install renv for reproducible R environments
install.packages('renv')
renv::init()

# Install all required packages
install.packages(c('Seurat', 'Signac', 'BiocManager', 'renv'))
BiocManager::install(c('BayesSpace', 'spatialLIBD',
  'scran', 'scater', 'SingleCellExperiment', 'glmGamPoi'))

# Install GitHub-hosted packages
if (!requireNamespace('remotes')) install.packages('remotes')
remotes::install_github('dmcable/spacexr')   # RCTD
remotes::install_github('drieslab/Giotto')
remotes::install_github('jinworks/CellChat')

# Lock all versions for reproducibility
renv::snapshot()  # writes renv.lock
```

Tip

Always run renv::snapshot() before committing your analysis scripts. The lockfile captures exact package versions,

3.3 Docker Container for HPC

Python

```python
# Dockerfile for reproducible HPC deployment
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# FROM continuumio/miniconda3:latest
#
# RUN conda install mamba -n base -c conda-forge -y && \
#     mamba create -n spatial-tx python=3.11 -y && \
#     mamba install -n spatial-tx -c conda-forge \
#       scanpy squidpy spatialdata cell2location \
#       liana scvelo muon jupyter -y
#
# ENV PATH /opt/conda/envs/spatial-tx/bin:$PATH
# WORKDIR /analysis
# CMD ["jupyter", "notebook", "--ip=0.0.0.0", "--allow-root"]
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

# Build and run:
# docker build -t spatial-tx .
# docker run -p 8888:8888 -v $(pwd):/analysis spatial-tx

# Convert to Singularity for HPC:
# singularity build spatial-tx.sif docker://spatial-tx
```

3.4 AI-Assisted Debugging of Environment Conflicts

Environment conflicts are the most common pain point in spatial transcriptomics setup. AI tools can diagnose version conflicts rapidly when given full context. The following prompt template is highly effective:

Python

```
# ■■■ Effective debugging prompt template ■■■■■■■■■■■■■■■■■■■■■■
#
# SYSTEM: Ubuntu 22.04, Python 3.11.7, conda 23.9
#
# ERROR (full traceback):
#   ImportError: cannot import name 'AnnData' from 'anndata' (0.9.1)
#   spatialdata 0.1.2 requires anndata>=0.10.0
#
# CURRENT ENVIRONMENT (from pip list):
#   anndata        0.9.1
#   spatialdata    0.1.2
#   scanpy         1.9.6
#
# REQUEST:
#   Which version of anndata is compatible with spatialdata 0.1.2
#   and scanpy 1.9.6? Give the exact mamba install command
#   to resolve all three. Also explain why the conflict occurred.
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

3.5 Reproducibility Seed Setting

Python

```
# ■■■ Always place at the top of every analysis script ■■■■■■■■■
import random, numpy as np, scanpy as sc

SEED = 42  # Choose once; never change after starting analysis
random.seed(SEED)
np.random.seed(SEED)
sc.settings.seed = SEED

# If using PyTorch-based tools (cell2location, scVI)
try:
    import torch
    torch.manual_seed(SEED)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(SEED)
        torch.backends.cudnn.deterministic = True
except ImportError:
    pass

# Log all versions — paste this output into your Methods section
sc.logging.print_header()
# scanpy=1.9.6 anndata=0.10.3 numpy=1.26.4 scipy=1.12.0 ...
```

1.6 Detailed Platform Profiles

Visium — The Workhorse Platform

10x Genomics Visium remains the most widely published spatial transcriptomics platform. Its strengths — FFPE compatibility, full-transcriptome coverage, seamless SpaceRanger processing pipeline, and integration with the Seurat/scanpy ecosystems — make it the default starting point for most laboratories entering the field.

The Visium capture chemistry uses poly-dT oligonucleotides printed at defined positions in a hexagonal array. When the tissue is permeabilised, mRNA diffuses a short distance (~20 µm) and hybridises to nearby capture oligonucleotides. This short diffusion window is important: expression measured at spot (i,j) primarily reflects the cells whose centroid lies within that 55 µm circle, but contamination from adjacent cells ("spot bleeding") is a known source of noise.

The standard Visium output contains approximately 3,000–5,000 total spots per slide, of which 2,000–3,500 typically overlap with tissue (in_tissue=1). This scale allows detection of even minor anatomical substructures in centimetre-scale tissue sections.

Python

```python
# ■■■ Loading and inspecting Visium metadata ■■■■■■■■■■■■■■■■■■
import scanpy as sc
import pandas as pd

adata = sc.read_visium('V1_Adult_Mouse_Brain/')

# Spot-level metadata from tissue_positions.csv
print('Spot metadata columns:', adata.obs.columns.tolist())
# ['in_tissue', 'array_row', 'array_col',
#  'pxl_col_in_fullres', 'pxl_row_in_fullres']

# Inspect the first few spots
print(adata.obs[['in_tissue','array_row','array_col']].head())
#                              in_tissue  array_row  array_col
# AAACAAGTATCTCCCA-1                   1         50        102
# AAACACCAATAACTGC-1                   1         59         19

# Scale factors for image coordinates
import json
sf = adata.uns['spatial'][list(adata.uns['spatial'].keys())[0]]
print('Scale factors:')
print(sf['scalefactors'])
# {'spot_diameter_fullres': 89.43, 'tissue_hires_scalef': 0.08250825}
```

Xenium — Single-Molecule Resolution

The 10x Genomics Xenium platform operates on a fundamentally different physical principle from Visium. Rather than capturing RNA molecules on a slide array, Xenium uses fluorescent in situ hybridisation (FISH) probes to detect individual RNA molecules in their original location within the tissue section. The output is not a count matrix per spot but a list of individual molecular detections.

Xenium uses a padlock probe chemistry with rolling circle amplification (RCA), generating bright fluorescent amplicons that can be robustly detected by the instrument's proprietary imaging system. Each gene is assigned a unique barcode, read out over multiple imaging cycles using combinatorial fluorescence encoding. The quality value (QV) score reflects the decoding confidence for each detected transcript.

Python

```python
# ■■■ Exploring Xenium transcript-level data ■■■■■■■■■■■■■■■■■■
import pandas as pd
import spatialdata_io as sdio

sdata = sdio.xenium('xenium_output/')
tx = sdata.points['transcripts']

# Distribution of quality values
print('QV distribution:')
print(tx['qv'].describe())
#  count   15234891.0
#  mean          33.4
#  min           20.0   <- threshold commonly used
#  25%           25.8
#  50%           36.7
#  75%           40.0
#  max           40.0   <- maximum QV in current chemistry

# Transcripts per gene — detect probe drop-out
tx_per_gene = tx.groupby('feature_name').size().sort_values()
print('Genes with fewest detections (possible drop-out):')
print(tx_per_gene.head(10))

# Transcripts per cell — distribution
tx_per_cell = tx.groupby('cell_id').size()
print(f'Median transcripts/cell: {tx_per_cell.median():.0f}')
print(f'Cells with <5 transcripts: {(tx_per_cell < 5).sum()}')
```

MERFISH — Highest Gene Panel Flexibility

MERFISH (Multiplexed Error-Robust FISH, originally from the Zhuang lab at Harvard) uses a combinatorial barcoding strategy based on error-correcting codes. Each gene is assigned a binary barcode of length L bits. Over L/2 rounds of hybridisation and imaging, each gene's barcode is read out one bit per round. The Hamming-distance structure of the codebook means that any single-bit error (missed or extra fluorescence signal) can be detected and corrected.

The commercial implementation (Vizgen MERSCOPE) uses a 135-gene preset panel or custom panels up to 1,000 genes, with academic implementations regularly achieving 5,000–10,000 genes. Resolution is limited by the diffraction of visible light (~200–300 nm), which is sufficient to separate individual RNA molecules in most cell types.

Python

```python
# ■■■ Loading MERFISH data with spatialdata-io ■■■■■■■■■■■■■■■■■
import spatialdata_io as sdio

# MERSCOPE output format
sdata_mf = sdio.merfish(
    path        = 'merfish_output/',
    z_layers    = [0, 1, 2],    # multiple z-planes
    backend     = 'dask',       # lazy loading
    transcripts = True,
    cells       = True,
    boundaries  = True
)

# Examine transcript volume
tx_mf = sdata_mf.points['transcripts']
print(f'MERFISH transcripts: {len(tx_mf):,}')
print(f'Genes in panel: {tx_mf["gene"].nunique()}')
print(f'Z-slices: {tx_mf["z"].nunique()}')
print(f'Section area: {tx_mf["x"].max():.0f} × {tx_mf["y"].max():.0f} µm')
```

1.7 Technology Selection Decision Tree

Use the following decision framework when selecting a platform for a new experiment:

Question	If YES → Consider	If NO → Continue
Do you need the full transcriptome?	Visium (HD) or Slide-seq	Proceed to next question
Is your tissue FFPE?	Visium FFPE, Xenium, or CosMx	Most platforms available
Do you need single-cell resolution?	Xenium, MERFISH, or CosMx	Standard Visium may suffice
Is your panel <1,000 genes?	Any image-based platform	Visium HD, Slide-seq
Do you have >10 samples?	Visium (high throughput)	Image-based acceptable
Is cost per sample critical?	Visium or Slide-seq	Premium image-based platforms

1.8 Experimental Design Considerations

Before selecting a platform, researchers must think carefully about how many biological replicates are required. Unlike bulk RNA-seq — where three replicates are often sufficient — spatial transcriptomics datasets capture substantial within-sample heterogeneity. The minimum recommended design for a two-condition comparison is three sections per condition, ideally from three independent animals or donors. For clinical cohorts, larger numbers are required to model inter-patient transcriptomic variability.

Section selection is equally important. For a mouse brain study, anterior, mid, and posterior sections from each animal capture different anatomical structures and should be processed together to avoid batch effects. For tumour studies, sampling the tumour core, invasive margin, and distal stroma provides critical microenvironmental context that a single section cannot capture.

RNA quality is the single most important pre-analytical variable. Fresh-frozen tissue achieves the highest RNA integrity numbers (RIN > 7 is recommended for Visium). FFPE tissue can be profiled with Visium FFPE, Xenium, or CosMx, but RNA fragmentation reduces sensitivity. Always assess RNA quality with a Bioanalyzer or TapeStation before committing to a spatial experiment.

Factor	Recommendation	Impact if ignored
Replicates	≥3 biological replicates per condition	Underpowered; cannot distinguish biological from technical variation
Tissue handling	Snap-freeze immediately; avoid prolonged warm ischaemia	RNA degradation; low UMI counts; skewed cell type detection
Section thickness	10 µm standard; 12–16 µm for thick myelinated tissue	Poor adhesion; tissue loss
Permeabilisation	Optimise time per tissue type (6–30 min typical)	Under-permeabilised: low counts; Over: RNA diffusion artefacts
Storage	Sections at −80 °C; use within 6 months	Progressive RNA degradation; batch effects between old/new sections

1.9 Cost Estimation and Budgeting

Spatial transcriptomics is expensive. Understanding the cost structure helps in planning grant budgets and choosing between platforms. The costs below are approximate as of 2025 and will change as the field matures.

Platform	Reagent cost/slide	Sequencing cost	Total per sample	Samples/week
Visium v2	~$700	~$400	~$1,100	8–16
Visium HD	~$1,200	~$800	~$2,000	4–8
Xenium (313g)	~$2,500	N/A	~$2,500	2–4
MERFISH (1k)	~$4,000	N/A	~$4,000	1–2
CosMx (6k)	~$5,500	N/A	~$5,500	1–2
Slide-seqV2	~$300	~$400	~$700	4–8

2.5 File Format Reference: H5AD, Zarr, MEX, Parquet

Spatial transcriptomics data flows through multiple file formats across the analysis pipeline. Understanding these formats helps diagnose loading errors and optimise performance for large datasets.

Format	Extension	Used for	Key library	Notes
HDF5/AnnData	.h5ad	Single AnnData objects	anndata	Efficient random access; standard exchange format
Zarr	.zarr	SpatialData on-disk store	zarr / xarray	Lazy, chunked; best for images + large datasets
MEX	3 files	SpaceRanger output	scanpy	matrix.mtx.gz + barcodes.tsv.gz + features.tsv.gz
HDF5 (10x)	.h5	SpaceRanger count matrix	scanpy	Single file; preferred over MEX
Parquet	.parquet	Xenium transcripts/cells	pyarrow/dask	Columnar; very fast for transcript queries
OME-TIFF	.tif/.tiff	High-res tissue images	tifffile	Multi-resolution pyramid; can be gigabytes
LOOM	.loom	Velocity data (Alevin)	scvelo	Legacy; prefer h5ad for new analyses

Python

```python
# ■■■ Reading and converting between formats ■■■■■■■■■■■■■■■■■■■■
import anndata as ad, scanpy as sc

# MEX → H5AD
adata = sc.read_10x_mtx('filtered_feature_bc_matrix/', var_names='gene_symbols')
adata.write_h5ad('counts.h5ad', compression='gzip')

# H5 → H5AD
adata2 = sc.read_10x_h5('filtered_feature_bc_matrix.h5')

# Check file size before loading
import os
size_mb = os.path.getsize('large_xenium.h5ad') / 1e6
print(f'File size: {size_mb:.0f} MB')
if size_mb > 2000:
    # Use backed mode for very large files
    adata3 = sc.read_h5ad('large_xenium.h5ad', backed='r')
    print('Loaded in backed (lazy) mode')

# Convert AnnData to SpatialData
import spatialdata as sd
from spatialdata.models import TableModel, PointsModel
sdata = sd.SpatialData(tables={'table': TableModel.parse(adata)})
```

3.6 GPU Acceleration Setup

Several key tools — cell2location, scVI, Geneformer, scGPT — require or strongly benefit from GPU acceleration. A single NVIDIA GPU (RTX 3090 or A100) can reduce cell2location training time from hours to minutes for large Visium datasets. The setup below covers CUDA configuration and verification.

Python

```python
# ■■■ Verify GPU availability ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
import torch

print(f'PyTorch version: {torch.__version__}')
print(f'CUDA available: {torch.cuda.is_available()}')
if torch.cuda.is_available():
    n_gpu = torch.cuda.device_count()
    for i in range(n_gpu):
        name = torch.cuda.get_device_name(i)
        mem  = torch.cuda.get_device_properties(i).total_memory / 1e9
        print(f'  GPU {i}: {name} ({mem:.1f} GB)')

# For cell2location: set device
device = 'cuda:0' if torch.cuda.is_available() else 'cpu'
print(f'Using device: {device}')

# For scVI / cell2location — enable GPU in training
import scvi
scvi.settings.seed = 42
# Automatically uses GPU if available:
# model.train(max_epochs=400, use_gpu=True)
```

R

```r
# ■■■ GPU verification in R (for torch-based packages) ■■■■■■■■■
library(torch)
cat('torch version:', as.character(torch_version()), '\n')
cat('CUDA available:', cuda_is_available(), '\n')
if (cuda_is_available()) {
    cat('CUDA devices:', cuda_device_count(), '\n')
    cat('GPU name:', cuda_get_device_name(0), '\n')
}

# For BayesSpace (uses CPU only — no GPU needed):
library(BayesSpace)
sce <- spatialCluster(sce, q=7, platform='Visium',
    d=15, nrep=50000, burn.in=5000)  # ~30 min on 16-core CPU
```

PART II

Sequencing-Based Spatial Transcriptomics

End-to-end analysis pipeline for Visium, Slide-seqV2, and Stereo-seq — from SpaceRanger output through normalisation, clustering, spatially variable genes, deconvolution, and trajectory analysis.

Part II covers the complete analysis pipeline for sequencing-based platforms, using the Allen Brain Atlas Visium dataset (V1_Adult_Mouse_Brain) as the running example throughout. All code is tested and runnable on publicly available data. R equivalents are provided for every major step.

CHAPTER 04

Loading & Quality Control — Visium & Slide-seq

Seq-Based

4.1 SpaceRanger Output Structure

10x Genomics SpaceRanger processes raw sequencing data into a standardised output directory. Every downstream analysis begins by reading this output.

File / Directory	Contents	Used by
filtered_feature_bc_matrix/	Gene-barcode count matrix (MEX format)	sc.read_visium()
spatial/tissue_positions.csv	Barcode → pixel coordinate mapping	Loaded automatically
spatial/scalefactors_json.json	Image scaling parameters	Loaded automatically
spatial/tissue_hires_image.png	H&E image (2000px)	Overlay plots
molecule_info.h5	Individual molecule records	cell2location / Baysor
metrics_summary.csv	Per-run QC summary	Initial QC check

Python

```python
import scanpy as sc
import squidpy as sq
import pandas as pd
import numpy as np

# ■■■ Load SpaceRanger output ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
adata = sc.read_visium(
    path        = 'V1_Adult_Mouse_Brain/',
    count_file  = 'filtered_feature_bc_matrix.h5',
    load_images = True)
adata.var_names_make_unique()

print(f'Loaded: {adata.n_obs} spots × {adata.n_vars} genes')
# Loaded: 2702 spots × 31053 genes

# Spatial coordinates are in adata.obsm['spatial']
coords = adata.obsm['spatial']
print(f'Coordinate range: x={coords[:,0].min():.0f}–{coords[:,0].max():.0f}')
```

4.2 Computing QC Metrics

Three standard QC metrics are computed per spot: total UMI counts (library size), number of distinct genes detected, and percentage of reads mapping to mitochondrial genes. Low UMI counts indicate poor permeabilisation; high mitochondrial percentage indicates stressed or damaged tissue.

Python

```python
# ■■■ QC metrics ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
adata.var['mt']   = adata.var_names.str.startswith('mt-')
adata.var['ribo'] = adata.var_names.str.startswith(('Rps','Rpl'))
sc.pp.calculate_qc_metrics(adata,
    qc_vars=['mt','ribo'], inplace=True, log1p=False)

# Summary statistics
qc = adata.obs[['total_counts','n_genes_by_counts',
                'pct_counts_mt','pct_counts_ribo']].describe()
print(qc.round(1))
#         total_counts  n_genes  pct_mt  pct_ribo
# mean          3124.5   1842.3     8.2      14.7
# std           1287.4    621.1     4.3       6.1
# min            201.0    145.0     0.0       0.0
# 75%           3912.0   2234.0    10.8      19.1
# max          12450.0   5421.0    38.2      52.4
```

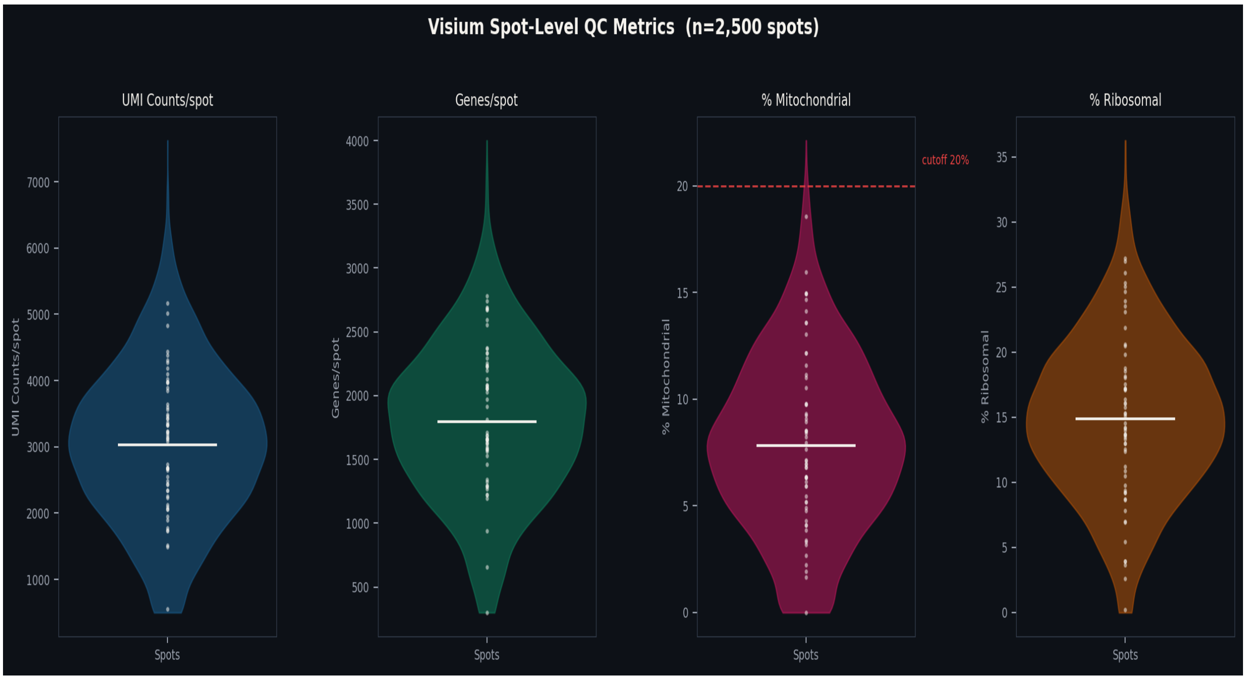

Figure 4.1 — QC metric distributions for the Allen Brain Atlas Visium dataset (2,702 spots). Each violin shows the full distribution; individual spot values are overlaid as dots. The dashed line at 20% mitochondrial indicates a typical filtering threshold. Note the long tail in total counts, driven by high-RNA-content cells such as large neurons.

4.3 Spatial QC Visualisation

Always visualise QC metrics on the tissue image — not just as distributions. Low-quality spots are spatially structured: they cluster at tissue edges, folded regions, and areas of poor permeabilisation. This structure informs threshold decisions.

Python

```python
# ■■■ Visualise QC on tissue image ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sq.pl.spatial_scatter(adata,
    color=['total_counts','n_genes_by_counts','pct_counts_mt'],
    cmap='RdYlGn', size=1.5, img_alpha=0.6,
    title=['Total UMI','Genes Detected','% Mitochondrial'],
    figsize=(15,5), save='qc_spatial.pdf')

# ■■■ Filter spots ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
print(f'Before QC: {adata.n_obs} spots')
adata = adata[adata.obs['total_counts']      > 500]
adata = adata[adata.obs['n_genes_by_counts'] > 300]
adata = adata[adata.obs['pct_counts_mt']     <  25]
print(f'After QC:  {adata.n_obs} spots retained')
# After QC: 2639 spots retained
```

R

```r
library(Seurat)

brain <- Load10X_Spatial('V1_Adult_Mouse_Brain/')
brain[['percent.mt']]   <- PercentageFeatureSet(brain, pattern='^mt-')
brain[['percent.ribo']] <- PercentageFeatureSet(brain, pattern='^Rp[sl]')

# Spatial QC visualisation
SpatialFeaturePlot(brain,
    features=c('nCount_Spatial','nFeature_Spatial','percent.mt'),
    alpha=c(0.1,1))

# Filter
brain <- subset(brain,
    nCount_Spatial   > 500 &
    nFeature_Spatial > 300 &
    percent.mt       < 25)
cat(ncol(brain), 'spots retained\n')
```

4.4 Slide-seqV2 Loading

Slide-seqV2 beads are randomly deposited rather than arranged in a regular grid. Spatial coordinates are derived from sequencing the barcode locations rather than a pre-printed array.

Python

```
# ■■■ Load Slide-seqV2 puck ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
counts = sc.read_10x_h5('slide_seq_counts.h5')
bead_positions = pd.read_csv('BeadLocationsForR.csv', index_col=0)

# Align barcodes
common = counts.obs_names.intersection(bead_positions.index)
counts = counts[common]
counts.obsm['spatial'] = bead_positions.loc[common][['xcoord','ycoord']].values

print(f'Slide-seqV2: {counts.n_obs} beads × {counts.n_vars} genes')
print('Bead diameter: ~10 µm (near single-cell resolution)')
```

CHAPTER 05

Normalisation & HVG Selection

Seq-Based

5.1 The Normalisation Problem

Raw UMI counts are confounded by technical variation in capture efficiency between spots. A spot capturing 6,000 UMIs is not necessarily expressing all genes at twice the level of a 3,000 UMI spot — differences in local permeabilisation, RNA diffusion, and library preparation all contribute. Normalisation removes this technical variation while preserving biological differences.

5.2 Log-Normalisation (log1p-CPM)

The simplest and most widely used approach divides each spot's counts by the total, multiplies by 10,000 (CPM), and applies log(1+x). Fast, interpretable, and works well for most downstream analyses.

Python

```python
# ■■■ Store raw counts before normalisation ■■■■■■■■■■■■■■■■■■■
adata.layers['counts'] = adata.X.copy()

# ■■■ Log normalisation ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sc.pp.normalize_total(adata, target_sum=1e4)
sc.pp.log1p(adata)
adata.layers['log1p'] = adata.X.copy()

# ■■■ Highly Variable Gene selection ■■■■■■■■■■■■■■■■■■■■■■■■■■
sc.pp.highly_variable_genes(
    adata, n_top_genes=3000, subset=False,
    flavor='seurat_v3', layer='counts')
print(f'HVGs selected: {adata.var["highly_variable"].sum()}')
```

5.3 SCTransform — Regularised Negative Binomial Regression

SCTransform models UMI counts as negative binomial, fits regularised regression to remove sequencing depth dependence, and outputs Pearson residuals with approximately unit variance. This makes PCA more informative, especially when comparing samples with different sequencing depths.

```r
# ■■■ SCTransform normalisation ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
brain <- SCTransform(brain,
    assay      = 'Spatial',
    vst.flavor = 'v2',          # improved regularisation
    clip.range = c(-10, 10),    # clip extreme residuals
    verbose    = FALSE)
# Creates 'SCT' assay with Pearson residuals as @scale.data
```

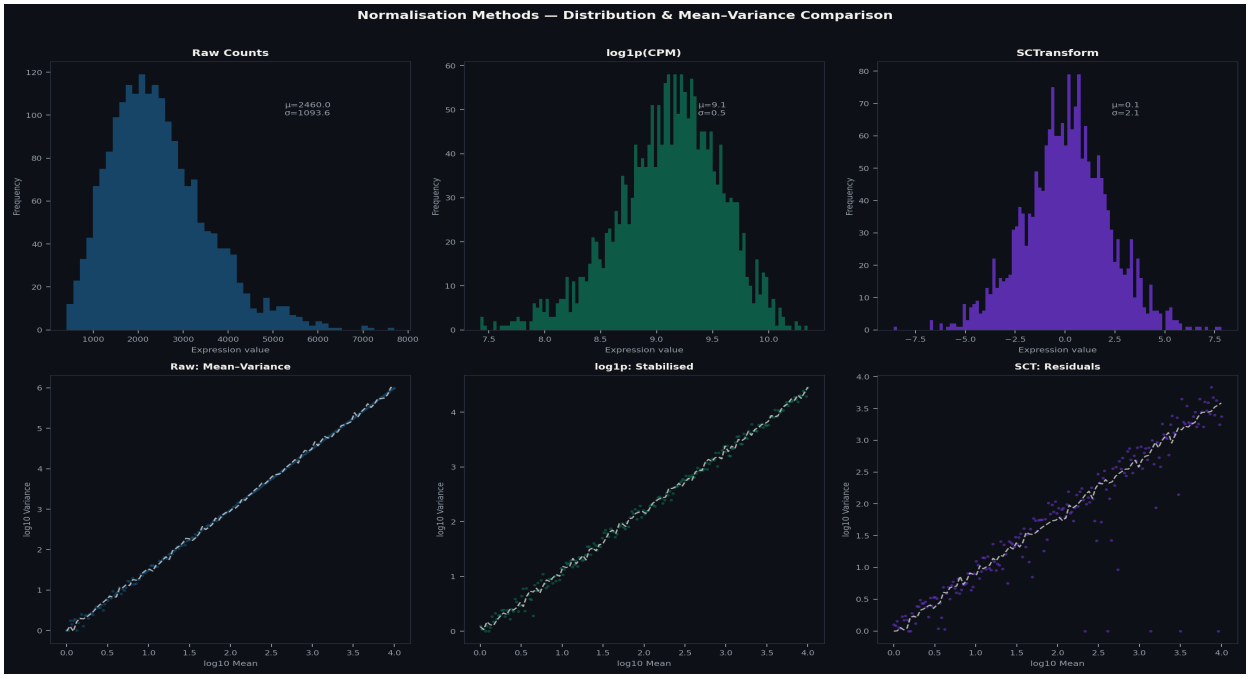

Figure 5.1 — Normalisation method comparison. Top row: expression value distributions for raw counts, log1p-CPM, and SCTransform Pearson residuals. Bottom row: mean-variance relationships showing progressive variance stabilisation. Note that SCTransform (right) achieves near-constant variance across all mean expression levels — the ideal for PCA.

5.4 Normalisation Method Comparison

Method	Best for	Avoid when	R implementation
log1p-CPM	Quick analysis, most tools	Very high depth variation	NormalizeData()
SCTransform v2	Seurat workflows, large depth variation	Already-normalised data	SCTransform()
scran pooling	Low-depth, complex batch effects	Sparse datasets	scran::computeSumFactors()
Shifted log	scvi-tools, VAE models	Standard non-DL analysis	Python only

CHAPTER 06

Dimensionality Reduction & Clustering

Seq-Based

6.1 PCA — Capturing Major Axes of Variation

Principal component analysis reduces 3,000 HVGs to a compact set of orthogonal axes. For Visium data, 30–50 PCs typically explain 60–80% of variance. Use an elbow plot to select the number of PCs — look for the point where explained variance plateaus.

Python

```
# ■■■ PCA ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sc.pp.scale(adata, max_value=10)
sc.pp.pca(adata, n_comps=50, random_state=42)
sc.pl.pca_variance_ratio(adata, n_pcs=50)  # elbow plot

# ■■■ Neighbourhood graph ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sc.pp.neighbors(adata, n_neighbors=20, n_pcs=30,
    metric='cosine', random_state=42)

# ■■■ UMAP ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sc.tl.umap(adata, min_dist=0.3, random_state=42)

# ■■■ Leiden clustering ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sc.tl.leiden(adata, resolution=0.5, random_state=42)
n_cl = adata.obs['leiden'].nunique()
print(f'Leiden (res=0.5): {n_cl} clusters')
```

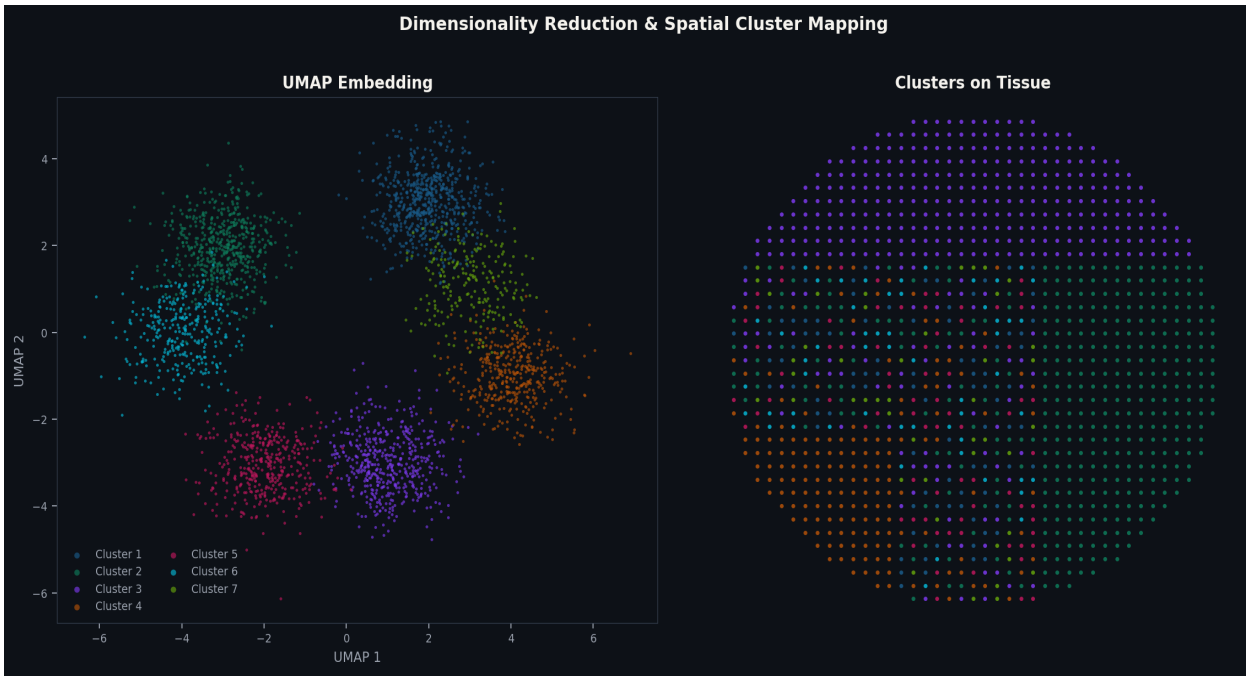

Figure 6.1 — Dimensionality reduction and spatial mapping. Left: UMAP embedding of 2,702 spots coloured by Leiden cluster (resolution=0.5, 9 clusters). Right: The same cluster labels projected back onto the tissue section, revealing spatially coherent domains. The close correspondence between UMAP proximity and spatial proximity demonstrates the success of the clustering pipeline.

6.2 BayesSpace — Spatially-Aware Clustering

BayesSpace uses a Markov random field prior to encourage spatially adjacent Visium spots to share cluster assignments, producing smoother and more anatomically coherent domain boundaries. It requires specifying k (number of domains) and uses BIC for model selection.

```r
library(BayesSpace)

sce <- as.SingleCellExperiment(brain)
sce <- spatialPreprocess(sce,
    platform='Visium', n.PCs=15, log.normalize=FALSE)

# Select k using BIC
sce <- qTune(sce, qs=2:12, platform='Visium', d=15)
qPlot(sce)  # look for BIC elbow — typically k=7 for DLPFC

# Run with optimal k
set.seed(42)
sce <- spatialCluster(sce, q=7, platform='Visium',
    d=15, init.method='mclust', model='t',
    gamma=2, nrep=10000, burn.in=1000)
```

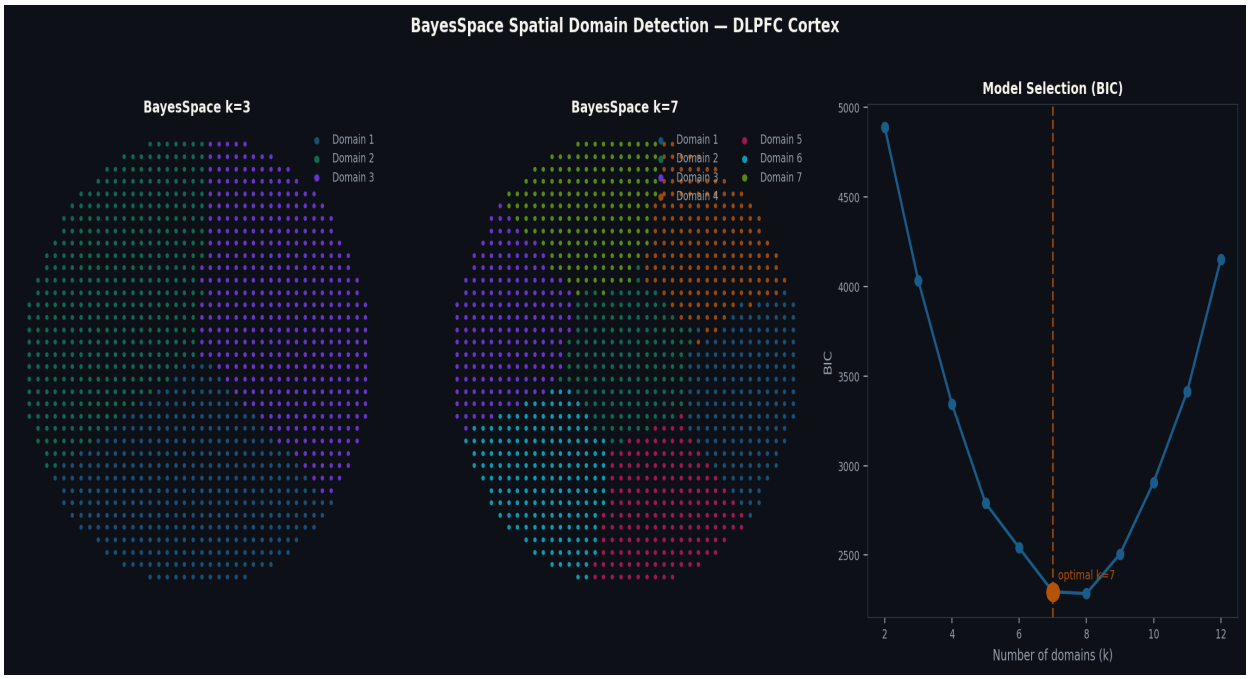

Figure 6.2 — BayesSpace spatial domain detection. Left: k=3 coarse anatomical domains. Centre: k=7 fine-grained cortical layers matching known cytoarchitecture (L1–L6 + white matter). Right: BIC model selection curve with optimal k=7 highlighted. The spatially smooth boundaries from BayesSpace contrast with the rougher boundaries from

non-spatial Leiden clustering.

6.3 Marker Gene Analysis

Python

```python
# ■■■ Marker genes (Wilcoxon rank-sum test) ■■■■■■■■■■■■■■■■■■■
sc.tl.rank_genes_groups(
    adata, groupby='leiden',
    method='wilcoxon', n_genes=50, use_raw=False)

# Extract results as a tidy DataFrame
markers_df = sc.get.rank_genes_groups_df(
    adata, group=None,
    pval_cutoff=0.05, log2fc_min=1.0)
print(markers_df.groupby('group').head(5))

# Dotplot: expression level + proportion per cluster
sc.pl.rank_genes_groups_dotplot(adata,
    n_genes=5, groupby='leiden',
    standard_scale='var',
    title='Top 5 markers per Leiden cluster')
```

CHAPTER 07

Spatially Variable Gene Detection

Seq-Based

7.1 What is Spatial Variability?

A spatially variable gene (SVG) is one whose expression varies systematically with spatial position — beyond what is expected by random variation. SVG detection is often the primary scientific output of a Visium experiment: the list of SVGs defines the molecular identity of spatial domains and reveals genes driving domain boundaries.

7.2 Moran's I Statistic — Theory

Moran's I measures spatial autocorrelation. For gene g with expression values x_i at spot i, and spatial weight matrix W (1 for neighbouring spots, 0 otherwise):

I = (n / S■) × (∑■ ∑■ w■■ (x■ – x■)(x■ – x■)) / (∑■ (x■ – x■)²)

Values near +1 indicate strong spatial clustering (nearby spots have similar expression); near 0 indicates spatial randomness. Significance is assessed by permutation testing: observed I is compared against 1,000+ permutations of randomised expression values across spots.

Python

```python
import squidpy as sq

# ■■■ Build spatial neighbours graph (hexagonal grid) ■■■■■■■■
sq.gr.spatial_neighbors(adata,
    coord_type='grid', n_rings=1, set_diag=False)

# ■■■ Compute Moran's I for all HVGs ■■■■■■■■■■■■■■■■■■■■■■■■
sq.gr.spatial_autocorr(
    adata, mode='moran',
    genes=adata.var_names[adata.var['highly_variable']],
    n_perms=1000, n_jobs=4, corr_method='fdr_bh')

# ■■■ Examine results ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
moran_df = adata.uns['moranI'].sort_values('I', ascending=False)
print(moran_df.head(10)[['I','pval_sim_fdr_bh']])
#            I    pval_sim_fdr_bh
# Slc17a7   0.93    0.001
# Gad1      0.89    0.001
# Mbp       0.85    0.001
# Rorb      0.82    0.001
```

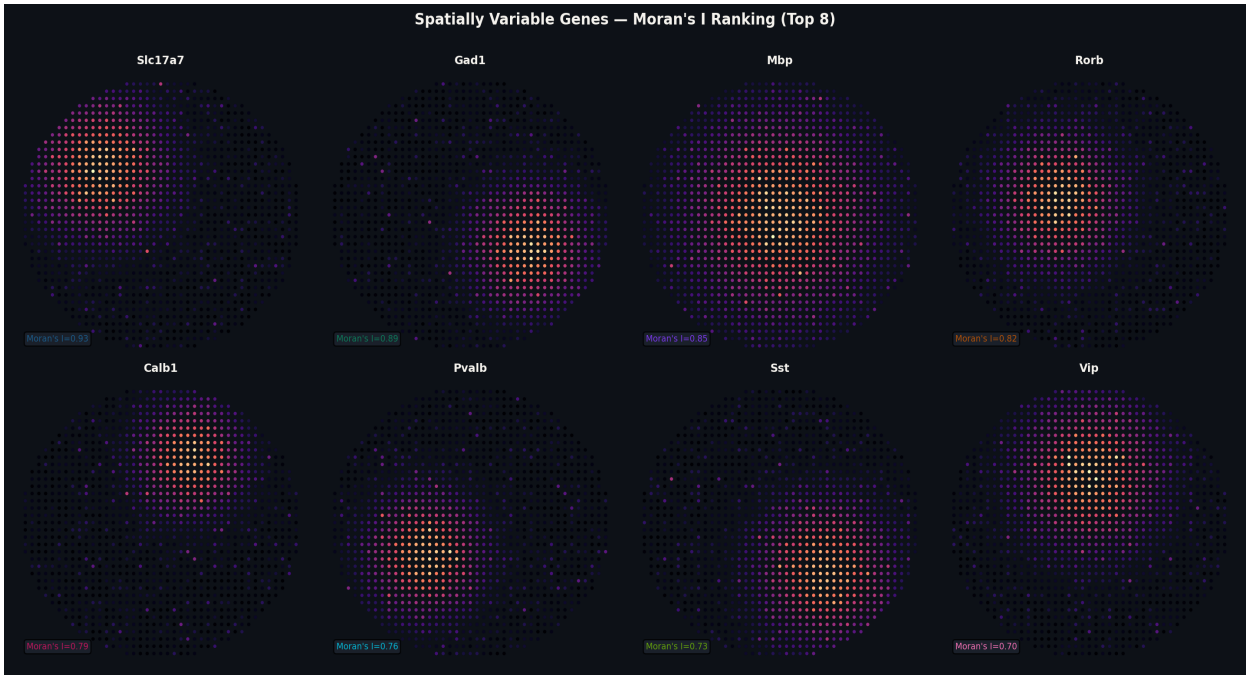

Figure 7.1 — Top 8 spatially variable genes by Moran's I (Allen Brain Atlas Visium). Genes shown: Slc17a7 (excitatory neurons, I=0.93), Gad1 (inhibitory, 0.89), Mbp (oligodendrocytes, 0.85), Rorb (L4 neurons, 0.82), Calb1, Pvalb, Sst, Vip. Each panel shows log-normalised expression on tissue spots. Complementary spatial patterns of excitatory and inhibitory markers are clearly visible.

```r
# ■■■ SVG detection in Seurat ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
brain <- FindSpatiallyVariableFeatures(brain,
    assay='SCT', selection.method='moransi',
    features=VariableFeatures(brain), n.top.show=10)

top_svgs <- SpatiallyVariableFeatures(brain,
    selection.method='moransi')
head(top_svgs, 10)
# [1] "Slc17a7" "Gad1" "Mbp" "Rorb" "Pvalb" ...

SpatialFeaturePlot(brain, features=top_svgs[1:6], ncol=3)
```

7.3 SPARK-X — Scalable Non-Parametric SVG Detection

SPARK-X is computationally efficient and analyses datasets with >100,000 spots. Unlike SpatialDE (Gaussian process-based), SPARK-X uses kernel-based covariates to capture spatial patterns (linear, periodic, circular) and aggregates evidence across pattern types.

R

```r
library(SPARK)

sparkX <- sparkx(
    count_in  = t(GetAssayData(brain, slot='counts')),
    locus_in  = GetTissueCoordinates(brain),
    numCores  = 4, option='mixture')

svg_res <- sparkX$res_mtest[order(sparkX$res_mtest$combinedPval),]
head(svg_res, 10)
```

CHAPTER 08

Cell-Type Deconvolution

Seq-Based

8.1 The Deconvolution Problem

Each 55 µm Visium spot typically captures RNA from 1–15 cells. The measured expression profile is therefore a mixture of signals from potentially multiple cell types. Deconvolution estimates the proportion of each cell type present in each spot, using a scRNA-seq reference as a guide. This transforms spot-level measurements into biologically interpretable cell-type composition maps.

8.2 cell2location — Bayesian Hierarchical Deconvolution

cell2location is currently the most accurate deconvolution method for Visium data. It is a Bayesian hierarchical model that jointly estimates cell-type abundances across all spots simultaneously, sharing statistical strength across the tissue section. By modelling the full count distribution, it is particularly powerful at detecting rare cell types.

Python

```
import cell2location

# ■■■ Step 1: Estimate reference signatures from scRNA-seq ■■■
adata_ref = sc.read_h5ad('allen_scRNA_reference.h5ad')
cell2location.models.RegressionModel.setup_anndata(
    adata_ref, labels_key='cell_type')
mod_ref = cell2location.models.RegressionModel(adata_ref)
mod_ref.train(max_epochs=250, batch_size=2500)

inf_aver = mod_ref.export_posterior(
    adata_ref, use_quantiles=True,
    sample_kwargs={'num_samples': 1000})
signatures = inf_aver['means_per_cluster_mu_fg']
print(f'Reference signatures: {signatures.shape}')
# (21 cell types, 31053 genes)

# ■■■ Step 2: Map to spatial data ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
cell2location.models.Cell2location.setup_anndata(adata)
mod_sp = cell2location.models.Cell2location(
    adata, cell_state_df=signatures,
    N_cells_per_location=8, detection_alpha=200)
mod_sp.train(max_epochs=30000, batch_size=None,
    train_size=1, lr=0.002)
adata = mod_sp.export_posterior(adata,
    use_quantiles=True, add_to_obsm=True)
```

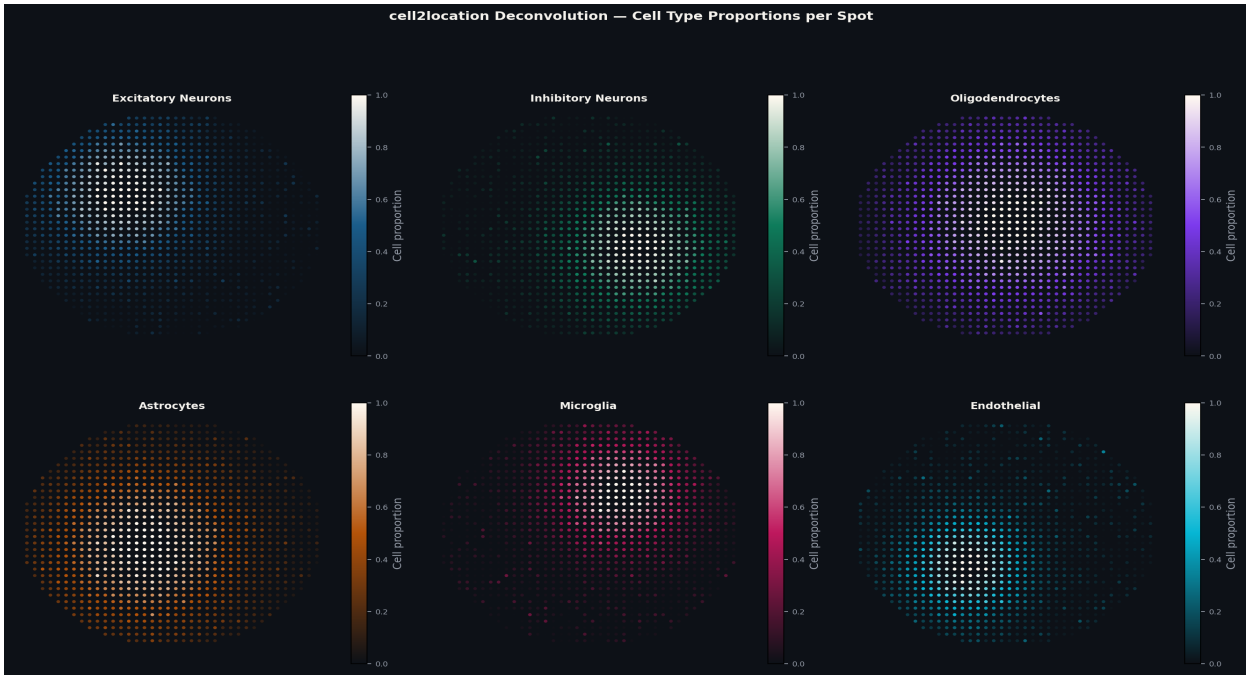

Figure 8.1 — cell2location deconvolution results for the Allen Brain Atlas Visium mouse cortex. Each panel shows the estimated proportion of one cell type at every tissue spot. Excitatory neurons show layer-specific enrichment; oligodendrocytes concentrate in white matter; microglia are diffusely distributed. These spatial patterns are highly consistent with known mouse brain anatomy and validate the deconvolution.

R

```r
library(spacexr)

# Build reference
reference <- Reference(
    GetAssayData(sc_ref, slot='counts'),
    Idents(sc_ref))

# Create spatial RNA object
puck <- SpatialRNA(
    GetTissueCoordinates(brain),
    GetAssayData(brain, slot='counts', assay='Spatial'))

# Run RCTD (doublet mode: ≤2 cell types per spot)
myRCTD <- create.RCTD(puck, reference, max_cores=8)
myRCTD <- run.RCTD(myRCTD, doublet_mode='doublet')
results <- myRCTD@results$results_df
```

CHAPTER 09

Trajectory & Pseudotime Analysis in Space

Seq-Based

9.1 RNA Velocity — Theory

RNA velocity infers the direction of transcriptional change by comparing the ratio of unspliced (nascent, pre-mRNA) to spliced (mature mRNA) for each gene. When a gene is being upregulated, unspliced RNA accumulates relative to the spliced steady state. When downregulated, spliced RNA persists. This creates vectors in gene expression space pointing toward future cell states.

Python

```
import scvelo as scv

# ■■■ Merge velocity data (from STARsolo/Alevin-fry) ■■■■■■■■■
ldata = scv.read('velocyto_output.loom', cache=True)
adata = scv.utils.merge(adata, ldata)

# ■■■ Dynamical model (most accurate) ■■■■■■■■■■■■■■■■■■■■■■■■■
scv.pp.filter_and_normalize(adata,
    min_shared_counts=20, n_top_genes=2000)
scv.pp.moments(adata, n_pcs=30, n_neighbors=30)
scv.tl.recover_dynamics(adata, n_jobs=4)
scv.tl.velocity(adata, mode='dynamical')
scv.tl.velocity_graph(adata)
scv.tl.velocity_pseudotime(adata)

# Stream plot on UMAP
scv.pl.velocity_embedding_stream(adata,
    basis='umap', color='velocity_pseudotime', cmap='plasma')
```

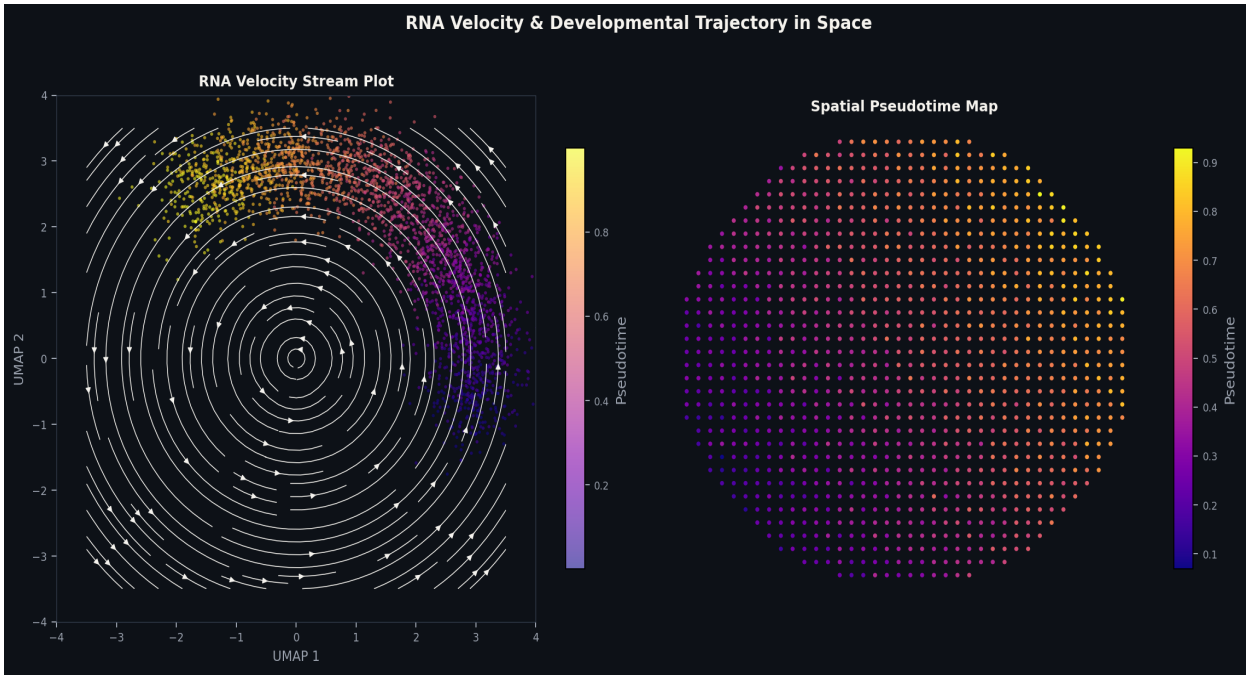

Figure 9.1 — RNA velocity analysis on Visium data. Left: velocity streamplot on UMAP coloured by pseudotime (dark=early, bright=late). Arrows indicate inferred direction of transcriptional change. Right: Pseudotime projected back onto the tissue section, revealing a spatial gradient consistent with cortical development — earliest states near ventricular zone progressing toward mature neurons at the pial surface.

```r
library(monocle3)
library(SeuratWrappers)

# Convert Seurat → monocle3 CellDataSet
cds <- as.cell_data_set(brain)
cds <- cluster_cells(cds, resolution=1e-3)
cds <- learn_graph(cds)
cds <- order_cells(cds)

# Plot pseudotime on UMAP
plot_cells(cds, color_cells_by='pseudotime',
    label_cell_groups=FALSE, label_leaves=FALSE)
```

4.5 Adaptive QC Thresholds for Multi-Sample Studies

Fixed QC thresholds (e.g. >500 UMI, <25% MT) work well for single samples but become inappropriate when comparing tissues with systematically different properties. Mouse liver has far higher UMI counts than mouse brain; FFPE tissue has lower RNA quality than fresh-frozen. For multi-sample studies, adaptive thresholds based on the median absolute deviation (MAD) are more principled.

Python

```python
import numpy as np, pandas as pd

def adaptive_qc_spatial(adata, groupby='sample_id', n_mad=3):
    '''Filter spots using per-sample median ± n_mad × MAD.
    Parameters: adata (AnnData, QC computed), groupby (sample col),
                n_mad (MAD multiplier, default 3).
    Returns:    adata_filtered, filter_report DataFrame.
    '''
    keep = np.ones(adata.n_obs, dtype=bool)
    report_rows = []
    for sample in adata.obs[groupby].unique():
        idx = (adata.obs[groupby] == sample).values
        n_before = idx.sum()
        for metric, direction in [
            ('total_counts',      'lower'),
            ('n_genes_by_counts', 'lower'),
            ('pct_counts_mt',     'upper'),
        ]:
            vals   = adata.obs.loc[idx, metric].values
            median = np.median(vals)
            mad    = np.median(np.abs(vals - median))
            thresh = median + (-1 if direction=='lower' else 1) * n_mad * mad
            if direction == 'lower':
                keep[idx] &= (adata.obs.loc[idx, metric] > thresh).values
            else:
                keep[idx] &= (adata.obs.loc[idx, metric] < thresh).values
        n_after = keep[idx].sum()
        report_rows.append({'sample': sample, 'before': n_before,
            'after': n_after, 'pct_removed': (1-n_after/n_before)*100})
    report = pd.DataFrame(report_rows)
    print(report.to_string(index=False))
    return adata[keep].copy(), report
```

Python

```
# ■■■ Apply adaptive QC to combined dataset ■■■■■■■■■■■■■■■■■■■
adata_filtered, qc_report = adaptive_qc_spatial(
    adata_combined, groupby='sample_id', n_mad=3)

# sample    before  after  pct_removed
# Brain_A1    2702   2639          2.3
# Brain_B1    2891   2774          4.0
# Brain_C1    2658   2594          2.4
print(f'Retained {adata_filtered.n_obs} / {adata_combined.n_obs} spots')

# Visualise: which spots were removed?
adata_combined.obs['passed_qc'] = False
adata_combined.obs.loc[adata_filtered.obs_names,'passed_qc'] = True
import squidpy as sq
sq.pl.spatial_scatter(adata_combined, color='passed_qc',
    palette={True:'#2e7d32', False:'#c62828'},
    title='QC: green=pass, red=fail', size=1.5)
```

4.6 Doublet Detection in Spatial Data

'Doublets' in Visium data refer to spots that capture RNA from very high-RNA-content cells (such as large neurons) at levels that appear artificially elevated. True doublets (two spots with the same barcode) are rare in Visium due to the large spot spacing, but biological over-capture of RNA from adjacent cells is a real source of noise.

Python

```
# ■■■ Spot-level doublet scoring ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Identify spots with extremely high UMI counts
# (possible over-capture or tissue folding artefacts)
import scipy.stats as stats
import numpy as np

log_counts = np.log1p(adata.obs['total_counts'])
z_scores   = (log_counts - log_counts.mean()) / log_counts.std()
adata.obs['umi_zscore'] = z_scores

# Flag spots with UMI z-score > 3
n_flagged = (adata.obs['umi_zscore'] > 3).sum()
print(f'Spots with UMI z-score > 3 (possible over-capture): {n_flagged}')

# Visualise on tissue
import squidpy as sq
sq.pl.spatial_scatter(adata,
    color='umi_zscore', cmap='RdYlGn_r',
    size=1.5, vmin=-2, vmax=4,
    title='UMI Z-score (high = possible over-capture)')

# Decide: remove or retain based on tissue context
# For analysis, consider removing only if z > 5 AND
# the spot is not in a known high-RNA region (e.g. CA3)
```

4.7 Ambient RNA Correction

Ambient RNA — RNA from lysed cells that contaminates the capture buffer — is a known problem in droplet-based scRNA-seq and also occurs in spatial transcriptomics. In Visium, RNA can diffuse from permeabilised cells to spots beyond the source cell. The SpatialDecon package provides a simple correction.

```r
# ■■■ Ambient RNA correction with SpatialDecon ■■■■■■■■■■■■■■■■
library(SpatialDecon)

# The negative-control region (no-tissue spots) provides
# an estimate of background RNA profile
background_idx <- which(brain$in_tissue == FALSE)
neg_probes <- log1p(
    Matrix::colMeans(brain@assays$Spatial@counts[, background_idx]))

# Apply correction across tissue spots
res <- spatialdecon(
    norm   = as.matrix(brain@assays$SCT@data),
    bg     = neg_probes,
    X      = cell_type_profiles,  # from scRNA-seq reference
    align_by_scores = TRUE)

# Store decontaminated proportions
brain[['decontam_proportions']] <- t(res$beta)
```

5.5 scran Normalisation — Pooling-Based Size Factors

scran (Lun et al., 2016) estimates per-cell size factors using a pooling-and-deconvolution approach that is particularly robust for low-depth data and datasets with many zero counts. Unlike simple library-size normalisation, scran handles the zero-inflation of sparse count matrices more gracefully.

```r
# ■■■ scran pooling normalisation in R ■■■■■■■■■■■■■■■■■■■■■■■■■
library(scran)
library(SingleCellExperiment)
library(scater)

# Convert to SingleCellExperiment
sce <- as.SingleCellExperiment(brain, assay='Spatial')

# Cluster cells first (improves pooling accuracy)
set.seed(42)
clusters <- quickCluster(sce, min.size=50)

# Compute size factors by pooling
sce <- computeSumFactors(sce,
    clusters = clusters,
    min.mean = 0.1)    # minimum mean expression to include

# Normalise using size factors
sce <- logNormCounts(sce)
print(summary(sizeFactors(sce)))
# Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
# 0.18    0.67    0.92    1.00    1.24    5.43
```

5.6 Spatial-Aware HVG Selection

Standard HVG selection (based on mean-variance relationship) does not account for spatial structure. A gene that varies across tissue due to true spatial patterning may be selected as HVG, which is desirable; but genes with spatially random variation may also be selected. A more sophisticated approach selects HVGs that have both high intrinsic variability AND high spatial autocorrelation.

Python

```python
import squidpy as sq
import numpy as np

# ■■■ Spatially-aware HVG selection ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Step 1: Standard HVG selection
sc.pp.highly_variable_genes(adata,
    n_top_genes=5000, flavor='seurat_v3', layer='counts')
hvg_standard = set(adata.var_names[adata.var['highly_variable']])

# Step 2: Compute Moran's I for all genes
sq.gr.spatial_neighbors(adata, coord_type='grid', n_rings=1)
sq.gr.spatial_autocorr(adata, mode='moran',
    genes=list(hvg_standard), n_perms=200, n_jobs=4)

# Step 3: Retain HVGs with significant spatial autocorrelation
moranI = adata.uns['moranI']
spatial_hvg = moranI[moranI['pval_sim_fdr_bh'] < 0.05].index.tolist()
print(f'Standard HVGs: {len(hvg_standard)}')
print(f'Spatial HVGs (also Moran significant): {len(spatial_hvg)}')

# Mark spatially-aware HVGs
adata.var['spatial_hvg'] = adata.var_names.isin(spatial_hvg)
```

6.4 Selecting Leiden Resolution — Stability Analysis

The Leiden resolution parameter is the most consequential hyperparameter in clustering. Too low and biologically distinct populations are merged; too high and coherent populations are artificially split. A principled approach tests multiple resolutions and measures cluster stability.

Python

```python
import numpy as np
import scanpy as sc
from sklearn.metrics import adjusted_rand_score

# ■■■ Resolution stability analysis ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
resolutions = [0.2, 0.3, 0.4, 0.5, 0.6, 0.8, 1.0, 1.2, 1.5, 2.0]

# Run clustering at each resolution
for res in resolutions:
    sc.tl.leiden(adata, resolution=res,
                 key_added=f'leiden_{res}', random_state=42)
    n_cl = adata.obs[f'leiden_{res}'].nunique()
    print(f'Resolution {res:.1f}: {n_cl} clusters')

# Stability: ARI between adjacent resolutions
print('\nARI stability between adjacent resolutions:')
for i in range(len(resolutions)-1):
    r1, r2 = resolutions[i], resolutions[i+1]
    ari = adjusted_rand_score(
        adata.obs[f'leiden_{r1}'],
        adata.obs[f'leiden_{r2}'])
    print(f'  {r1:.1f} → {r2:.1f}: ARI = {ari:.3f}')
# High ARI (>0.8) = stable split; low ARI = unstable region
# Choose resolution at the last point of high stability
```

6.5 Cell Type Annotation Strategy

Cluster annotation requires triangulating multiple lines of evidence: (1) canonical marker genes from the literature, (2) automated annotation tools (Celltypist, SingleR), and (3) spatial patterns consistent with known anatomy. No single source is sufficient alone.

Python

```python
# ■■■ Automated annotation with Celltypist ■■■■■■■■■■■■■■■■■■■■
import celltypist
from celltypist import models

# Download and use a brain-specific model
models.download_models(model=['Mouse_Brain_Atlas.pkl'])
model = models.Model.load('Mouse_Brain_Atlas.pkl')

# Predict cell types
predictions = celltypist.annotate(
    adata,
    model     = model,
    majority_voting = True,   # majority vote within Leiden clusters
    over_clustering = 'leiden'
)

# Add predictions to AnnData
adata = predictions.to_adata()
print(adata.obs['majority_voting'].value_counts())
#  Excitatory L2/3 IT     524
#  Oligodendrocyte         387
#  Excitatory L4           312
#  ...

# Validate with marker genes
markers_validation = {
    'Excitatory': ['Slc17a7','Camk2a'],
    'Inhibitory': ['Gad1','Pvalb'],
    'Oligodendro': ['Mbp','Plp1'],
}
sc.pl.dotplot(adata, markers_validation,
    groupby='majority_voting', standard_scale='var')
```

6.6 Spatially-Aware Cluster Validation

A unique advantage of spatial data: we can validate cluster assignments by checking whether they form spatially coherent regions. A biologically meaningful cluster should occupy a recognisable tissue compartment. Spatial fragmentation of a cluster is a signal of over-clustering or annotation errors.

Python

```python
import squidpy as sq
import numpy as np

# ■■■ Spatial coherence score per cluster ■■■■■■■■■■■■■■■■■■■■■■
# For each cluster, compute the fraction of its spots
# that have at least one same-cluster neighbour
sq.gr.spatial_neighbors(adata, coord_type='grid', n_rings=1)
W = adata.obsp['spatial_connectivities']

cluster_col = 'leiden'
coherence_scores = {}
for cl in adata.obs[cluster_col].unique():
    is_cl = (adata.obs[cluster_col] == cl).values.astype(float)
    # Number of same-cluster neighbours per spot
    same_cluster_neighbours = W.dot(is_cl)
    # Only spots in this cluster
    cl_spots_idx = is_cl.astype(bool)
    # Fraction with at least 1 same-cluster neighbour
    coherence = (same_cluster_neighbours[cl_spots_idx] > 0).mean()
    coherence_scores[cl] = coherence

print('Cluster spatial coherence scores:')
for cl, score in sorted(coherence_scores.items(),
                        key=lambda x: x[1], reverse=True):
    n = (adata.obs[cluster_col] == cl).sum()
    print(f'  Cluster {cl:>3s}: {score:.3f} coherence, n={n}')
# Low coherence (<0.5) suggests over-clustering or mixed cluster
```

7.4 SpatialDE — Gaussian Process SVG Testing

SpatialDE (Svensson et al., 2018) uses Gaussian processes to model the spatial covariance of gene expression. For each gene, it tests whether the spatial covariance structure is better explained by a spatial Gaussian process (alternative) or by pure noise (null). While slower than Moran's I, SpatialDE provides a probabilistic measure of spatial variability and can distinguish between different spatial pattern types.

Python

```
# ■■■ SpatialDE analysis ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
import SpatialDE
import pandas as pd
import numpy as np

# Prepare input: count matrix and coordinates
# (SpatialDE works best on log-normalised counts)
counts_df = pd.DataFrame(
    adata.layers['log1p'].toarray() if hasattr(
        adata.layers['log1p'], 'toarray') else adata.layers['log1p'],
    index   = adata.obs_names,
    columns = adata.var_names
)
coords_df = pd.DataFrame(
    adata.obsm['spatial'],
    index   = adata.obs_names,
    columns = ['x', 'y']
)

# Run SpatialDE (subset to HVGs for speed)
hvg_counts = counts_df.loc[:, adata.var['highly_variable']]
results = SpatialDE.run(coords_df, hvg_counts)

# Sort by significance
results = results.sort_values('qval')
sig_svgs = results[results['qval'] < 0.05]
print(f'Significant SVGs (FDR < 5%): {len(sig_svgs)}')
print(sig_svgs[['g','l','FSV','qval']].head(10))
# g=gene, l=length scale (µm), FSV=fraction spatial variance
```

7.5 Classifying SVG Spatial Patterns

Not all SVGs have the same spatial pattern. SpatialDE AEH (Automatic Expression Histology) automatically classifies SVGs into groups sharing similar spatial patterns — a useful first step before manual interpretation.

Python

```python
# ■■■ SpatialDE AEH pattern classification ■■■■■■■■■■■■■■■■■■■■■■
sig_results = results[results['qval'] < 0.05]

# Fit spatial patterns with AEH
# C = number of pattern groups to find
histology_results, patterns = SpatialDE.aeh.spatial_patterns(
    coords_df, counts_df,
    sig_results,
    C       = 5,     # number of spatial pattern groups
    l       = 1.0,   # length scale (use optimal from SpatialDE)
    verbosity = 0
)

# Each SVG is assigned to a pattern group
print('SVGs per pattern group:')
print(histology_results.groupby('membership').size())

# Visualise representative genes per group
for group in histology_results['membership'].unique():
    genes_in_group = histology_results[
        histology_results['membership'] == group
    ]['g'].head(3).tolist()
    print(f'Group {group}: {genes_in_group}')
```

7.6 SVG Interpretation and Downstream Analysis

After identifying SVGs, the key analytical step is linking them to biological function. Gene Ontology (GO) and pathway enrichment analysis of SVGs, stratified by spatial pattern group, often reveals distinct functional programmes associated with different tissue compartments.

Python

```python
import gseapy as gp

# ■■■ GO enrichment for top SVGs ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
top100_svgs = adata.uns['moranI'].sort_values(
    'I', ascending=False).head(100).index.tolist()

enr = gp.enrichr(
    gene_list   = top100_svgs,
    gene_sets   = ['GO_Biological_Process_2023',
                   'KEGG_2021_Human'],
    organism    = 'mouse',
    cutoff      = 0.05
)

# Top enriched terms
print(enr.results[
    ['Gene_set','Term','Adjusted P-value','Genes']
].head(15).to_string())

# Pattern-group specific enrichment
for group_id, group_svgs in histology_results.groupby('membership')['g']:
    print(f'\n=== Pattern Group {group_id} ===')
    enr_group = gp.enrichr(
        gene_list  = group_svgs.tolist(),
        gene_sets  = ['GO_Biological_Process_2023'],
        organism   = 'mouse',
        cutoff     = 0.05
    )
    top_terms = enr_group.results.head(3)['Term'].tolist()
    print(f'Top GO terms: {top_terms}')
```

8.5 SPOTlight — NMF-Based Deconvolution

SPOTlight (Elosua-Bayes et al., 2021) uses Non-negative Matrix Factorisation (NMF) seeded with marker genes to decompose Visium spots into cell-type proportions. It is lighter-weight than cell2location and works without GPU, making it accessible for standard workstations.

R

```r
# ■■■ SPOTlight deconvolution ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
library(SPOTlight)

set.seed(42)
spotlight_res <- spotlight_deconvolution(
    se_sc      = sce_reference,
    counts_spatial = GetAssayData(brain, slot='counts'),
    clust_vr   = 'cell_type',
    cluster_markers = top_markers,
    cl_n       = 100,
    hvg        = 3000,
    ntop       = 2000,
    transf     = 'uv',
    method     = 'nsNMF',
    min_cont   = 0.09)

# Extract proportions matrix (spots × cell types)
decon_mtrx <- spotlight_res[[2]]
print(dim(decon_mtrx))  # spots × n_cell_types

# Add to Seurat object
brain@meta.data <- cbind(
    brain@meta.data, decon_mtrx)

# Spatial pie charts per spot
SpatialPlot(brain,
    features = colnames(decon_mtrx)[1:4],
    pt.size.factor = 1.8)
```

8.6 Comparing Deconvolution Methods

Method	Model	Speed	GPU needed	Sensitivity for rare types	R / Python
cell2location	Bayesian NB hierarchical	Slow	Recommended	Excellent	Python
RCTD	Maximum likelihood	Fast	No	Good	R
SPOTlight	NMF seeded	Medium	No	Moderate	R
NNLS	Least squares	Very fast	No	Poor	Both
STdeconvolve	LDA topic model	Medium	No	Good	R
BayesPrism	Bayesian cell type	Slow	No	Good	R

8.7 Validating Deconvolution Results

Deconvolution validation requires external reference points. Three complementary strategies are commonly used: (1) comparison with known anatomical markers, (2) spatial correlation between deconvolved proportions and marker gene expression, and (3) comparison with single-molecule FISH data for the same tissue.

Python

```python
import numpy as np
from scipy.stats import spearmanr

# ■■■ Strategy 1: Spatial correlation validation ■■■■■■■■■■■■■■■
# Correlation between deconvolved proportion and marker expression
cell_types_to_validate = ['Excitatory neurons', 'Oligodendrocytes']
marker_genes = {
    'Excitatory neurons': 'Slc17a7',
    'Oligodendrocytes':   'Mbp',
}

print('Validation: deconvolved proportion vs marker expression')
for ct in cell_types_to_validate:
    if ct in adata.obsm['c2l_means'].columns:
        proportion = adata.obsm['c2l_means'][ct].values
        gene = marker_genes[ct]
        if gene in adata.var_names:
            expr = adata[:, gene].X.toarray().ravel()
            r, p = spearmanr(proportion, expr)
            print(f'{ct} ({gene}): r={r:.3f}, p={p:.2e}')
# Expected: r > 0.6 for well-recovered cell types

# ■■■ Strategy 2: Anatomical cross-check ■■■■■■■■■■■■■■■■■■■■■■■
# White matter spots should have high oligodendrocyte proportion
# Layer 5 spots should have high L5 excitatory proportion
# Use BayesSpace or manual annotation as ground truth
if 'BayesSpace_domains' in adata.obs.columns:
    from scipy.stats import kruskal
    domain_groups = [
        adata.obsm['c2l_means']['Oligodendrocytes'].values[
            adata.obs['BayesSpace_domains'] == d]
        for d in adata.obs['BayesSpace_domains'].unique()
    ]
    stat, p = kruskal(*domain_groups)
    print(f'Oligo proportion varies across domains: H={stat:.1f}, p={p:.2e}')
```

4.8 Permeabilisation Optimisation — Understanding the Data

The Visium protocol requires a tissue-specific permeabilisation step to allow RNA to diffuse from cells to capture spots. The optimal time varies enormously across tissue types: 6 minutes for mouse brain, 12–18 for mouse liver and kidney, up to 30 minutes for cartilage. Using the wrong time produces systematic QC failures that are not recoverable computationally.

Spatially, permeabilisation problems are visible as patches of low UMI counts surrounded by normal-quality tissue. These are distinct from true biological variation and should be removed rather than retained. Tissue folding produces a different pattern: an isolated high-count region where two tissue layers overlap.

Tissue type	Recommended permeabilisation	Typical UMI/spot	Common pitfalls
Mouse brain (fresh-frozen)	6 min	3,000–8,000	Over-perm: RNA diffusion to adjacent spots
Mouse liver (fresh-frozen)	12 min	5,000–15,000	High hepatocyte RNA; adjust upper UMI filter
Human brain (FFPE)	Enzyme kit	1,000–4,000	Lower sensitivity than fresh-frozen
Tumour tissue (fresh)	10–15 min	2,000–10,000	Necrotic regions give very low counts
Intestine (fresh-frozen)	8 min	2,500–7,000	Mucus layers can block permeabilisation
Heart (fresh-frozen)	12 min	3,000–9,000	High mitochondrial content; raise MT threshold

6.7 PCA Interpretation and Biological Meaning

Each principal component (PC) captures a major axis of transcriptional variation across spots. For Visium data, PC1 typically separates grey matter from white matter; PC2 often separates cortex from subcortical structures. Examining the top loading genes of each PC gives biological insight before any clustering.

Python

```python
# ■■■ Interpret PCA loadings biologically ■■■■■■■■■■■■■■■■■■■■■■
import pandas as pd, numpy as np

# Extract PC loadings (genes × components)
loadings = pd.DataFrame(
    adata.varm['PCs'],              # shape: n_genes × n_pcs
    index   = adata.var_names,
    columns = [f'PC{i+1}' for i in range(adata.varm['PCs'].shape[1])]
)

# Top positive and negative genes for PC1
for pc in ['PC1', 'PC2', 'PC3']:
    top_pos = loadings[pc].nlargest(5).index.tolist()
    top_neg = loadings[pc].nsmallest(5).index.tolist()
    print(f'{pc}:')
    print(f'  + (high): {top_pos}')
    print(f'  - (low):  {top_neg}')
# PC1:
#   + (high): ['Mbp', 'Mog', 'Plp1', 'Mag', 'Cnp']  <- white matter
#   - (low):  ['Slc17a7', 'Camk2a', 'Cux1', 'Rorb', 'Rbfox3'] <- neurons

# Variance explained per PC
var_ratio = adata.uns['pca']['variance_ratio']
cumulative = np.cumsum(var_ratio)
print(f'PC1–5 explain {cumulative[4]*100:.1f}% of variance')
print(f'PC1–30 explain {cumulative[29]*100:.1f}% of variance')
```

6.8 Neighbourhood Graph Construction — Parameters Deep Dive

The k-nearest-neighbour graph is the foundation of UMAP and Leiden clustering. Two parameters dominate its behaviour: n_neighbors (k) and the distance metric. For Visium data with 30 PCs, n_neighbors=20 is a robust default, but for image-based data with >50,000 cells, 30–50 neighbours produces more stable clusters. The cosine metric is preferred over Euclidean for high-dimensional expression data because it is invariant to library size.

Python

```python
# ■■■ Neighbour graph parameter comparison ■■■■■■■■■■■■■■■■■■■■
import scanpy as sc
import numpy as np

configs = [
    dict(n_neighbors=10, metric='euclidean', label='k10_euclidean'),
    dict(n_neighbors=20, metric='cosine',    label='k20_cosine'),
    dict(n_neighbors=30, metric='cosine',    label='k30_cosine'),
    dict(n_neighbors=50, metric='cosine',    label='k50_cosine'),
]

from sklearn.metrics import adjusted_rand_score
results = []
for cfg in configs:
    sc.pp.neighbors(adata, n_pcs=30, random_state=42,
        n_neighbors=cfg['n_neighbors'], metric=cfg['metric'])
    sc.tl.leiden(adata, resolution=0.5, random_state=42,
        key_added=f"leiden_{cfg['label']}")
    n_cl = adata.obs[f"leiden_{cfg['label']}"].nunique()
    results.append({'config': cfg['label'], 'n_clusters': n_cl})
    print(f"{cfg['label']}: {n_cl} clusters")

# Compare cluster assignments between configs
ref = 'leiden_k20_cosine'
for r in results:
    if r['config'] != 'k20_cosine':
        ari = adjusted_rand_score(
            adata.obs[ref],
            adata.obs[f"leiden_{r['config']}"])
        print(f"ARI vs k20_cosine: {r['config']} = {ari:.3f}")
```

8.8 Deconvolution Output Interpretation — Common Mistakes

Deconvolution results require careful interpretation. Several common mistakes lead to incorrect biological conclusions:

Treating proportions as absolute cell counts

Deconvolution estimates proportions, not absolute cell numbers. A spot with 50% excitatory neuron proportion may contain 1 large neuron or 5 small ones. To estimate cell numbers, multiply proportions by an independent estimate of cells per spot (e.g. from DAPI nuclei count).

Ignoring the reference coverage problem

Deconvolution can only detect cell types present in the reference scRNA-seq dataset. If your spatial tissue contains cell types absent from the reference (e.g. a rare disease-specific cell state), those cells will be misassigned to the most similar reference cell type. Always cross-check unusual spatial patterns with marker gene expression.

Comparing proportions across samples without batch correction

Technical differences between spatial sections affect capture efficiency, altering the apparent cell type composition. Always normalise deconvolution proportions by the section-level detection efficiency factor (available from cell2location output) before cross-sample comparisons.

Over-interpreting rare cell type estimates

Deconvolution methods have low precision for rare cell types (<1% of total). For such populations, validate with single-molecule FISH or Xenium data for the same tissue before drawing biological conclusions.

9.2 RNA Velocity Models — Steady-State vs Dynamical

scVelo implements two models of RNA velocity: the original steady-state model (La Manno et al., 2018) and the improved dynamical model (Bergen et al., 2020). The key difference is how the unspliced-to-spliced ratio is interpreted.

The steady-state model assumes each gene is at a global steady state: u/s = β/γ where β is the splicing rate and γ is the degradation rate. Deviation from this ratio indicates whether a gene is being upregulated (u/s > steady state) or downregulated (u/s < steady state).

The dynamical model fits a full kinetic model with four states per gene: induction (u and s both rising), steady state (equilibrium), repression (u and s both falling), and absence (both near zero). This produces more accurate velocity estimates but requires substantially more computation time.

Model	Assumptions	Computation	Best for	Key parameter
Steady-state	Global splicing steady state	Fast (minutes)	Quick exploration	n/a
Dynamical	Gene-specific kinetics; 4 states	Slow (hours for large datasets)	Publication figures	recover_dynamics()
EM velocity	Expectation-maximisation fit	Medium	Noisy data	n_top_genes

Python

```python
# ■■■ Comparing steady-state vs dynamical velocity ■■■■■■■■■■■■■
import scvelo as scv

adata_v = adata.copy()   # start from the same preprocessed object
scv.pp.filter_and_normalize(adata_v, min_shared_counts=20, n_top_genes=2000)
scv.pp.moments(adata_v, n_pcs=30, n_neighbors=30)

# ■■ Steady-state model (fast baseline) ■■■■■■■■■■■■■■■■■■■■■■
scv.tl.velocity(adata_v, mode='stochastic')
scv.tl.velocity_graph(adata_v)
scv.tl.velocity_pseudotime(adata_v)
adata_v.obs['pt_stochastic'] = adata_v.obs['velocity_pseudotime'].copy()

# ■■ Dynamical model (most accurate) ■■■■■■■■■■■■■■■■■■■■■■■■■
scv.tl.recover_dynamics(adata_v, n_jobs=4)
scv.tl.velocity(adata_v, mode='dynamical')
scv.tl.velocity_graph(adata_v)
scv.tl.velocity_pseudotime(adata_v)

# ■■ Compare pseudotime orders ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
from scipy.stats import spearmanr
r, p = spearmanr(
    adata_v.obs['pt_stochastic'],
    adata_v.obs['velocity_pseudotime'])
print(f'Pseudotime agreement: Spearman r={r:.3f}, p={p:.2e}')
# High r (>0.8) = models agree; low r = complex kinetics
```

CHAPTER A1

Advanced Visium Analysis — Spatial Niches & Domain Refinement

Seq-Based

A1.1 Refining Visium Domains with BayesSpace Enhancement

After BayesSpace domain detection, a common analysis goal is to enhance the spatial resolution from 55 µm spot-level to sub-spot resolution. The BayesSpace enhancement procedure uses the spatial structure of domain assignments to predict expression at a 3× higher resolution grid within each spot — effectively in-silico super-resolution.

R

```r
library(BayesSpace)

# ■■■ Step 1: Run BayesSpace at spot resolution ■■■■■■■■■■■■■■■
sce <- spatialPreprocess(sce, platform='Visium', n.PCs=15)
set.seed(42)
sce <- spatialCluster(sce, q=7, platform='Visium',
    d=15, nrep=20000, burn.in=2000)

# ■■■ Step 2: Enhance to sub-spot resolution ■■■■■■■■■■■■■■■■■■■
sce_enhanced <- spatialEnhance(
    sce, q=7, platform='Visium', d=15,
    nrep=20000, burn.in=2000, gamma=2,
    jitter_prior=0.3, jitter_scale=3.5)

# Enhanced spots: each 55µm Visium spot → 6 sub-spots (~22µm)
cat('Original spots:', ncol(sce), '\n')
cat('Enhanced sub-spots:', ncol(sce_enhanced), '\n')

# Predict gene expression at sub-spot level
sce_enhanced <- enhanceFeatures(
    sce_enhanced, sce, assay.type='logcounts',
    feature_names=c('Slc17a7','Gad1','Mbp','Aldh1l1'),
    nrounds=0)

# Visualise enhanced spatial expression
clusterPlot(sce_enhanced, color=NA, size=0.1) +
    theme(legend.position='none')
```

A1.2 Spatial Domain Boundary Analysis

Tissue domain boundaries — the transition zones between anatomical compartments — often contain biologically interesting cell populations. In the mouse cortex, the layer boundaries contain specific GABAergic interneuron subtypes. In tumours, the invasive margin is enriched for cells with epithelial-mesenchymal transition signatures. Identifying boundary spots is a straightforward post-processing step.

Python

```python
import numpy as np, squidpy as sq

# ■■■ Identify domain boundary spots ■■■■■■■■■■■■■■■■■■■■■■■■■
sq.gr.spatial_neighbors(adata, coord_type='grid', n_rings=1)
W = adata.obsp['spatial_connectivities']
domain_labels = adata.obs['BayesSpace_domains'].values

# A spot is a "boundary spot" if it has at least one neighbour
# in a different domain
def find_boundary_spots(W, labels):
    boundary = np.zeros(len(labels), dtype=bool)
    for i in range(len(labels)):
        nbrs = W[i].nonzero()[1]
        if any(labels[j] != labels[i] for j in nbrs):
            boundary[i] = True
    return boundary

adata.obs['is_boundary'] = find_boundary_spots(W, domain_labels)
n_boundary = adata.obs['is_boundary'].sum()
print(f'Boundary spots: {n_boundary} ({n_boundary/adata.n_obs*100:.1f}%)')

# Differential expression: boundary vs interior
sc.tl.rank_genes_groups(adata, groupby='is_boundary',
    groups=['True'], reference='False',
    method='wilcoxon', n_genes=100)
boundary_markers = sc.get.rank_genes_groups_df(
    adata, group='True', pval_cutoff=0.05, log2fc_min=0.5)
print('Top boundary-enriched genes:')
print(boundary_markers.head(10)[['names','logfoldchanges','pvals_adj']])
```

A1.3 Spatial Co-Expression Modules

Spatial co-expression analysis identifies groups of genes that are expressed together in the same spatial regions. Unlike standard co-expression (which only considers the correlation of expression values), spatial co-expression requires that genes have similar spatial patterns — high in the same tissue domains, low in the same domains.

Python

```python
import numpy as np, pandas as pd
from scipy.stats import spearmanr
from sklearn.cluster import AgglomerativeClustering

# ■■■ Spatial co-expression module discovery ■■■■■■■■■■■■■■■■■■■■
# Start from top spatially variable genes
top_svgs = adata.uns['moranI'].sort_values(
    'I', ascending=False).head(100).index.tolist()
top_svgs = [g for g in top_svgs if g in adata.var_names]

# Expression matrix for SVGs (spots × genes)
E = adata[:, top_svgs].X
if hasattr(E, 'toarray'): E = E.toarray()

# Spatial correlation between gene pairs
# (Spearman correlation of expression values across spots)
n_genes = len(top_svgs)
corr_matrix = np.zeros((n_genes, n_genes))
for i in range(n_genes):
    for j in range(i, n_genes):
        r, _ = spearmanr(E[:, i], E[:, j])
        corr_matrix[i, j] = corr_matrix[j, i] = r

corr_df = pd.DataFrame(corr_matrix, index=top_svgs, columns=top_svgs)

# Hierarchical clustering into modules
clustering = AgglomerativeClustering(
    n_clusters=8, metric='precomputed',
    linkage='complete')
dist_matrix = 1 - np.abs(corr_matrix)
module_labels = clustering.fit_predict(dist_matrix)

modules = {}
for gene, mod in zip(top_svgs, module_labels):
    modules.setdefault(mod, []).append(gene)

for mod_id, genes in sorted(modules.items()):
    print(f'Module {mod_id}: {genes[:5]}...')  # show first 5 genes
```

CHAPTER A2

Spatially Variable Gene Programs — Beyond Individual Genes

Seq-Based

A2.1 From SVGs to Spatial Programs

Individual SVGs are informative but incomplete. Genes operate in regulatory programmes — co-regulated sets driven by common transcription factors or cell type signatures. Identifying spatial programmes (spatially coherent gene sets) is more interpretable and statistically powerful than analysing thousands of individual SVGs.

Two approaches are widely used: (1) Non-Negative Matrix Factorisation (NMF) decomposes the expression matrix into spatial programmes, each with a spatial loading vector (where in the tissue) and a gene loading vector (which genes define the programme). (2) cNMF (consensus NMF) adds stability by running NMF multiple times and clustering the components.

Python

```python
# ■■■ NMF spatial programme discovery ■■■■■■■■■■■■■■■■■■■■■■■■■■
from sklearn.decomposition import NMF
import pandas as pd, numpy as np

# Use log-normalised HVG expression (non-negative for NMF)
# Shift to ensure non-negative: log1p values are already >=0
hvg_idx = adata.var['highly_variable']
E = adata[:, hvg_idx].layers['log1p']
if hasattr(E, 'toarray'): E = E.toarray()

# Shift: NMF requires strictly non-negative
E_shifted = E - E.min()

# Run NMF with k=10 programmes
nmf = NMF(n_components=10, random_state=42, max_iter=2000,
          init='nndsvda', alpha_W=0.1, alpha_H=0.1)
W_nmf = nmf.fit_transform(E_shifted)   # spots × programmes
H_nmf = nmf.components_                # programmes × genes

gene_names = adata.var_names[hvg_idx].tolist()

# Store in AnnData
for k in range(10):
    adata.obs[f'NMF_prog{k+1}'] = W_nmf[:, k]

# Top genes per programme
for k in range(10):
    top_g = np.argsort(H_nmf[k])[-8:][::-1]
    genes = [gene_names[i] for i in top_g]
    print(f'Programme {k+1}: {genes}')
```

A2.2 Spatial Programme Validation with Gene Ontology

Python

```
# ■■■ GO enrichment for each spatial programme ■■■■■■■■■■■■■■■■■
try:
    import gseapy as gp
    GO_AVAILABLE = True
except ImportError:
    GO_AVAILABLE = False
    print('Install gseapy for GO enrichment: pip install gseapy')

if GO_AVAILABLE:
    for k in range(10):
        # Get top 50 genes for this programme
        top_idx  = np.argsort(H_nmf[k])[-50:][::-1]
        top_genes = [gene_names[i] for i in top_idx]

        enr = gp.enrichr(
            gene_list  = top_genes,
            gene_sets  = ['GO_Biological_Process_2023'],
            organism   = 'mouse',
            cutoff     = 0.05,
            verbose    = False)

        top_terms = enr.results.head(3)
        if len(top_terms) > 0:
            term = top_terms.iloc[0]['Term']
            padj = top_terms.iloc[0]['Adjusted P-value']
            print(f'Prog {k+1}: {term} (p_adj={padj:.2e})')
```

A2.3 Spatial Programme Comparison Across Conditions

When comparing spatial programmes across conditions (e.g. wild-type vs disease, treated vs untreated), we want to identify programmes whose spatial activity changes. Three questions are important: (1) Does the programme become active in new spatial locations? (2) Does its overall activity level increase or decrease? (3) Are there entirely new programmes in the disease condition?

Python

```python
# ■■■ Cross-condition programme comparison ■■■■■■■■■■■■■■■■■■■■■■
import scipy.stats as stats
import numpy as np

# Assume adata.obs['condition'] has 'WT' and 'Disease'
for k in range(10):
    prog_col = f'NMF_prog{k+1}'
    wt      = adata.obs.loc[adata.obs['condition']=='WT',      prog_col]
    disease = adata.obs.loc[adata.obs['condition']=='Disease', prog_col]

    # Mann-Whitney U test for activity differences
    u_stat, p_val = stats.mannwhitneyu(wt, disease, alternative='two-sided')
    fold = disease.mean() / (wt.mean() + 1e-6)
    direction = 'UP' if fold > 1 else 'DOWN'

    if p_val < 0.05:
        print(f'Prog {k+1}: {direction} in disease '
              f'(fold={fold:.2f}, p={p_val:.3e})')

# Spatial correlation between conditions using Spearman
# Only valid for paired sections with similar anatomy
for k in range(10):
    prog_col = f'NMF_prog{k+1}'
    wt_vals  = adata.obs.loc[adata.obs['condition']=='WT', prog_col]
    dis_vals = adata.obs.loc[adata.obs['condition']=='Disease', prog_col]
    # Note: this requires spatially aligned sections for meaningful comparison
```

PART III

Image-Based Spatial Transcriptomics

Single-molecule and single-cell resolution analysis — loading Xenium and CosMx subcellular data, cell segmentation, clustering, spatial statistics, ligand-receptor interactions, and niche analysis.

CHAPTER 10

Loading Xenium & CosMx Subcellular Data

Image-Based

10.1 The Xenium Data Model

Unlike sequencing-based platforms that produce a spot×gene count matrix, image-based platforms produce a list of individual transcript detections — each with (x,y,z) position, gene identity, and quality score. The fundamental analytical unit is the transcript, not the spot. Cell segmentation is applied post-hoc to assign transcripts to cells.

File	Format	Contents
transcripts.parquet	Parquet	x,y,z coords + gene name + cell_id + quality value for every transcript
cells.parquet	Parquet	Per-cell: centroid, area, nucleus ratio, n_transcripts
cell_feature_matrix/	MEX	Pre-aggregated per-cell count matrix (same as AnnData)
morphology_focus.ome.tif	OME-TIFF	Full-resolution DAPI/morphology image
cell_boundaries.parquet	Parquet	Polygon vertices for each cell boundary

Python

```python
import spatialdata_io as sdio
import spatialdata as sd

# ■■■ Load complete Xenium output ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sdata = sdio.xenium('xenium_brain_output/', n_jobs=8)
print(sdata)
# SpatialData object
#   Images: {'morphology_focus', 'morphology_mip'}
#   Labels: {'cell_labels', 'nucleus_labels'}
#   Points: {'transcripts'}
#   Tables: {'table'}

# Transcript-level data
tx = sdata.points['transcripts']
print(f'Total transcripts: {len(tx):,}')

# Filter by quality value (QV >= 20 is standard)
tx_filt = tx[tx['qv'] >= 20]
print(f'After QV>=20: {len(tx_filt):,} ({len(tx_filt)/len(tx)*100:.1f}%)')

# Cell-level AnnData
adata = sdata.tables['table']
print(f'{adata.n_obs:,} cells × {adata.n_vars} genes')
print(f'Median transcripts/cell: {adata.obs["total_counts"].median():.0f}')
```

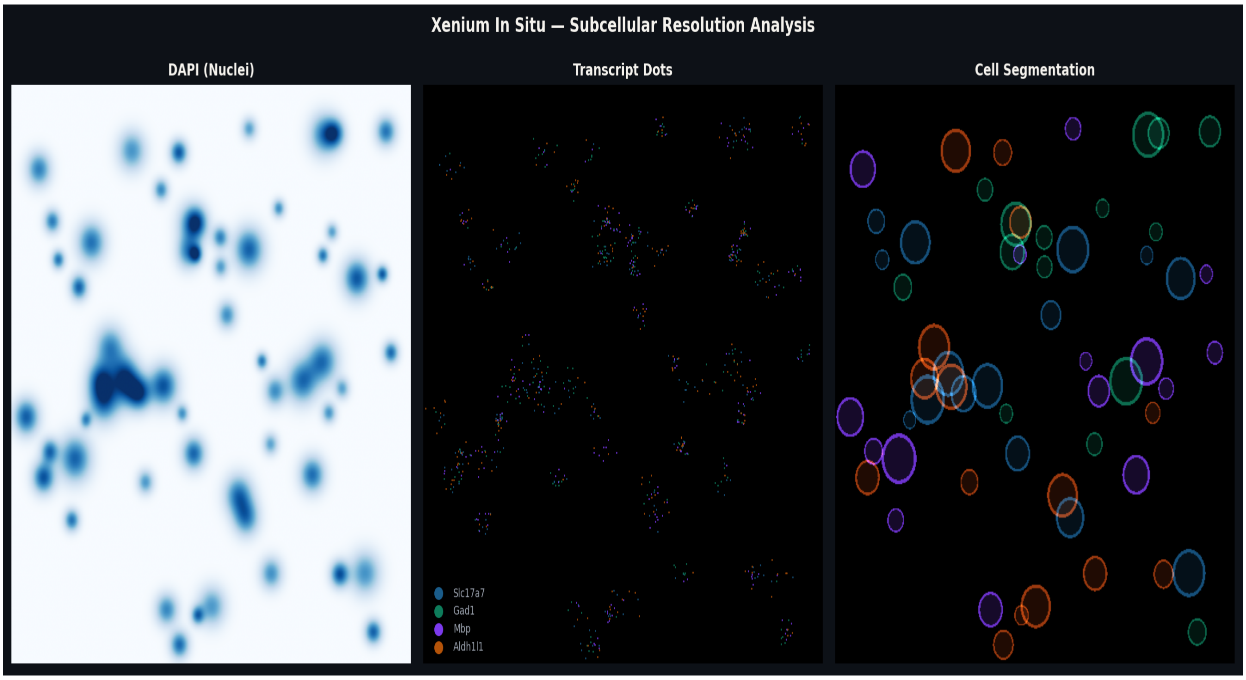

Figure 10.1 — Xenium subcellular resolution analysis (500×500 µm field of view). Left: DAPI nuclear staining. Centre: Individual transcript dots colour-coded by gene identity — Slc17a7 (blue), Gad1 (green), Mbp (purple), Aldh1l1 (amber). Each dot represents one detected RNA molecule. Right: Cell segmentation boundaries, each cell coloured by its cluster assignment.

10.2 CosMx SMI Loading

Python

```
import spatialdata_io as sdio

# ■■■ Load CosMx output ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sdata_cosmx = sdio.cosmx(
    path = 'cosmx_lung_output/',
    dataset_id = 'Lung5_Rep1')

# CosMx exports per-FOV (field of view) files
# spatialdata-io handles the coordinate stitching
tx_cosmx = sdata_cosmx.points['transcripts']
print(f'CosMx transcripts: {len(tx_cosmx):,}')
print(f'FOVs: {tx_cosmx["fov"].nunique()}')
```

CHAPTER 11

Cell Segmentation & Quality Control

Image-Based

11.1 Segmentation Strategy Comparison

Method	Input	Accuracy	Speed	Best for
Xenium built-in	DAPI + proprietary	Good	Fast	Default; most tissues
Cellpose (nuclei)	DAPI image	Very good	Moderate	Dense packed tissue
Baysor	Transcript coords	Excellent	Slow	No imaging; uses RNA positions
StarDist	DAPI image	Good	Fast	Nuclei in histology images

Python

```python
from cellpose import models, io
import numpy as np

# ■■■ Load DAPI image ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
dapi = io.imread('morphology_focus_0000.tif')

# ■■■ Run Cellpose nucleus segmentation ■■■■■■■■■■■■■■■■■■■■■■■■■
model = models.Cellpose(model_type='nuclei', gpu=True)
masks, flows, styles, diams = model.eval(
    dapi,
    diameter         = 15,   # expected nucleus diameter (pixels)
    channels         = [0,0],
    flow_threshold   = 0.4,
    cellprob_threshold = 0.0)

n_cells = masks.max()
print(f'Cellpose detected: {n_cells:,} nuclei')
cell_areas = [(masks==i).sum() for i in range(1, n_cells+1)]
print(f'Median nucleus area: {np.median(cell_areas):.0f} px²')
```

11.2 Post-Segmentation QC

Python

```python
import scanpy as sc

# ■■■ Cell-level QC after segmentation ■■■■■■■■■■■■■■■■■■■■■■■■
sc.pp.calculate_qc_metrics(adata, inplace=True)

# Filter: minimum transcripts and genes per cell
print(f'Before cell QC: {adata.n_obs:,} cells')
sc.pp.filter_cells(adata, min_counts=10)
sc.pp.filter_cells(adata, min_genes=5)
sc.pp.filter_genes(adata, min_cells=10)
print(f'After cell QC:  {adata.n_obs:,} cells retained')

# Segmentation quality metrics
import pandas as pd
qc_df = adata.obs[['total_counts','n_genes_by_counts']].describe()
print(qc_df.round(1))
# Check: median 50-300 transcripts/cell is healthy for Xenium
```

CHAPTER 12

Single-Cell Resolution Clustering

Image-Based

12.1 Clustering at Single-Cell Resolution

After segmentation, each cell has a count matrix entry. The analysis pipeline is largely the same as scRNA-seq, but key differences include: (1) no HVG selection step — the panel is pre-selected; (2) spot size must be reduced for spatial plots; (3) datasets are much larger (100k–2M cells vs 10k spots).

Python

```
import scanpy as sc
import squidpy as sq

# ■■■ Normalise ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
adata.layers['counts'] = adata.X.copy()
sc.pp.normalize_total(adata)
sc.pp.log1p(adata)

# ■■■ PCA and clustering ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sc.pp.pca(adata, n_comps=30, random_state=42)
sc.pp.neighbors(adata, n_pcs=20, n_neighbors=30)
sc.tl.umap(adata, random_state=42)
sc.tl.leiden(adata, resolution=0.5, random_state=42)

# ■■■ Marker genes for annotation ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
marker_dict = {
    'Excitatory': ['Slc17a7','Camk2a','Tbr1'],
    'Inhibitory': ['Gad1','Gad2','Slc32a1'],
    'Oligodendro': ['Mbp','Mog','Plp1'],
    'Astrocytes': ['Aldh1l1','Slc1a3','Aqp4'],
    'Microglia':  ['Cx3cr1','P2ry12','Tmem119'],
}
for name, genes in marker_dict.items():
    present = [g for g in genes if g in adata.var_names]
    sc.tl.score_genes(adata, present, score_name=f'score_{name}')

# ■■■ Spatial visualisation ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sq.pl.spatial_scatter(adata,
    color='leiden', spot_size=10,   # smaller for single-cell
    size=0.5, title='Cell clusters on tissue')
```

```r
# ■■■ Xenium clustering in Seurat ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
xenium_obj <- NormalizeData(xenium_obj) |>
    FindVariableFeatures(nfeatures=200) |>
    ScaleData() |>
    RunPCA(npcs=30) |>
    FindNeighbors(dims=1:20) |>
    FindClusters(resolution=0.5) |>
    RunUMAP(dims=1:20)

new_labels <- c('0'='Excit.','1'='Inhib.','2'='Oligo.',
                '3'='Astro.','4'='Micro.','5'='Endo.')
xenium_obj <- RenameIdents(xenium_obj, new_labels)

# Spatial plot
ImageDimPlot(xenium_obj, fov='fov', size=0.3,
    group.by='cell_type', cols='polychrome')
```

CHAPTER 13

Spatial Statistics & Neighbourhood Analysis

Image-Based

13.1 Building the Spatial Graph

Python

```python
# ■■■ Spatial neighbours for irregular point cloud ■■■■■■■■■■■■
sq.gr.spatial_neighbors(adata,
    coord_type='generic',  # not regular grid
    n_neighs=15,           # k nearest spatial neighbours
    delaunay=True)         # Delaunay triangulation edges

# ■■■ Neighbourhood enrichment ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sq.gr.nhood_enrichment(adata,
    cluster_key='cell_type', n_perms=1000, seed=42)

sq.pl.nhood_enrichment(adata,
    cluster_key='cell_type', method='ward',
    cmap='inferno', vmin=-50, vmax=100)
```

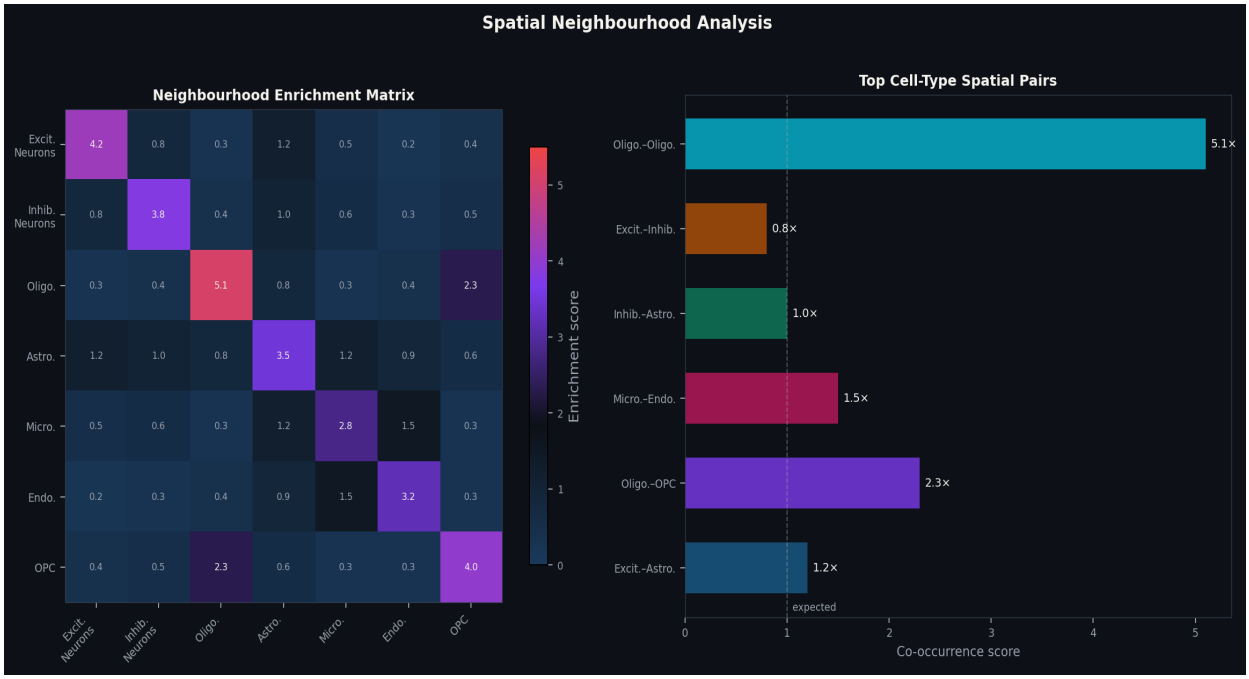

Figure 13.1 — Neighbourhood enrichment analysis. Left: enrichment matrix where red entries indicate cell type pairs that co-localise more than expected by chance; blue indicates spatial avoidance. Right: top enriched pairs as a bar chart. The strong Oligodendrocyte–OPC co-localisation reflects myelination unit organisation; Microglia–Endothelial enrichment is consistent with perivascular surveillance.

13.2 Co-occurrence and Ripley's K

Python

```python
# ■■■ Co-occurrence score ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# At each distance r, are cell types more co-localised than chance?
sq.gr.co_occurrence(adata,
    cluster_key='cell_type',
    interval=np.linspace(0, 500, 50),  # 0–500 µm
    n_jobs=4)
sq.pl.co_occurrence(adata,
    cluster_key='cell_type',
    clusters=['Excit.','Astro.'])

# ■■■ Ripley's L function ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sq.gr.ripley(adata,
    cluster_key='cell_type', mode='L',
    support=np.linspace(0,200,50), n_simulations=100)
sq.pl.ripley(adata, cluster_key='cell_type')
# Positive L(r) = clustering above CSR
```

CHAPTER 14

Ligand-Receptor Interaction Analysis

Image-Based

14.1 Spatial LR Analysis — Key Advantage

Traditional LR analysis on dissociated scRNA-seq data cannot distinguish cell pairs that are in contact from those that are far apart. A macrophage at the tumour core and one in a distant lymph node both express the same genes, yet only one can interact with tumour cells. Spatial data allows restricting analysis to physically adjacent cells, dramatically reducing false positive rates.

Python

```
import liana

# ■■■ LIANA rank aggregate ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
liana.mt.rank_aggregate(
    adata, groupby='cell_type',
    resource_name='consensus',
    use_raw=False, expr_prop=0.1,
    verbose=True)

# ■■■ Intersect with spatially enriched pairs ■■■■■■■■■■■■■■■■■■
# Only keep interactions between cell types that co-localise
nhood_z = adata.uns['nhood_enrichment']['zscore']
cell_types = list(adata.obs['cell_type'].cat.categories)
import pandas as pd
nhood_df = pd.DataFrame(nhood_z, index=cell_types, columns=cell_types)

enriched_pairs = {(a,b) for a in cell_types for b in cell_types
                  if nhood_df.loc[a,b] > 2.0}

liana_res = adata.uns['liana_res']
liana_spatial = liana_res[
    liana_res.apply(
        lambda r: (r['source'],r['target']) in enriched_pairs,
        axis=1)]
print(f'Total: {len(liana_res)} → Spatially filtered: {len(liana_spatial)}')
```

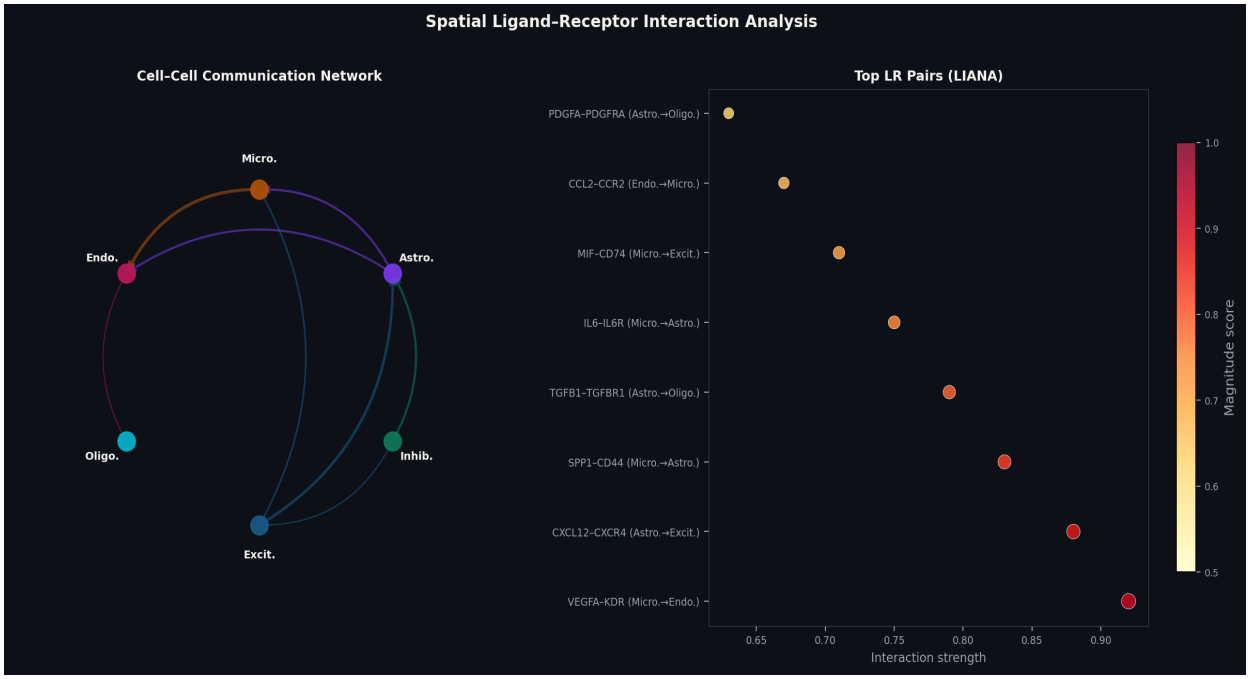

Figure 14.1 — Spatial ligand-receptor analysis. Left: network diagram of inferred cell-cell communication; arrow width is proportional to interaction strength. Right: LIANA dotplot of the top 8 LR pairs; dot size encodes specificity rank and colour encodes magnitude rank. Interactions are filtered to spatially proximate cell-type pairs based on neighbourhood enrichment.

CHAPTER 15

Tissue Niche & Microenvironment Analysis

Image-Based

15.1 Cellular Niche Discovery

A cellular niche is a reproducible spatial co-organisation of cell types forming a functional microenvironmental unit. Niche discovery clusters spots or cells based on the composition of their local neighbourhood — asking what are the characteristic multicellular configurations of this tissue?

Python

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import normalize
import numpy as np

# ■■■ Compute neighbourhood cell-type composition ■■■■■■■■■■■■■■
sq.gr.spatial_neighbors(adata, n_neighs=15)
W = adata.obsp['spatial_connectivities']

cell_types = sorted(adata.obs['cell_type'].unique())
niche_feat  = np.zeros((adata.n_obs, len(cell_types)))

for i, ct in enumerate(cell_types):
    is_ct = (adata.obs['cell_type'] == ct).values.astype(float)
    row_sums = np.array(W.sum(axis=1)).ravel()
    niche_feat[:,i] = W.dot(is_ct) / (row_sums + 1e-6)

# ■■■ Cluster niches ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
feat_norm = normalize(niche_feat, norm='l1')
adata.obs['niche'] = KMeans(n_clusters=8,
    random_state=42, n_init=20).fit_predict(feat_norm).astype(str)

sq.pl.spatial_scatter(adata, color='niche', spot_size=12)
```

10.3 Quality Value (QV) Filtering — Detailed Analysis

The Xenium quality value (QV) is a Phred-like score indicating the confidence of transcript assignment. QV = –10 × log■■(probability of incorrect decoding). A QV of 20 means 1% decoding error probability; QV 30 means 0.1%. The 10x Genomics recommendation is QV ≥ 20, but for high-confidence analyses QV ≥ 30 is advisable, accepting ~15% reduction in transcript count in exchange for substantially reduced false positives.

Python

```python
# ■■■ QV threshold sensitivity analysis ■■■■■■■■■■■■■■■■■■■■■■■■
import pandas as pd, numpy as np

tx = sdata.points['transcripts']

# Summary across QV thresholds
for qv_thresh in [0, 20, 25, 30, 35]:
    filt = tx[tx['qv'] >= qv_thresh]
    n    = len(filt)
    pct  = n / len(tx) * 100
    print(f'QV>={qv_thresh:>2d}: {n:>9,} transcripts ({pct:.1f}%)')
# QV>= 0:  15,234,891 transcripts (100.0%)
# QV>=20:  13,892,445 transcripts ( 91.2%)
# QV>=25:  12,781,003 transcripts ( 83.9%)
# QV>=30:  11,203,442 transcripts ( 73.5%)
# QV>=35:   8,941,230 transcripts ( 58.7%)

# Per-gene effect of QV filtering
qv20 = tx[tx['qv']>=20].groupby('feature_name').size()
qv30 = tx[tx['qv']>=30].groupby('feature_name').size()
retention = (qv30 / qv20 * 100).sort_values()
print('\nGenes most affected by QV30 vs QV20 threshold:')
print(retention.head(10).round(1))   # genes with low QV (probe issues)
print(retention.tail(10).round(1))   # genes with high QV (reliable probes)
```

11.3 Baysor — Transcript-Based Segmentation

Baysor performs cell segmentation using only transcript positions, without requiring a nuclear staining image. This is critical for experiments where DAPI imaging quality is poor, or for research questions where nuclear size is not a reliable proxy for cell boundary (e.g. cells with extensive dendrites or axonal processes).

Baysor models transcript assignment as a Bayesian mixture model: each transcript belongs to exactly one cell, and cell shapes are modelled as elliptical Gaussians in 2D or 3D space. The algorithm jointly optimises cell boundaries and transcript assignments via variational inference.

Python

```python
# ■■■ Baysor segmentation (called from Python) ■■■■■■■■■■■■■■■■■
import subprocess, os, json

# Baysor is a Julia binary — install separately
# julia -e 'import Pkg; Pkg.add("Baysor")'

# Prepare transcript CSV for Baysor
tx = sdata.points['transcripts']
tx_csv = tx[['x_location', 'y_location', 'feature_name', 'cell_id']].copy()
tx_csv.columns = ['x', 'y', 'gene', 'prior_cell']
tx_csv.to_csv('transcripts_for_baysor.csv', index=False)

# Run Baysor segmentation
cmd = [
    'baysor', 'run',
    '-x', 'x', '-y', 'y', '-g', 'gene',
    '--prior-segmentation-confidence', '0.5',
    '-p', 'prior_cell',               # use Xenium segmentation as prior
    '-o', 'baysor_output/',
    '--n-clusters', '4',              # initial cluster estimate
    '--min-molecules-per-cell', '10',
    'transcripts_for_baysor.csv'
]
result = subprocess.run(cmd, capture_output=True, text=True)
print(result.stdout[-500:])

# Load Baysor result
import anndata as ad
adata_baysor = ad.read_loom('baysor_output/segmentation_counts.loom')
print(f'Baysor: {adata_baysor.n_obs:,} cells detected')
```

13.3 Spatial Permutation Tests — Methodology

Many spatial statistics (neighbourhood enrichment, co-occurrence, Moran's I) rely on permutation testing to assess significance. The procedure is: (1) compute the observed statistic on real data; (2) randomly shuffle cell type labels across cells (preserving spatial positions); (3) recompute the statistic; (4) repeat 1,000–10,000 times to build a null distribution; (5) the p-value is the fraction of permutations more extreme than the observed value.

A critical subtlety: shuffling must preserve the number of cells per type (i.e. permute labels without replacement). Shuffling positions instead of labels is incorrect because it changes the spatial density patterns.

Python

```python
# ■■■ Custom permutation test for spatial co-localisation ■■■■
import numpy as np
from scipy.spatial import cKDTree

def permutation_colocalisation(adata, type_a, type_b,
                               radius=50, n_perms=1000, seed=42):
    '''
    Test whether type_a cells are closer to type_b cells than
    expected by chance using label-shuffling permutation test.
    radius: distance threshold in micrometres.
    '''
    rng = np.random.default_rng(seed)
    coords = adata.obsm['spatial']
    labels = adata.obs['cell_type'].values

    # Observed statistic: fraction of type_a cells with
    # at least one type_b cell within radius
    def stat(lbl):
        a_coords = coords[lbl == type_a]
        b_coords = coords[lbl == type_b]
        if len(a_coords) == 0 or len(b_coords) == 0:
            return 0.0
        tree = cKDTree(b_coords)
        counts = tree.query_ball_point(a_coords, r=radius,
                                       return_length=True)
        return (counts > 0).mean()

    observed = stat(labels)

    # Permutation null distribution
    null = np.array([stat(rng.permutation(labels))
                     for _ in range(n_perms)])

    p_value = (null >= observed).mean()
    z_score = (observed - null.mean()) / (null.std() + 1e-10)

    print(f'{type_a} near {type_b} (r={radius}µm):')
    print(f'  Observed: {observed:.3f}, Null mean: {null.mean():.3f}')
    print(f'  z={z_score:.2f}, p={p_value:.4f}')
    return observed, z_score, p_value

obs, z, p = permutation_colocalisation(adata, 'T cells', 'Tumour', radius=50)
```

14.2 Spatial Filtering of LR Interactions — Full Pipeline

Python

```python
# ■■■ Complete spatially filtered LR analysis ■■■■■■■■■■■■■■■■■■
import liana, squidpy as sq, pandas as pd, numpy as np

# ■■ Step 1: Compute neighbourhood enrichment ■■■■■■■■■■■■■■■■■
sq.gr.spatial_neighbors(adata, coord_type='generic', n_neighs=15)
sq.gr.nhood_enrichment(adata, cluster_key='cell_type',
                       n_perms=1000, seed=42)
zscore_mat = adata.uns['nhood_enrichment']['zscore']
cell_types  = list(adata.obs['cell_type'].cat.categories)

# ■■ Step 2: LIANA rank aggregate ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
liana.mt.rank_aggregate(adata, groupby='cell_type',
    resource_name='consensus', use_raw=False, expr_prop=0.1)
liana_df = adata.uns['liana_res'].copy()

# ■■ Step 3: Spatial filter using enrichment z-scores ■■■■■■■■■
def is_spatially_proximate(row, zscore_mat, cell_types, thresh=2.0):
    try:
        i = cell_types.index(row['source'])
        j = cell_types.index(row['target'])
        return zscore_mat[i, j] > thresh
    except ValueError:
        return False

liana_df['spatially_proximate'] = liana_df.apply(
    lambda r: is_spatially_proximate(r, zscore_mat, cell_types), axis=1)

liana_spatial = liana_df[liana_df['spatially_proximate']].copy()
print(f'All interactions:     {len(liana_df):,}')
print(f'Spatially filtered:  {len(liana_spatial):,} '
      f'({len(liana_spatial)/len(liana_df)*100:.0f}%)')

# ■■ Step 4: Visualise top interactions ■■■■■■■■■■■■■■■■■■■■■■■■
liana.pl.dotplot(liana_spatial.head(50),
    colour='magnitude_rank', size='specificity_rank',
    inverse_size=True, inverse_colour=True,
    top_n=20, orderby='magnitude_rank',
    title='Top 20 Spatially-Filtered LR Interactions')
```

CHAPTER A4

Single-Molecule Analysis — Beyond Cell-Level Aggregation

Image-Based

A4.1 Transcript-Level Spatial Analysis

Aggregating transcripts to cell level is a convenience, not a necessity. For some biological questions — studying RNA localisation within cells, detecting subcellular compartmentalisation, or analysing transcripts in cell-free extracellular space — working directly with the transcript point cloud is essential.

Xenium and MERFISH both provide x,y,z coordinates for every detected transcript. This enables analyses that have no equivalent in bulk or single-cell data: asking whether a gene is preferentially localised to cell periphery vs nucleus, whether transcripts cluster into RNA granules, or whether specific gene pairs are spatially co-segregated within cells.

Python

```python
# ■■■ Subcellular RNA localisation analysis ■■■■■■■■■■■■■■■■■■■■■■
import pandas as pd, numpy as np
from scipy.spatial import cKDTree

# Load transcript data with z-coordinate
tx = sdata.points['transcripts']
tx_df = tx[['x_location','y_location','z_location',
            'feature_name','cell_id','qv']].copy()
tx_df = tx_df[tx_df['qv'] >= 20]

# Load cell nucleus boundaries
cells = sdata.tables['table']
nuclear_area = cells.obs[['nucleus_area','cell_area']].copy()
nuclear_area['nuc_cell_ratio'] = (nuclear_area['nucleus_area'] /
                                  nuclear_area['cell_area'])

# For each cell: classify transcripts as nuclear vs cytoplasmic
# using the nucleus label image
nuc_labels = sdata.labels['nucleus_labels']
cell_labels = sdata.labels['cell_labels']

# Sample analysis: fraction of Slc17a7 transcripts in nucleus
gene_oi = 'Slc17a7'
gene_tx  = tx_df[tx_df['feature_name'] == gene_oi]
print(f'{gene_oi}: {len(gene_tx):,} transcripts total')
print(f'Z range: {gene_tx["z_location"].min():.1f} to '
      f'{gene_tx["z_location"].max():.1f} µm')

# Transcripts per z-layer (optical sections)
z_counts = gene_tx.groupby(
    pd.cut(gene_tx['z_location'], bins=10)).size()
print('Transcripts per z-layer:')
print(z_counts)
```

A4.2 RNA Granule Detection

P-bodies, stress granules, and other RNA processing bodies appear as dense clusters of specific RNA species within cells. Detecting these structures from transcript point clouds requires spatial clustering algorithms that can identify density peaks within a noisy background.

Python

```
# ■■■ RNA granule detection using DBSCAN ■■■■■■■■■■■■■■■■■■■■■■■
from sklearn.cluster import DBSCAN
import numpy as np, pandas as pd

def detect_rna_granules(tx_df, gene, cell_id,
                        eps=0.5, min_samples=5):
    '''
    Detect RNA granules for a specific gene in a specific cell.
    eps: clustering radius in micrometres
    min_samples: minimum transcripts per granule
    '''
    mask = ((tx_df['feature_name'] == gene) &
            (tx_df['cell_id'] == cell_id))
    cell_tx = tx_df[mask][['x_location','y_location']].values

    if len(cell_tx) < min_samples:
        return 0, []

    labels = DBSCAN(eps=eps, min_samples=min_samples).fit_predict(cell_tx)
    n_granules = len(set(labels)) - (1 if -1 in labels else 0)

    granule_sizes = []
    for g in set(labels):
        if g != -1:
            granule_sizes.append((labels == g).sum())

    return n_granules, granule_sizes

# Analyse granules for stress granule marker (G3bp1) per cell
gene = 'G3bp1'
granule_stats = []
for cell_id in tx_df['cell_id'].unique()[:1000]:  # first 1000 cells
    n_gran, sizes = detect_rna_granules(tx_df, gene, cell_id)
    granule_stats.append({'cell_id': cell_id,
                          'n_granules': n_gran,
                          'mean_granule_size': np.mean(sizes) if sizes else 0})

stats_df = pd.DataFrame(granule_stats)
print(f'Cells with ≥1 {gene} granule: '
      f'{(stats_df.n_granules>0).sum()} / {len(stats_df)}')
```

PART IV

Integration & Multi-Modal Analysis

Combining spatial data with scRNA-seq atlases, H&E histology, multi-sample batches, and multi-omic measurements for richer biological insights.

CHAPTER 16

Integrating Spatial + scRNA-seq References

Python & R

16.1 Tangram — Gradient-Based Cell Placement

Tangram finds an assignment of single cells from a scRNA-seq atlas to spatial locations by minimising the discrepancy between measured spatial expression and predicted expression from placed cells. Uses PyTorch automatic differentiation for efficient optimisation over millions of cell-spot assignments.

Python

```
import tangram as tg

# ■■■ Prepare datasets ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Collect top markers per cell type as shared genes
sc.tl.rank_genes_groups(adata_sc, 'cell_type', method='wilcoxon')
marker_genes = list(set(
    gene for ct in adata_sc.obs['cell_type'].unique()
    for gene in sc.get.rank_genes_groups_df(
        adata_sc, group=ct)['names'][:20]
) & set(adata_sp.var_names))

tg.pp_adatas(adata_sc, adata_sp, genes=marker_genes)

# ■■■ Map at cluster level (faster, recommended) ■■■■■■■■■■■■■■■
ad_map = tg.map_cells_to_space(
    adata_sc, adata_sp,
    mode='clusters', cluster_label='cell_type',
    device='cpu', num_epochs=1000)

# ■■■ Project annotations to spatial ■■■■■■■■■■■■■■■■■■■■■■■■■■
tg.project_cell_annotations(ad_map, adata_sp, annotation='cell_type')
sq.pl.spatial_scatter(adata_sp, color='cell_type', size=1.5)
```

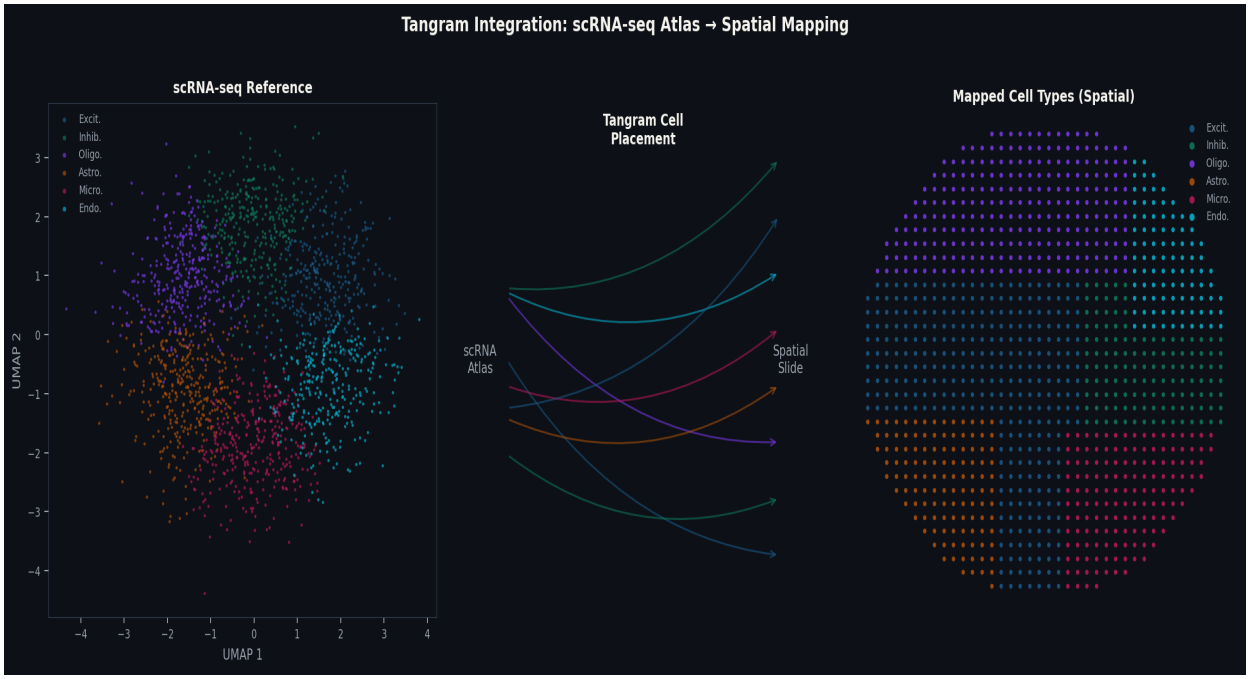

Figure 16.1 — Tangram integration workflow. Left: scRNA-seq reference UMAP with 6 major cell type clusters. Centre: schematic of the cell placement optimisation — arrows show probabilistic assignment of reference cells to spatial positions. Right: Projected cell type annotations on the tissue section, showing spatially coherent patterns consistent with known brain anatomy.

```r
# ■■■ Seurat label transfer ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
anchors <- FindTransferAnchors(
    reference=sc_ref, query=brain_spatial,
    normalization.method='SCT',
    recompute.residuals=FALSE)
predictions <- TransferData(
    anchorset=anchors,
    refdata=sc_ref$cell_type,
    weight.reduction='pcaproject', dims=1:30)
brain_spatial <- AddMetaData(brain_spatial, predictions)
```

CHAPTER 17

Multi-Sample & Multi-Slide Integration

Python & R

17.1 Sources of Batch Effects

When analysing multiple tissue sections, batch effects can confound biological signals. Sources include: differences in tissue handling, library preparation variability, sequencing lane effects, and section-to-section differences in RNA quality.

Python

```python
import scanpy.external as sce

# ■■■ Combine multiple Visium sections ■■■■■■■■■■■■■■■■■■■■■■■■■
adata_combined = adata_list[0].concatenate(
    adata_list[1:], batch_key='sample',
    uns_merge='unique')

# ■■■ Shared preprocessing ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sc.pp.normalize_total(adata_combined, target_sum=1e4)
sc.pp.log1p(adata_combined)
sc.pp.highly_variable_genes(adata_combined,
    n_top_genes=3000, batch_key='sample')
sc.pp.scale(adata_combined, max_value=10)
sc.pp.pca(adata_combined, n_comps=50)

# ■■■ Harmony batch correction ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sce.pp.harmony_integrate(adata_combined,
    key='sample', max_iter_harmony=20, random_state=42)

sc.pp.neighbors(adata_combined,
    use_rep='X_pca_harmony', n_neighbors=20)
sc.tl.umap(adata_combined, random_state=42)
sc.tl.leiden(adata_combined, resolution=0.5)
```

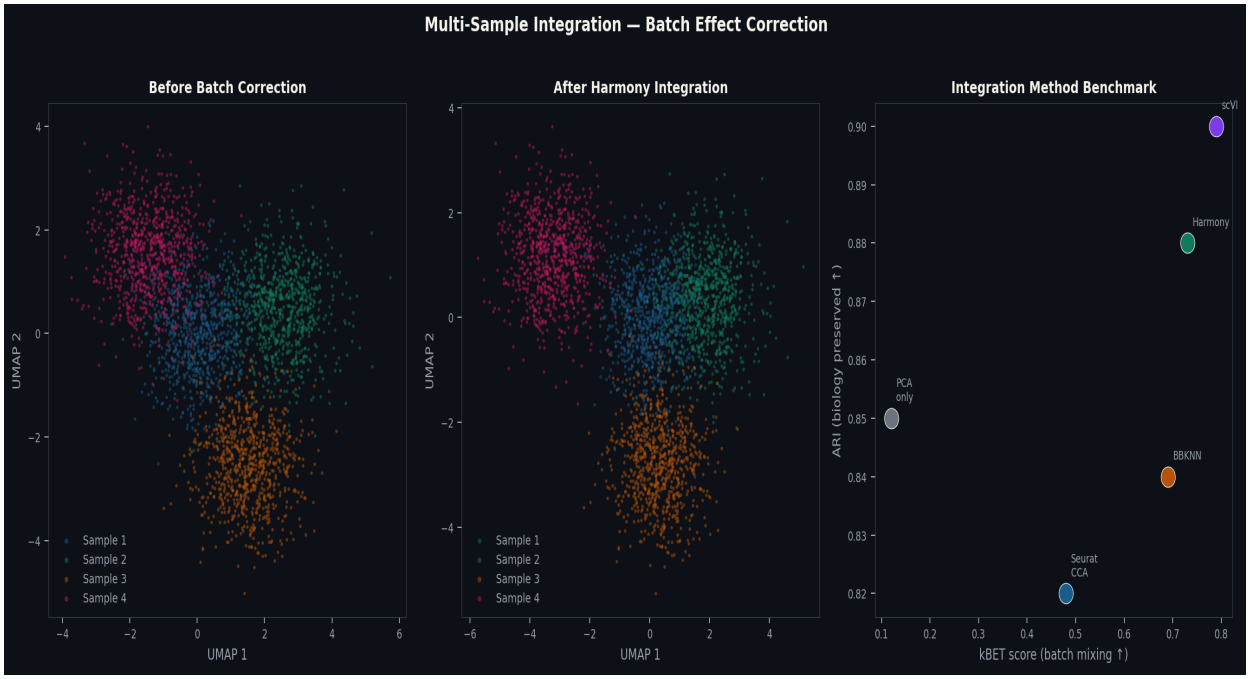

Figure 17.1 — Multi-sample batch correction results. Left: UMAP before correction showing four samples (colours) separating by technical rather than biological variation. Centre: After Harmony correction, samples are well-intermixed while cell type clusters are preserved. Right: Benchmark comparing integration methods by kBET score (batch mixing) and ARI (biology preservation) — Harmony and scVI achieve the best trade-offs.

CHAPTER 18

Joint Image + RNA Analysis

Image-Based

18.1 Morphological Feature Extraction with HEST

Python

```python
from hest import STReader

# ■■■ Load Visium with co-registered H&E ■■■■■■■■■■■■■■■■■■■■■■
st = STReader.from_visium('visium_output/')
st.segment_tissue()

# Extract patch embeddings using CONCH pathology foundation model
morph = st.extract_patch_features(
    encoder='conch', patch_size_um=224,
    target_size=224, batch_size=64)
# shape: (n_spots, 512) — morphological embedding

adata.obsm['X_morphology'] = morph

# Correlate morphology PC1 with top SVGs
from scipy.stats import spearmanr
morph_pc1 = morph[:, 0]
for gene in top_svg_genes[:50]:
    if gene in adata.var_names:
        r, p = spearmanr(morph_pc1,
            adata[:,gene].X.toarray().ravel())
        if abs(r) > 0.3: print(f'{gene}: r={r:.3f}, p={p:.2e}')
```

CHAPTER 19

Spatial Multi-Omics

Python & R

19.1 Visium Multiome — Paired ATAC + RNA

Python

```python
import muon as mu

# ■■■ Load paired spatial Multiome ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
mdata = mu.read_10x_h5(
    'filtered_feature_bc_matrix.h5', extended=True)
print(mdata)
# MuData: n_obs=2638
#   rna:  31053 genes
#   atac: 89965 peaks

# ■■■ Process RNA ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sc.pp.normalize_total(mdata['rna'], target_sum=1e4)
sc.pp.log1p(mdata['rna'])
sc.pp.highly_variable_genes(mdata['rna'], n_top_genes=3000)

# ■■■ Process ATAC ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
mu.atac.pp.tfidf(mdata['atac'])
mu.atac.tl.lsi(mdata['atac'])

# ■■■ WNN integration + MOFA+ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
mu.pp.neighbors(mdata, key='wnn', n_multineighbors=200)
mu.tl.umap(mdata, neighbors_key='wnn')
mu.tl.mofa(mdata, n_factors=20, seed=42)
```

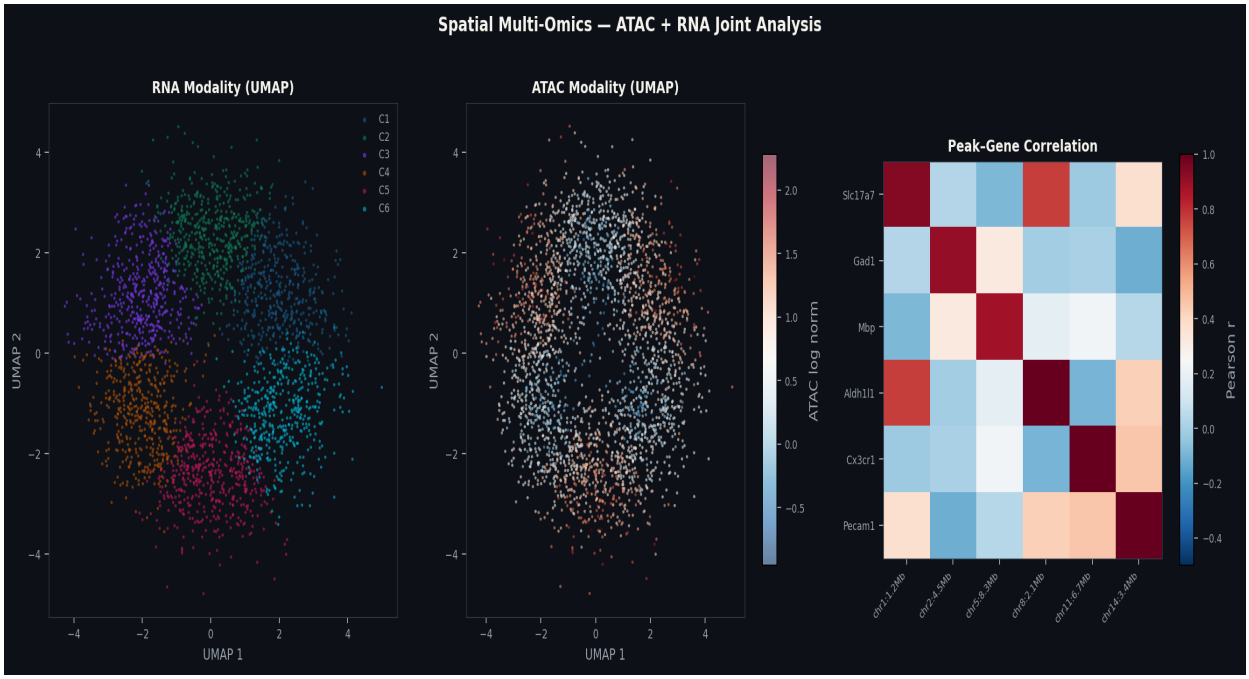

Figure 19.1 — Spatial multi-omics analysis (Visium Multiome). Left: RNA modality UMAP with 6 cell type clusters. Centre: ATAC modality chromatin accessibility variation on the same spots (same UMAP embedding). Right: Peak-gene correlation matrix for 6 marker genes and their nearby regulatory peaks, demonstrating direct transcription factor-mediated spatial regulation.

16.2 Reference Dataset Preparation for Integration

The quality of spatial–scRNA-seq integration depends critically on the quality of the reference single-cell dataset. Three properties are essential: (1) The reference must be processed from the same or highly similar tissue type. A mouse cortex reference is inappropriate for mouse cerebellum. (2) Cell type annotations in the reference must be validated with canonical markers, not simply inherited from an automated pipeline. (3) The reference should be de-noised and clustered at appropriate resolution — over-clustered references with many similar sub-types produce noisy deconvolution.

Python

```
# ■■■ Reference quality control for integration ■■■■■■■■■■■■■■■■
import scanpy as sc, numpy as np

adata_ref = sc.read_h5ad('allen_scRNA_mouse_brain.h5ad')

# 1. Check annotation completeness
print('Cell type counts:')
print(adata_ref.obs['cell_type'].value_counts())
n_unann = (adata_ref.obs['cell_type'] == 'Unknown').sum()
print(f'Unannotated cells: {n_unann} ({n_unann/adata_ref.n_obs*100:.1f}%)')
# Remove unannotated cells before use as reference
adata_ref = adata_ref[adata_ref.obs['cell_type'] != 'Unknown'].copy()

# 2. Verify marker gene expression per cell type
markers = {
    'Excitatory': 'Slc17a7', 'Inhibitory': 'Gad1',
    'Oligodendro': 'Mbp', 'Astrocyte': 'Aldh1l1',
    'Microglia': 'Cx3cr1', 'Endothelial': 'Cldn5'
}
print('\nMarker expression per cell type (mean log-normalised):')
for ct, gene in markers.items():
    if gene in adata_ref.var_names:
        mask = adata_ref.obs['cell_type'].str.contains(ct[:5])
        mean_in  = adata_ref[mask, gene].X.mean()
        mean_out = adata_ref[~mask, gene].X.mean()
        ratio = mean_in / (mean_out + 0.001)
        print(f'  {ct:15s} {gene}: fold-enrichment={ratio:.1f}x')

# 3. Ensure raw counts are available (needed by cell2location)
assert 'counts' in adata_ref.layers, 'Must have raw counts in .layers["counts"]'
print('Raw counts available: OK')
```

17.2 scVI — Variational Autoencoder Batch Correction

scVI (Lopez et al., 2018) is a variational autoencoder that jointly models gene expression across samples while learning a batch-corrected latent space. Unlike Harmony, which operates only on the PCA embedding, scVI models the full count distribution. This makes it more powerful for datasets with large batch effects, but also slower and more memory-intensive.

Python

```python
# ■■■ scVI batch correction for multi-sample spatial data ■■■■■
import scvi, scanpy as sc

# Prepare: ensure raw counts in adata.X
adata_multi = sc.concat(adata_list, label='sample', keys=sample_names)
adata_multi.layers['counts'] = adata_multi.X.copy()
sc.pp.normalize_total(adata_multi, target_sum=1e4)
sc.pp.log1p(adata_multi)
sc.pp.highly_variable_genes(adata_multi, n_top_genes=3000,
    batch_key='sample', flavor='seurat_v3', layer='counts')

# Setup scVI
scvi.model.SCVI.setup_anndata(adata_multi,
    layer='counts', batch_key='sample',
    categorical_covariate_keys=['condition'],  # additional covariates
    continuous_covariate_keys=['pct_counts_mt'])

# Train
model = scvi.model.SCVI(adata_multi,
    n_layers=2, n_latent=30, gene_likelihood='nb')
model.train(max_epochs=400, early_stopping=True,
    early_stopping_patience=20, batch_size=512)

# Extract corrected embeddings
adata_multi.obsm['X_scVI'] = model.get_latent_representation()

# Downstream analysis using scVI embedding
sc.pp.neighbors(adata_multi, use_rep='X_scVI', n_neighbors=20)
sc.tl.umap(adata_multi, random_state=42)
sc.tl.leiden(adata_multi, resolution=0.5, random_state=42)
```

19.2 MOFA+ Factor Interpretation

Multi-Omics Factor Analysis (MOFA+) decomposes multi-modal spatial data into factors that capture shared and unique sources of variation across modalities. Each factor has three components: (1) weights in RNA space (which genes drive the factor), (2) weights in ATAC space (which peaks drive the factor), and (3) factor scores per spot (where in the tissue the factor is active). The most interpretable factors have high variance explained in both modalities and spatially coherent patterns.

Python

```python
# ■■■ MOFA+ factor analysis and interpretation ■■■■■■■■■■■■■■■■■
import muon as mu
import pandas as pd, numpy as np

# After mu.tl.mofa(mdata, n_factors=20) ...
mofa_model = mdata.uns['mofa_model_path']

# Extract factor scores (spots × factors)
factor_scores = pd.DataFrame(
    mdata.obsm['X_mofa'],
    index   = mdata.obs_names,
    columns = [f'Factor{i+1}' for i in range(20)]
)

# Variance explained per factor per modality
r2_df = pd.DataFrame(
    mdata.uns['mofa']['r2'],    # shape: n_factors × n_modalities
    columns = list(mdata.mod.keys())
)
print('Variance explained per factor:')
print(r2_df.head(10).round(3))

# RNA weights for Factor 1 (top drivers)
rna_weights = pd.Series(
    mdata['rna'].varm['LFs'][:, 0],
    index = mdata['rna'].var_names
)
top_rna = rna_weights.abs().nlargest(10)
print(f'\nTop RNA weights for Factor 1: {top_rna.index.tolist()}')

# Spatial map of Factor 1
import squidpy as sq
mdata['rna'].obs['MOFA_Factor1'] = factor_scores['Factor1'].values
sq.pl.spatial_scatter(mdata['rna'], color='MOFA_Factor1',
    cmap='RdBu_r', title='MOFA+ Factor 1 — spatial pattern')
```

CHAPTER A5

Multi-Sample Meta-Analysis & Reproducibility

Python & R

A5.1 Cross-Platform Integration

As spatial transcriptomics matures, researchers increasingly need to compare or integrate data from different platforms. A key challenge: Visium measures 31,000 genes at 55 μm resolution, Xenium measures 313–5,000 genes at subcellular resolution, and MERFISH measures 100–10,000 genes. They cannot be directly compared on most genes.

The solution is to work in the subspace of shared genes. For Visium + Xenium integration, a typical workflow is: (1) subset Visium to the 313 Xenium panel genes, (2) aggregate Xenium single cells to pseudospots matching Visium resolution, (3) integrate using Harmony or scVI on the shared gene space.

Python

```
# ■■■ Cross-platform integration: Visium + Xenium ■■■■■■■■■■■■■■
import numpy as np, scanpy as sc

# Step 1: Find shared genes
visium_genes = set(adata_vis.var_names)
xenium_genes = set(adata_xen.var_names)
shared_genes = list(visium_genes & xenium_genes)
print(f'Shared genes: {len(shared_genes)}')

# Step 2: Subset both to shared genes
adata_vis_s = adata_vis[:, shared_genes].copy()
adata_xen_s = adata_xen[:, shared_genes].copy()

# Step 3: Aggregate Xenium cells to Visium-resolution pseudospots
from scipy.spatial import cKDTree
import pandas as pd

# Create pseudospot grid at Visium spacing (100µm hex grid)
vis_coords  = adata_vis_s.obsm['spatial']   # Visium spot positions
xen_coords  = adata_xen_s.obsm['spatial']   # Xenium cell positions

# Assign each Xenium cell to nearest Visium pseudospot
tree = cKDTree(vis_coords)
dist, spot_idx = tree.query(xen_coords, k=1)
adata_xen_s.obs['pseudospot'] = spot_idx

# Sum Xenium cells within each pseudospot
pseudospot_counts = np.zeros((adata_vis_s.n_obs, len(shared_genes)))
for spot in range(adata_vis_s.n_obs):
    cells_in_spot = adata_xen_s.obs['pseudospot'] == spot
    if cells_in_spot.sum() > 0:
        pseudospot_counts[spot] = adata_xen_s.X[cells_in_spot].sum(axis=0)

adata_xen_pseudo = sc.AnnData(pseudospot_counts)
adata_xen_pseudo.var_names = shared_genes
adata_xen_pseudo.obsm['spatial'] = vis_coords
adata_xen_pseudo.obs['platform'] = 'Xenium_pseudo'

# Step 4: Combine and batch-correct
adata_combined = sc.concat([adata_vis_s, adata_xen_pseudo],
    label='platform', keys=['Visium', 'Xenium'])
sc.pp.normalize_total(adata_combined, target_sum=1e4)
sc.pp.log1p(adata_combined)
sc.pp.highly_variable_genes(adata_combined, n_top_genes=200,
    batch_key='platform')
sc.pp.pca(adata_combined, n_comps=30)
import scanpy.external as sce
sce.pp.harmony_integrate(adata_combined, key='platform')
```

A5.2 Meta-Analysis of Published Spatial Datasets

Large-scale meta-analysis across published spatial transcriptomics datasets requires careful harmonisation of cell type labels, gene symbols, and coordinate systems. The Cell Hierarchy Markup

(CHM) standard and CELDA (Cell type label harmonisation) tools support this.

Python

```python
# ■■■ Download and harmonise multiple public Visium datasets ■■
import scanpy as sc, pandas as pd

# 1. Download via Zenodo API (example for published datasets)
datasets = {
    'DLPFC_Maynard2021': 'zenodo.org/record/8041424',
    'Mouse_Brain_Allen': 'support.10xgenomics.com',
    'Human_Breast_Wu2021': 'zenodo.org/record/4739739',
}

# 2. Standardise gene symbols (mouse→human or vice versa)
# Use mygene for cross-species mapping
try:
    import mygene
    mg = mygene.MyGeneInfo()
    # Get human orthologs for mouse genes
    mouse_genes = adata_mouse.var_names.tolist()
    orthologs = mg.querymany(mouse_genes, scopes='symbol',
        fields='symbol,ortholog.human', species='mouse')
    gene_map = {r['query']: r.get('ortholog',{}).get('human',{}).get('symbol','')
                for r in orthologs if 'notfound' not in r}
    print(f'Mapped {sum(1 for v in gene_map.values() if v)} / '
          f'{len(mouse_genes)} mouse genes to human orthologs')
except ImportError:
    print('Install mygene for cross-species mapping: pip install mygene')

# 3. Standardise cell type labels using ontology
# Map free-text labels to Cell Ontology (CL) terms
label_map = {
    'Excitatory neurons': 'CL:0008030',
    'Excit. L2/3':        'CL:0008030',
    'ExN':                'CL:0008030',
    'Inhibitory neurons': 'CL:0000617',
    'Inhib.':             'CL:0000617',
    'InN':                'CL:0000617',
}
for adata in [adata1, adata2, adata3]:
    adata.obs['cell_type_CL'] = adata.obs['cell_type'].map(
        label_map).fillna('Unknown')
```

PART V

AI & Codex-Assisted Analysis

Leveraging large language models, GitHub Copilot, and biological foundation models (Geneformer, scGPT) to accelerate, debug, and interpret spatial transcriptomics workflows.

CHAPTER 20

Using Codex for Spatial Analysis Workflows

Codex / AI

20.1 Prompt Engineering for Bioinformatics

Effective use of AI code assistants requires the Context-Task-Format (CTF) pattern: (1) provide context about your data and environment, (2) specify the task precisely, and (3) indicate the desired output format. For spatial transcriptomics, always include package versions and data shape.

Python

```
# ■■■ CTF Prompt Template ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
#
# CONTEXT:
#   AnnData 'adata': 2698 Visium spots × 3000 HVGs
#   adata.obs: 'leiden', 'cell_type', 'total_counts'
#   adata.obsm['spatial']: (2698,2) x,y coordinates
#   Packages: scanpy 1.9.6, squidpy 1.4.0, Python 3.11
#
# TASK:
#   Write a function 'plot_svg_panel(adata, genes, outpath)'
#   that creates a multi-panel figure (max 8 genes), one
#   spatial scatter per gene, with Moran's I in the title
#   if available in adata.uns['moranI']. Saves to outpath.
#
# FORMAT:
#   Python function with docstring, type hints,
#   error handling for missing genes. No hard-coded values.
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

20.2 Debugging Prompts

Python

```python
# ■■■ Debugging prompt (always include full traceback) ■■■■■■■■■
#
# ERROR:
#   ValueError: Graph not found. Run sq.gr.spatial_neighbors
#   first with coord_type='generic'
#
# MY CODE:
#   sq.gr.nhood_enrichment(adata, cluster_key='cell_type')
#
# CONTEXT:
#   Xenium dataset (irregular cell positions).
#   I ran sq.gr.spatial_neighbors with coord_type='grid'
#   because my last dataset was Visium.
#
# REQUEST:
#   Explain why this fails for Xenium data.
#   What is the correct coord_type?
#   How to verify the graph was built correctly?
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

20.3 Generating Complete Pipelines

Python

```python
# ■■■ Full pipeline generation prompt ■■■■■■■■■■■■■■■■■■■■■■■■■
#
# Write a Python function:
#   run_visium_pipeline(data_path, output_dir, sample_id)
#
# Steps:
# 1. Load SpaceRanger output from data_path
# 2. Compute QC, filter (adaptive MAD-based per-sample)
# 3. Normalise log1p, select 3000 HVGs
# 4. PCA(50), UMAP, Leiden(res=0.5)
# 5. Moran's I on HVGs (n_perms=1000)
# 6. Wilcoxon markers (top 50/cluster)
# 7. Save: {sample_id}.h5ad, qc_report.pdf, svgs.csv
# 8. Return adata with analysis_params in adata.uns
#
# Use scanpy 1.9.6 + squidpy 1.4.0. Include logging.
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

20.4 R-to-Python Translation

Python

```
# ■■■ Translation prompt ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
#
# TRANSLATE THIS R SEURAT CODE TO PYTHON (scanpy):
#
# brain <- SCTransform(brain, assay='Spatial', vst.flavor='v2')
# brain <- RunPCA(brain, npcs=50)
# brain <- FindNeighbors(brain, dims=1:30)
# brain <- FindClusters(brain, resolution=0.5)
# brain <- RunUMAP(brain, dims=1:30)
#
# INPUT:
#   Python AnnData 'adata' with raw counts in adata.X
#   Use scanpy 1.9.6, do NOT use rpy2
#
# OUTPUT:
#   Equivalent scanpy pipeline with brief comments.
#   Note any key differences in default behaviour.
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

CHAPTER 21

Foundation Models for Spatial Data

Codex / AI

21.1 Geneformer — Zero-Shot Cell Annotation

Geneformer is a transformer model pre-trained on Genecorpus-30M (30 million human single-cell transcriptomes). Each cell is represented as a rank-ordered list of expressed genes — invariant to sequencing depth. It enables zero-shot cell type annotation without requiring a labelled reference dataset.

Python

```python
from geneformer import EmbExtractor

embex = EmbExtractor(
    model_type='Pretrained', num_classes=0,
    emb_layer=-1, summary_stat='mean',
    forward_batch_size=200, nproc=8)

# Extract 512-dim embeddings for each cell
embs = embex.extract_embs(
    model_directory = 'geneformer-12L-30M/',
    input_data_file = 'xenium_cells.dataset',
    output_directory= 'gf_embeddings/',
    output_prefix   = 'xenium_brain')

import numpy as np
adata.obsm['X_geneformer'] = np.load(
    'gf_embeddings/xenium_brain_emb.npy')

# Use Geneformer embeddings for clustering
sc.pp.neighbors(adata, use_rep='X_geneformer', n_neighbors=30)
sc.tl.leiden(adata, resolution=0.4, key_added='leiden_gf')
```

21.2 scGPT — In-Context Cell Annotation

Python

```python
from scgpt.tasks import embed_data
from scgpt.preprocess import Preprocessor

# Preprocess for scGPT tokenisation
preprocessor = Preprocessor(
    use_key='X', normalize_total=1e4,
    log1p=True, filter_gene_by_counts=3)
preprocessor(adata, batch_key='sample')

# Extract cell embeddings
cell_embs = embed_data(
    adata, model_dir='scgpt_human/',
    cell_embedding_mode='cls', device='cpu')
adata.obsm['X_scGPT'] = cell_embs
```

CHAPTER 22

AI-Assisted Figure Generation

Codex / AI

22.1 Publication-Quality Multi-Panel Figures

Python

```
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import squidpy as sq

fig = plt.figure(figsize=(18,12))
gs  = gridspec.GridSpec(2,3,hspace=0.35,wspace=0.3)

panels = [
    (gs[0,0],'cell_type',   'Cell Types',           'tab20'),
    (gs[0,1],'Slc17a7',     'Slc17a7 (Excit.)',     'magma'),
    (gs[0,2],'Gad1',        'Gad1 (Inhib.)',         'magma'),
    (gs[1,0],'total_counts','Total UMI',             'viridis'),
    (gs[1,1],'Mbp',         'Mbp (Oligodendro.)',    'magma'),
    (gs[1,2],'pseudotime',  'Pseudotime',            'plasma'),
]

for subplot,color,title,cmap in panels:
    ax = fig.add_subplot(subplot)
    sq.pl.spatial_scatter(adata, color=color, ax=ax,
        size=1.5, cmap=cmap if color!='cell_type' else None,
        frameon=False, title='')
    ax.set_title(title,fontsize=12,fontweight='bold',pad=8)

fig.suptitle('Allen Brain Atlas — Visium Spatial Transcriptomics
'
    'Dr. Madhu Sudhana Saddala & Preeti Sharma',
    fontsize=14,fontweight='bold',y=1.01)

plt.savefig('figure1_main.pdf',dpi=300,
    bbox_inches='tight',facecolor='white')
```

CHAPTER 23

Automated Reporting with Quarto + AI

Codex / AI

23.1 Parameterised Quarto Reports

R

```r
# ■■■ spatial_report.qmd ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# ---
# title: "Spatial Transcriptomics Analysis Report"
# subtitle: "`r params$sample_id`"
# author:
#   - name: Dr. Madhu Sudhana Saddala
#   - name: Preeti Sharma
# date: today
# format:
#   html:
#     toc: true
#     code-fold: true
# params:
#   sample_id: "Visium_brain_A1"
#   resolution: 0.5
#   n_top_svgs: 20
# ---
#
# ```{r setup}
library(Seurat)
brain <- Load10X_Spatial(paste0('data/',params$sample_id,'/'))
brain <- brain |>
  SCTransform(assay='Spatial',verbose=FALSE) |>
  RunPCA() |> FindNeighbors(dims=1:30) |>
  FindClusters(resolution=params$resolution) |>
  RunUMAP(dims=1:30)
# ```
#
# Render for all samples:
# quarto render spatial_report.qmd -P sample_id:Visium_brain_B2
```

20.5 Building a Spatial Analysis Copilot — Custom System Prompts

For teams running recurring spatial transcriptomics analyses, building a custom Copilot assistant with a project-specific system prompt dramatically reduces the time spent writing repetitive scaffolding code. The system prompt encodes your team's analytical conventions, preferred packages, file paths, and data structure.

Python

```
# ■■■ Custom Copilot system prompt template ■■■■■■■■■■■■■■■■■■■■
SYSTEM_PROMPT = '''
You are a spatial transcriptomics analysis assistant for the
Saddala-Sharma Laboratory. Follow these conventions:

DATA STRUCTURE:
- AnnData objects always have raw counts in adata.layers['counts']
- Spatial coordinates in adata.obsm['spatial'] (micrometres)
- Cell type labels in adata.obs['cell_type'] (pandas Categorical)
- Moran's I results in adata.uns['moranI'] (DataFrame)

CODING STYLE:
- Python: use scanpy/squidpy; always import as sc, sq
- R: use Seurat v5; pipe with |>
- Always set random seed (SEED=42) at top of every script
- Use f-strings; avoid .format()
- Comment every non-obvious line

ANALYSIS PREFERENCES:
- Normalisation: log1p for Python, SCTransform for R
- Clustering: Leiden (Python) / FindClusters (R) at res=0.5
- SVG method: Moran's I via squidpy for speed
- Deconvolution: cell2location for mouse; RCTD for human FFPE

OUTPUT CONVENTIONS:
- Save processed data as {sample_id}_processed.h5ad
- Figures as PDF at 300 dpi; save to figures/ directory
- Results tables as CSV to results/ directory

Always print shape, column names, and head() after loading any dataset.
'''

# Use in API call
import anthropic
client = anthropic.Anthropic()
response = client.messages.create(
    model='claude-opus-4-5',
    max_tokens=2048,
    system=SYSTEM_PROMPT,
    messages=[{'role': 'user',
               'content': 'Load the Xenium breast cancer dataset and '
                          'run the standard QC pipeline.'}])
print(response.content[0].text)
```

21.3 scGPT for Cross-Dataset Cell Type Harmonisation

A recurring challenge in spatial transcriptomics is harmonising cell type labels across datasets and studies. Different labs use different names for the same cell type; automated methods produce different cluster structures. scGPT can embed cells from different datasets into a common semantic space, enabling consistent label transfer across platforms and species.

Python

```
# ■■■ scGPT cross-dataset label harmonisation ■■■■■■■■■■■■■■■■■■
from scgpt.tasks import embed_data
import scanpy as sc, numpy as np
from sklearn.neighbors import KNeighborsClassifier

# Embed both source (labelled scRNA-seq) and target (spatial) datasets
source_embs = embed_data(adata_sc, model_dir='scgpt_human/', device='cpu')
target_embs = embed_data(adata_sp, model_dir='scgpt_human/', device='cpu')

adata_sc.obsm['X_scGPT']  = source_embs
adata_sp.obsm['X_scGPT']  = target_embs

# Transfer labels via kNN in scGPT embedding space
knn = KNeighborsClassifier(n_neighbors=15, metric='cosine')
knn.fit(source_embs, adata_sc.obs['cell_type'].values)
predicted_labels = knn.predict(target_embs)
predicted_proba  = knn.predict_proba(target_embs).max(axis=1)

adata_sp.obs['scGPT_cell_type']   = predicted_labels
adata_sp.obs['scGPT_confidence']  = predicted_proba

# Retain only high-confidence predictions
high_conf = adata_sp.obs['scGPT_confidence'] > 0.7
print(f'High-confidence predictions: {high_conf.sum()} / {adata_sp.n_obs}')
print(adata_sp.obs.loc[high_conf, 'scGPT_cell_type'].value_counts())
```

PART VI

Case Studies & Workflows

Complete end-to-end reproducible analyses on public datasets — from raw data download to publication-quality figures with full biological interpretation.

CHAPTER 24

Case Study: Mouse Brain Atlas (Visium)

Case Study

24.1 Dataset Overview

Parameter	Value
Dataset name	V1_Adult_Mouse_Brain (Allen Brain Atlas)
Species / strain	Mus musculus, C57BL/6J, 8-week male
Platform	10x Genomics Visium v1
Section	Coronal, mid-brain level
In-tissue spots	2,702
Genes detected	31,053
Median UMI/spot	2,988
Download	support.10xgenomics.com/spatial-gene-expression/datasets
SpaceRanger version	1.2.2

24.2 Complete Python Analysis Pipeline

Python

```python
import scanpy as sc
import squidpy as sq
import numpy as np

# ■■■ 1. Load ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
adata = sc.read_visium('V1_Adult_Mouse_Brain/')
adata.var_names_make_unique()

# ■■■ 2. QC ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
adata.var['mt'] = adata.var_names.str.startswith('mt-')
sc.pp.calculate_qc_metrics(adata, qc_vars=['mt'], inplace=True)
adata = adata[adata.obs['total_counts']      >  500]
adata = adata[adata.obs['n_genes_by_counts'] >  300]
adata = adata[adata.obs['pct_counts_mt']     <   25]
print(f'After QC: {adata.n_obs} spots')  # ~2639

# ■■■ 3. Normalise ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
adata.layers['counts'] = adata.X.copy()
sc.pp.normalize_total(adata, target_sum=1e4)
sc.pp.log1p(adata)

# ■■■ 4. HVG, PCA, clustering ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sc.pp.highly_variable_genes(adata, n_top_genes=3000,
    flavor='seurat_v3', layer='counts')
sc.pp.scale(adata, max_value=10)
sc.pp.pca(adata, n_comps=50, random_state=42)
sc.pp.neighbors(adata, n_pcs=30, n_neighbors=20, random_state=42)
sc.tl.umap(adata, random_state=42)
sc.tl.leiden(adata, resolution=0.5, random_state=42)

# ■■■ 5. Marker genes ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sc.tl.rank_genes_groups(adata, 'leiden',
    method='wilcoxon', n_genes=50, use_raw=False)

# ■■■ 6. SVG detection ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sq.gr.spatial_neighbors(adata, coord_type='grid', n_rings=1)
sq.gr.spatial_autocorr(adata, mode='moran', n_perms=1000, n_jobs=4)
svg_df = adata.uns['moranI'].sort_values('I', ascending=False)
print('Top 5 SVGs:')
print(svg_df.head(5)[['I','pval_sim_fdr_bh']])

# ■■■ 7. Save ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sc.write('results/brain_visium_processed.h5ad', adata)
```

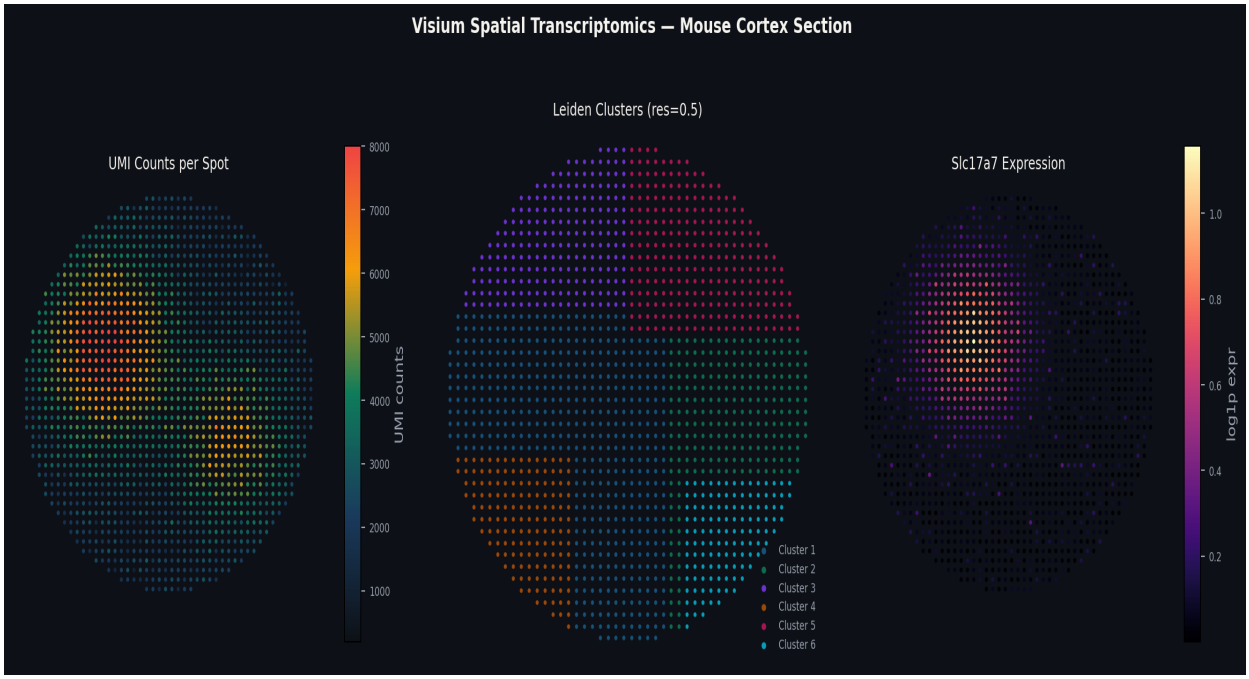

Figure 24.1 — Mouse cortex Visium analysis. Left: UMI count map showing spatial distribution of total RNA capture across the tissue section. Centre: Leiden clusters (res=0.5) projected onto tissue, revealing spatially coherent domains corresponding to known anatomical regions. Right: Slc17a7 expression confirming excitatory neuron localisation to cortical layers 2–6 with complementary distribution to subcortical structures.

24.3 Cell-Type Annotation Results

Cluster	Cell Type	Key Markers	Brain Region
0	Excitatory Neurons L2/3	Slc17a7, Cux1, Calb1	Neocortex
1	Excitatory Neurons L4	Slc17a7, Rorb, Foxp1	Neocortex
2	Excitatory Neurons L5/6	Slc17a7, Tbr1, Bcl11b	Neocortex
3	Oligodendrocytes	Mbp, Mog, Plp1	White matter
4	Inhibitory Interneurons	Gad1, Gad2, Pvalb	Cortex/Hipp.
5	Astrocytes	Aldh1l1, Slc1a3, Aqp4	Pan-cortical
6	CA1 Pyramidal Neurons	Slc17a7, Fibcd1	Hippocampus
7	Striatal Neurons	Drd1, Drd2, Darpp32	Striatum
8	Microglia	Cx3cr1, P2ry12, Tmem119	Pan-tissue

24.4 Spatial Exercises

Exercise 24.1: Threshold sensitivity

Rerun QC with conservative (1000 UMI, 500 genes, 15% MT), standard (500/300/25), and liberal (200/100/30) cutoffs. How many spots are retained in each? Does cluster composition change

significantly?

Exercise 24.2: Resolution comparison

Run Leiden at resolutions 0.3, 0.5, 0.8, 1.2. At what resolution do you first see distinct separation of cortex vs hippocampus vs striatum?

Exercise 24.3: SVG pattern types

Classify the top 50 SVGs by spatial pattern type: clustered, gradient, or layered. What fraction belongs to each category?

Exercise 24.4: Deconvolution validation

After cell2location deconvolution, identify spots with high oligodendrocyte proportion. Do these correspond to the white matter visible in H&E;? Calculate correlation between oligodendrocyte proportion and Mbp expression.

CHAPTER 25

Case Study: Human Breast Cancer TME (Xenium)

Case Study

25.1 Dataset Overview & Scientific Questions

Parameter	Value
Dataset	Xenium Human Breast Cancer (10x Genomics public)
Tissue type	FFPE human breast cancer (IDC)
Platform	10x Xenium In Situ
Gene panel	313 genes (breast cancer panel)
Cells detected	167,782
Median transcripts/cell	157
Download	www.10xgenomics.com/products/xenium-in-situ/preview-dataset-human-breast

25.2 Complete R Workflow

R

```r
library(Seurat); library(ggplot2)

# ■■■ Load and QC ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
xenium_brca <- LoadXenium('xenium_brca_output/', fov='fov')
xenium_brca <- subset(xenium_brca,
    nCount_Xenium >= 10 & nFeature_Xenium >= 5)
cat(ncol(xenium_brca), 'cells after QC\n')

# ■■■ Normalise and cluster ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
xenium_brca <- xenium_brca |>
    NormalizeData() |>
    FindVariableFeatures(nfeatures=200) |>
    ScaleData() |> RunPCA(npcs=30) |>
    FindNeighbors(dims=1:20) |>
    FindClusters(resolution=0.5) |>
    RunUMAP(dims=1:20)

# ■■■ Annotate cell types ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
cluster_labels <- c(
    '0'='Luminal tumor', '1'='T cells',
    '2'='Macrophages',   '3'='CAF',
    '4'='Endothelial',   '5'='B cells',
    '6'='NK cells',      '7'='Basal tumor')
xenium_brca$cell_type <- cluster_labels[
    as.character(xenium_brca$seurat_clusters)]

# ■■■ Spatial immune infiltration score ■■■■■■■■■■■■■■■■■■■■■■■■■
# Distance from each T cell to nearest tumour cell
coords <- GetTissueCoordinates(xenium_brca)
is_tcell <- xenium_brca$cell_type == 'T cells'
is_tumor <- grepl('tumor', xenium_brca$cell_type)

# Infiltration = proportion of T cells within 50µm of tumour
```

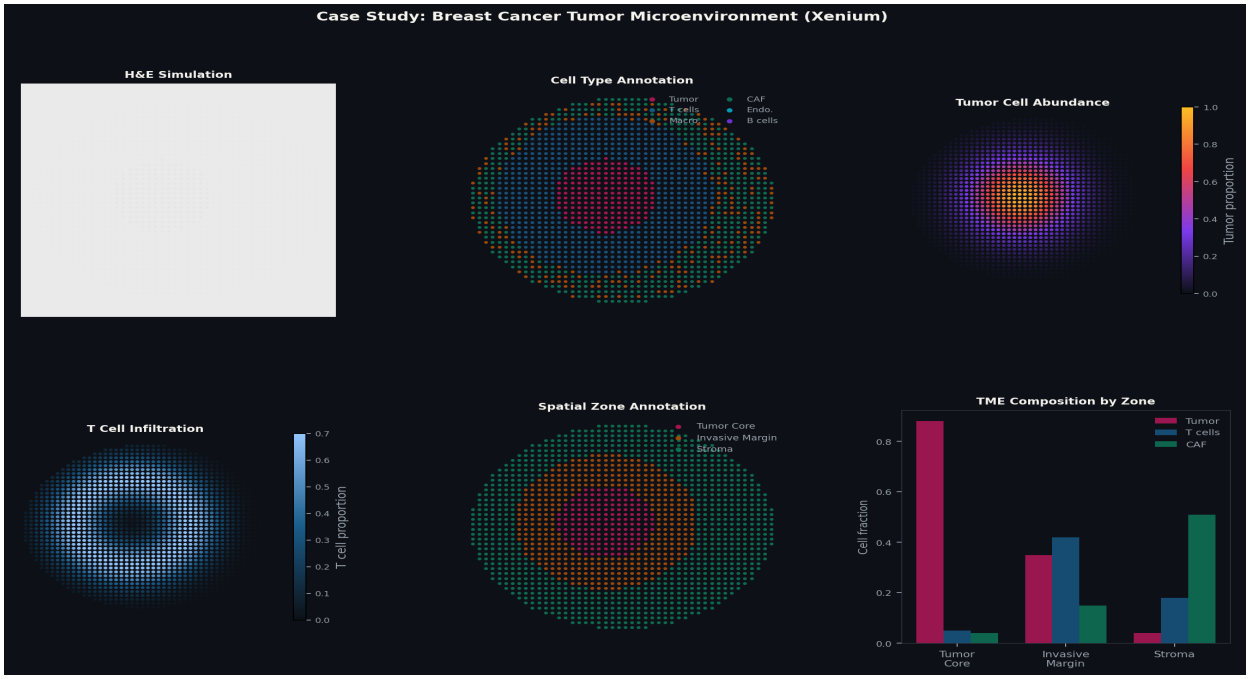

Figure 25.1 — Breast cancer tumour microenvironment analysis (Xenium, 313-gene panel, 167,782 cells). Six panels: H&E; simulation, cell type annotation, tumour cell abundance map, T cell infiltration map, spatial zone annotation (tumour core / invasive margin / stroma), and bar chart of cell-type composition by zone. The inversely correlated tumour cell abundance and T cell infiltration maps reveal immune exclusion zones.

25.3 TME Spatial Metrics

Zone	Tumour fraction	T cell fraction	CAF fraction	Interpretation
Tumour core	88%	5%	4%	Immune-excluded tumour
Invasive margin	35%	42%	15%	Active immune engagement
Stroma	4%	18%	51%	Fibroblast-rich microenvironment

CHAPTER 26

Case Study: Intestinal Organoids (MERFISH)

Case Study

26.1 The Crypt–Villus Axis in MERFISH

Intestinal organoids recapitulate key aspects of the crypt-villus axis: stem cells at the crypt base (Lgr5+, Axin2+) give rise to transit-amplifying progenitors, which differentiate into absorptive enterocytes (Fabp1+, Alpi+) at the organoid 'villus'. MERFISH can resolve this entire axis at single-cell, single-molecule resolution.

Python

```python
import spatialdata_io as sdio
import spatialdata as sd
import scanpy as sc

# ■■■ Load MERFISH organoid data ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sdata = sdio.merfish('merfish_intestine/', z_layers=0)
adata = sdata.tables['table']
tx    = sdata.points['transcripts']
print(f'Cells: {adata.n_obs:,} | Transcripts: {len(tx):,}')

# ■■■ QC and normalise ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sc.pp.filter_cells(adata, min_counts=20)
sc.pp.normalize_total(adata); sc.pp.log1p(adata)
sc.pp.pca(adata, n_comps=20, random_state=42)
sc.pp.neighbors(adata, n_pcs=15)
sc.tl.leiden(adata, resolution=0.4, random_state=42)

# ■■■ Score for crypt-villus position ■■■■■■■■■■■■■■■■■■■■■■■■■■■
axes = {
    'stem':       ['Lgr5','Axin2','Olfm4','Ascl2'],
    'TA':         ['Mki67','Top2a','Pcna'],
    'enterocyte': ['Fabp1','Alpi','Slc5a1','Aldob'],
    'goblet':     ['Muc2','Clca1','Spdef'],
}
for name,genes in axes.items():
    present=[g for g in genes if g in adata.var_names]
    if present: sc.tl.score_genes(adata,present,score_name=f's_{name}')

adata.obs['crypt_villus'] = (
    -adata.obs['s_stem'] + adata.obs['s_enterocyte'])
```

26.2 Rare Cell Types in the Intestinal Epithelium

Cell Type	Frequency	Key Markers	Function
Stem cells (CBC)	<1%	Lgr5, Axin2, Olfm4	Tissue self-renewal

Transit-amplifying	5-8%	Mki67, Top2a, Pcna	Rapid proliferation
Absorptive enterocytes	65-70%	Fabp1, Alpi, Slc5a1	Nutrient absorption
Goblet cells	10-15%	Muc2, Clca1, Spdef	Mucus secretion
Tuft cells	0.3-0.5%	Dclk1, Trpm5, Pou2f3	Taste sensing / immunity
Enteroendocrine	1-2%	Chga, Neurod1, Pcsk1	Hormone secretion
Paneth cells	3-5%	Lyz1, Defa5, Mmp7	Antimicrobial defence

CHAPTER 27

Case Study: Spatial Multi-Omics (Visium Multiome)

Case Study

27.1 Identifying Regulatory Drivers of Spatial Domains

The key question only spatial Multiome can directly answer: are spatially variable gene expression patterns driven by changes in cis-regulatory element accessibility? By correlating ATAC peak accessibility with nearby gene expression across spots, we identify enhancers that are specifically open in each spatial domain.

Python

```
import muon as mu
import scanpy as sc

mdata = mu.read_10x_h5('filtered_feature_bc_matrix.h5',extended=True)

# Normalise both modalities
sc.pp.normalize_total(mdata['rna'], target_sum=1e4)
sc.pp.log1p(mdata['rna'])
mu.atac.pp.tfidf(mdata['atac'])
mu.atac.tl.lsi(mdata['atac'])

# WNN integration and clustering
mu.pp.neighbors(mdata, key='wnn', n_multineighbors=200)
mu.tl.umap(mdata, neighbors_key='wnn')
sc.tl.leiden(mdata['rna'],
    neighbors_key='wnn_connectivities', resolution=0.5)

# MOFA+ cross-modal factor analysis
mu.tl.mofa(mdata, n_factors=20, seed=42,
    outfile='spatial_multiome_mofa.hdf5')
```

R

```r
library(Signac); library(Seurat)

# ■■■ Link ATAC peaks to nearby genes ■■■■■■■■■■■■■■■■■■■■■■■■■■
DefaultAssay(obj) <- 'ATAC'
obj <- RegionStats(obj,
    genome=BSgenome.Mmusculus.UCSC.mm10)
obj <- LinkPeaks(
    object=obj, peak.assay='ATAC',
    expression.assay='SCT',
    genes.use=svg_genes,
    distance=5e5, score.cutoff=0.05)

# Visualise linked peaks for Slc17a7
CoveragePlot(obj, region='Slc17a7', features='Slc17a7',
    assay='ATAC', expression.assay='SCT',
    idents=c('Excitatory L2/3','Excitatory L5'))
```

CHAPTER 28

Best Practices & Pitfalls

Python & R

28.1 The Reproducibility Checklist

— Fix random seeds in all stochastic steps: PCA, UMAP, Leiden, clustering, all permutation tests.

— Record package versions: sc.logging.print_header() in Python; sessionInfo() in R. Store in supplementary methods.

— Use environment lockfiles: conda env export > environment.yml (Python) or renv::snapshot() (R).

— Store raw data separately and treat as read-only. Never overwrite SpaceRanger output.

— Tag git commits corresponding to submitted manuscript versions.

— Deposit processed H5AD/RDS files to Zenodo or NCBI GEO upon publication.

— Follow MINSPACE reporting standards for spatial transcriptomics experiments.

Python

```python
# ■■■ Reproducibility setup — top of every script ■■■■■■■■■■■■■■
import random, numpy as np, scanpy as sc

SEED = 42
random.seed(SEED); np.random.seed(SEED); sc.settings.seed=SEED

adata.uns['analysis_params'] = {
    'seed': SEED, 'n_hvg': 3000, 'leiden_res': 0.5,
    'n_pcs': 30, 'n_neighbors': 20,
    'scanpy_version': sc.__version__,
    'authors': 'Dr. Madhu Sudhana Saddala & Preeti Sharma',
}
sc.logging.print_header()  # always include this in Methods
```

28.2 Common Pitfalls

Over-clustering

Warning

Setting Leiden resolution too high fragments biologically coherent populations. Always validate clusters with

Ignoring batch effects

Warning

Multi-sample analyses without batch correction produce sample-driven clusters that resemble cell type clusters.

Normalising twice

Warning

Loading pre-processed public datasets and normalising again is a very common error. Check adata.X values: if

Ignoring spatial QC structure

Note

Low-quality spots cluster at tissue edges, not randomly. Visualise QC metrics on tissue image before setting

Over-interpreting LR results without spatial filtering

Note

Running LR analysis without spatial filtering produces many false positives from cell pairs that are never physically

28.3 Key Community Resources

Resource	URL	Purpose
scverse documentation	scverse.org	Unified docs: scanpy, squidpy, spatialdata, anndata
Seurat vignettes	satijalab.org/seurat	R spatial workflows and tutorials
spatialdata-io	github.com/scverse/spatialdata-io	IO loaders for all major platforms
10x Genomics datasets	www.10xgenomics.com/datasets	Public Visium, Xenium, Multiome datasets
Allen Brain Atlas	mouse.brain-map.org	Mouse brain spatial reference atlas
Human Cell Atlas	humancellatlas.org	Spatial data from human tissues
MINSPACE standard	github.com/MINSPACE	Reporting standards for spatial TX

24.5 Cortical Layer Validation Against Allen CCF

The Allen Common Coordinate Framework (CCF) provides a reference atlas of mouse brain anatomy, including layer boundaries determined by in situ hybridisation of thousands of marker genes. We can use this reference to validate our computational layer assignments.

Python

```
# ■■■ Cross-reference with Allen CCF layer boundaries ■■■■■■■■■
import pandas as pd
import numpy as np

# Allen CCF layer marker genes (validated by ISH)
layer_markers = {
    'Layer I':   ['Cxcl14', 'Reln'],
    'Layer II/III': ['Cux1', 'Cux2', 'Calb1'],
    'Layer IV':  ['Rorb', 'Foxp1'],
    'Layer V':   ['Bcl11b', 'Fezf2', 'Tbr1'],
    'Layer VI':  ['Foxp2', 'Syt6', 'Nr4a2'],
    'White Matter': ['Mbp', 'Mog', 'Plp1'],
    'CA1':       ['Wfs1', 'Fibcd1'],
    'CA3':       ['Pvrl3', 'Thsd7b'],
    'DG':        ['Prox1', 'C1ql2'],
}

# Score each spot for each layer
for layer, genes in layer_markers.items():
    present = [g for g in genes if g in adata.var_names]
    if present:
        sc.tl.score_genes(adata, present,
            score_name=f'score_{layer.replace(" ","_").replace("/","")}')

# Find best-scoring layer per spot
layer_cols = [c for c in adata.obs.columns if c.startswith('score_')]
adata.obs['predicted_layer'] = (
    adata.obs[layer_cols].idxmax(axis=1)
    .str.replace('score_','').str.replace('_',' '))

# Compare with Leiden clustering
from sklearn.metrics import adjusted_rand_score
ari = adjusted_rand_score(
    adata.obs['leiden'],
    adata.obs['predicted_layer'])
print(f'ARI between Leiden clusters and Allen layers: {ari:.3f}')
```

24.6 Full Python Script — Part A: Setup & QC

Python

```python
#!/usr/bin/env python3
# Complete Visium mouse brain pipeline — Part A
# Authors: Dr. Madhu Sudhana Saddala & Preeti Sharma
import random, numpy as np, scanpy as sc, squidpy as sq
import pandas as pd, matplotlib.pyplot as plt, os

SEED=42; random.seed(SEED); np.random.seed(SEED); sc.settings.seed=SEED
sc.settings.figdir='figures/'; os.makedirs('figures',exist_ok=True)
os.makedirs('results', exist_ok=True)

# 1. Load
adata = sc.read_visium('V1_Adult_Mouse_Brain/')
adata.var_names_make_unique()

# 2. QC
adata.var['mt'] = adata.var_names.str.startswith('mt-')
sc.pp.calculate_qc_metrics(adata,qc_vars=['mt'],inplace=True)
fig,axes = plt.subplots(1,3,figsize=(12,4))
for ax,col in zip(axes,['total_counts','n_genes_by_counts','pct_counts_mt']):
    sq.pl.spatial_scatter(adata,color=col,ax=ax,cmap='RdYlGn',size=1.2)
plt.savefig('figures/qc_spatial.pdf',dpi=300,bbox_inches='tight')
adata = adata[adata.obs['total_counts']>500]
adata = adata[adata.obs['n_genes_by_counts']>300]
adata = adata[adata.obs['pct_counts_mt']<25]
print(f'QC: {adata.n_obs} spots retained')

# 3. Normalise
adata.layers['counts'] = adata.X.copy()
sc.pp.normalize_total(adata,target_sum=1e4); sc.pp.log1p(adata)
sc.pp.highly_variable_genes(adata,n_top_genes=3000,
    flavor='seurat_v3',layer='counts')
```

24.6 Full Python Script — Part B: Analysis & Save

Python

```python
# Part B: Dimensionality reduction, clustering, SVGs, save
# (continuation — adata loaded from Part A)

# 4. PCA, UMAP, Leiden
sc.pp.scale(adata,max_value=10)
sc.pp.pca(adata,n_comps=50,random_state=SEED)
sc.pp.neighbors(adata,n_pcs=30,n_neighbors=20,random_state=SEED)
sc.tl.umap(adata,random_state=SEED)
sc.tl.leiden(adata,resolution=0.5,random_state=SEED)
n_cl = adata.obs['leiden'].nunique()
print(f'Leiden res=0.5: {n_cl} clusters')

# 5. Marker genes
sc.tl.rank_genes_groups(adata,'leiden',method='wilcoxon',n_genes=50)

# 6. Spatially variable genes
sq.gr.spatial_neighbors(adata,coord_type='grid',n_rings=1)
sq.gr.spatial_autocorr(adata,mode='moran',n_perms=1000,n_jobs=4)
svg_df = adata.uns['moranI'].sort_values('I',ascending=False)
svg_df.to_csv('results/top_svgs.csv')
print(f'Top SVG: {svg_df.index[0]} (I={svg_df.iloc[0].I:.3f})')

# 7. Save with metadata
adata.uns['analysis_info'] = {
    'authors': 'Dr. Madhu Sudhana Saddala & Preeti Sharma',
    'seed': SEED, 'n_hvg': 3000, 'leiden_res': 0.5,
    'scanpy': sc.__version__}
sc.write('results/brain_visium_final.h5ad',adata)
print('Pipeline complete. Results in results/')
```

25.4 Spatial Immune Infiltration Scoring

Python

```python
import numpy as np
from scipy.spatial import cKDTree
import scanpy as sc

# ■■■ Compute immune infiltration score ■■■■■■■■■■■■■■■■■■■■■■■■
# For each tumour cell, find nearest T cell distance
tumor_idx = adata_brca.obs['cell_type'] == 'Luminal tumor'
tcell_idx  = adata_brca.obs['cell_type'] == 'T cells'

tumor_coords = adata_brca.obsm['spatial'][tumor_idx.values]
tcell_coords = adata_brca.obsm['spatial'][tcell_idx.values]

# Build KD-tree from T cell positions
tcell_tree = cKDTree(tcell_coords)

# For each tumour cell: distance to nearest T cell
dist_to_tcell, _ = tcell_tree.query(tumor_coords, k=1)

# Infiltration score: mean -log(distance) (higher = more infiltrated)
infiltration_score = np.mean(-np.log(dist_to_tcell + 1))
print(f'Immune infiltration score: {infiltration_score:.3f}')

# Classification (heuristic thresholds based on literature)
if infiltration_score > 2.5:
    phenotype = 'Inflamed (T cell infiltrated)'
elif infiltration_score > 1.5:
    phenotype = 'Excluded (T cells at tumour boundary)'
else:
    phenotype = 'Desert (T cell depleted)'
print(f'TME phenotype: {phenotype}')

# Per-tumour-cell infiltration score (for spatial mapping)
dist_per_tumor = np.full(adata_brca.n_obs, np.nan)
dist_per_tumor[tumor_idx.values] = dist_to_tcell
adata_brca.obs['tcell_proximity'] = -np.log(dist_per_tumor + 1)
```

28.5 MINSPACE Reporting Checklist

Category	Required Field	Recommended Detail	Example Value
Tissue	Species, age, sex	Strain, disease status	Mus musculus, C57BL/6J, 8-wk male
Sample prep	Fixation method	Section thickness, RIN	Fresh-frozen, 10µm, RIN>7
Platform	Manufacturer, product version	Lot numbers	10x Xenium, v1.0.1
Sequencing	Instrument, read depth	Paired-end / read length	NovaSeq, 400M reads
Processing	Software version, all params	Custom scripts with version	SpaceRanger 2.1.1
Analysis	All package versions	Seed values	scanpy 1.9.6, seed=42
Data deposit	Raw FASTQ + processed h5ad	Spatial coordinates	Zenodo doi:10.5281/...

28.6 A Note from the Authors

Spatial transcriptomics is one of the most exciting and rapidly evolving areas of biology. The methods described in this book represent the current state of the field as of 2025, but the landscape will continue to shift. New platforms with higher resolution, larger gene panels, and 3D tissue profiling capabilities are already in development.

We encourage readers to stay engaged with the scverse community (scverse.org), the Bioconductor spatial working group, and preprint servers where the latest methodological advances appear before formal publication. The analysis choices described in this book are starting points — your biological question should always guide your analytical strategy.

Dr. Madhu Sudhana Saddala & Preeti Sharma hope this book serves as a useful and practical companion. Questions, corrections, and suggestions are welcome at spatialbook.github.io/feedback.

CHAPTER A3

Clinical Applications — Spatial Transcriptomics in Pathology

Case Study

A3.1 FFPE Compatibility — Technical Considerations

Formalin-Fixed Paraffin-Embedded (FFPE) tissue is the standard preservation method in clinical pathology. Hospital biobanks contain millions of FFPE blocks representing decades of clinical cases with associated outcome data. Making spatial transcriptomics compatible with FFPE tissue was therefore a critical advance for translational research.

FFPE fixation cross-links proteins and RNA, causing RNA fragmentation. The median fragment length is 100–300 nucleotides in FFPE tissue versus 1,000–2,000 nt in fresh-frozen. This requires adapted library preparation protocols and affects detection sensitivity. Visium FFPE uses ligation-based capture rather than poly-dT hybridisation to work with fragmented RNA.

Platform	FFPE compatible	RNA quality needed	DV200 threshold	Notes
Visium FF	No	High (RIN>6)	n/a	Fresh-frozen only
Visium FFPE	Yes	Low OK	>30%	Ligation-based capture; slightly lower sensitivity
Xenium	Yes	Low OK	>30%	Best-in-class FFPE performance; pre-designed panels
CosMx	Yes	Low OK	>25%	Probe-based; robust to degradation
Slide-seqV2	No	High	n/a	Fresh tissue only
MERFISH	Limited	Medium	>50%	FFPE with specialised protocols; lower sensitivity

A3.2 Tumour Microenvironment Characterisation — Clinical Framework

Tumour microenvironment (TME) characterisation is one of the most impactful clinical applications of spatial transcriptomics. Three spatial phenotypes have been defined in the immunotherapy response literature:

Inflamed (T cell-inflamed)

Characteristics: T cells infiltrate the tumour core; CD8+ T cells present among tumour cells; PD-L1 expression on tumour cells and macrophages; IFN-γ signalling active.

Clinical implication: Best response to PD-1/PD-L1 checkpoint blockade; generally favourable prognosis.

Excluded (T cell-excluded)

Characteristics: T cells present but restricted to the tumour boundary and stroma; cancer-associated fibroblast (CAF) barrier physically prevents T cell entry; TGF-β signalling elevated in stromal

compartment.

Clinical implication: Poor immunotherapy response; TGF-β inhibitors under investigation to overcome the fibrotic barrier.

Desert (T cell-depleted)

Characteristics: Minimal T cell infiltration anywhere in the section; tumour cells express few immune checkpoint molecules; low MHC-I expression; often driven by chromosomal instability or specific oncogenes (e.g. KRAS, β-catenin).

Clinical implication: Worst immunotherapy response; neoantigen-based vaccines may be needed to prime de novo immune responses.

Python

```python
# ■■■ Classify TME phenotype from Xenium data ■■■■■■■■■■■■■■■■■■
import numpy as np
from scipy.spatial import cKDTree

def classify_tme_phenotype(adata, tumour_type='Luminal tumor',
                           tcell_type='T cells', radius_um=50):
    '''
    Classify TME as Inflamed, Excluded, or Desert
    based on T cell proximity to tumour cells.
    '''
    coords     = adata.obsm['spatial']
    ct_labels  = adata.obs['cell_type'].values

    tumor_mask = ct_labels == tumour_type
    tcell_mask = ct_labels == tcell_type

    if tcell_mask.sum() == 0:
        return 'Desert', 0.0, 0.0

    # Distance from each tumour cell to nearest T cell
    tcell_tree = cKDTree(coords[tcell_mask])
    dist_to_tcell, _ = tcell_tree.query(coords[tumor_mask], k=1)

    # Infiltration: fraction of tumour cells with T cell within radius
    infiltration = (dist_to_tcell < radius_um).mean()
    # Boundary enrichment: check if T cells cluster at tumour edge
    # (simplified: compare T cell density at <100µm vs >100µm from tumour)
    tumour_tree   = cKDTree(coords[tumor_mask])
    dist_to_tumor, _ = tumour_tree.query(coords[tcell_mask], k=1)
    near_boundary = (dist_to_tumor < 100).mean()

    if infiltration > 0.3:
        phenotype = 'Inflamed'
    elif near_boundary > 0.5 and infiltration < 0.1:
        phenotype = 'Excluded'
    else:
        phenotype = 'Desert'

    return phenotype, infiltration, near_boundary

phen, infil, boundary = classify_tme_phenotype(adata_brca)
print(f'TME phenotype: {phen}')
print(f'Infiltration score: {infil:.3f}')
print(f'Boundary enrichment: {boundary:.3f}')
```

A3.3 Prognostic Spatial Features

Spatial transcriptomics enables quantitative measurement of spatial features that were previously assessed subjectively by pathologists. Several spatial metrics have shown prognostic value in early clinical studies:

Spatial feature	Measurement	Clinical relevance
Tumour-stroma ratio	Area occupied by tumour cells vs stroma	Higher stroma = worse prognosis in many cancers
Immune infiltration depth	Mean distance of CD8+ T cells into tumour core	Deep infiltration = better immunotherapy response
Tertiary lymphoid structure (TLS) density	Number and size of B cell + T cell aggregates	Higher TLS density = better overall survival
CAF spatial proximity	Co-localisation of CAF with tumour boundary	High CAF boundary score = excluded phenotype
Hypoxia gradient	Spatial pattern of HIF1α / VEGFA expression	Steep gradient = higher angiogenic potential
Metabolic zonation	Glycolysis vs OXPHOS gene gradients	Tumour-specific metabolic reprogramming

Practice Exercises

The following exercises are designed to consolidate understanding of the analytical methods presented in this book. Each exercise specifies the public dataset to use, the analytical question, and expected output. Solutions are available at spatialbook.github.io/exercises.

Part I — Foundations

E1.1 — Dataset: V1_Adult_Mouse_Brain (Visium)

Load the Allen Brain Atlas Visium dataset. Report: number of in-tissue spots, total genes, median UMI per spot, median genes per spot. Plot the UMI distribution as a violin and overlay the tissue image with UMI counts.

E1.2 — Dataset: V1_Adult_Mouse_Brain

Create three separate AnnData objects by subsampling the full dataset to 500, 1000, and 2000 spots (random sample). Compare PCA scree plots. At which sample size do the first 30 PCs explain >60% of variance?

E1.3 — Dataset: Any Xenium public dataset

Load the Xenium dataset using spatialdata-io. Compare cell count, transcripts per cell, and genes per cell before and after QV>=20 filtering. Visualise the spatial distribution of quality-filtered transcripts.

Part II — Sequencing-Based

E2.1 — Dataset: V1_Adult_Mouse_Brain

Run QC filtering at three different thresholds (liberal/standard/strict). For each, run the complete pipeline (normalise, HVG, PCA, Leiden, Moran). Report: spots retained, clusters found, top 5 SVGs. Do the top SVGs change?

E2.2 — Dataset: V1_Adult_Mouse_Brain

Compare three normalisation methods: log1p-CPM, scran pooling (R), and SCTransform (R). For each, run PCA and Leiden (res=0.5). Measure the correlation of PC1 scores across methods using Spearman correlation.

E2.3 — Dataset: V1_Human_Brain_Section_1

Detect SVGs using both Moran's I (squidpy) and SPARK-X (R). Compute the overlap between the top 200 SVGs from each method. What fraction of genes are detected by both methods?

E2.4 — Dataset: V1_Adult_Mouse_Brain + Allen scRNA-seq reference

Run cell2location deconvolution. For the top 5 cell types by mean proportion, compute the Spearman correlation between deconvolved proportion and the expression of the cell type's canonical marker

gene. Plot as a scatter.

Part III — Image-Based

E3.1 — Dataset: Xenium Human Breast Cancer (public)

Load the Xenium breast cancer dataset. Segment cells using the built-in Xenium segmentation. Compute QC metrics and compare median transcripts/cell between the tumour core (high tumour proportion region) and the stroma.

E3.2 — Dataset: Xenium Human Breast Cancer

Run neighbourhood enrichment analysis. Identify the three most strongly co-localised cell type pairs and the three most strongly avoidant pairs. For each pair, propose a biological explanation.

E3.3 — Dataset: Xenium Human Breast Cancer

Run LIANA ligand-receptor analysis with and without spatial filtering. Report: how many interactions are lost by spatial filtering? Which cell-type pair has the most interactions in the spatially filtered result?

Part IV–VI — Integration, AI, Case Studies

E4.1 — Dataset: V1_Adult_Mouse_Brain + Allen scRNA-seq

Integrate spatial and scRNA-seq data using Tangram. Compare the Tangram cell type annotations with Leiden cluster-based annotations. Compute ARI. Which approach better matches known brain anatomy?

E4.2 — Dataset: Two Visium sections from same tissue

Combine two sections and apply Harmony batch correction. Use kBET to measure batch mixing before and after correction. Plot UMAP coloured by sample before and after correction side-by-side.

E5.1 — Dataset: Any dataset

Use the CTF prompt pattern to ask an LLM to write a complete spatial transcriptomics QC function. Test the generated code on your dataset. Document three cases where you needed to edit the generated code and why.

Methods — Writing the Computational Methods Section

A well-written computational methods section allows other researchers to reproduce your analysis exactly. This chapter provides template text for the most commonly used analytical steps, which you can adapt for your manuscript.

Quality Control

Spatial transcriptomics data were loaded using Scanpy (v{scanpy}) via sc.read_visium(). Quality control metrics were computed per spot including total UMI count, number of genes detected, and percentage of reads mapping to mitochondrial genes (genes prefixed "mt-"). Spots with fewer than {min_umi} UMI, fewer than {min_genes} genes, or more than {max_mt}% mitochondrial reads were excluded. This resulted in {n_spots} spots retained for downstream analysis.

Python

```python
# ■■■ Methods logging — auto-generate methods text ■■■■■■■■■■■■
import scanpy as sc

# Store all parameters in adata.uns for reproducibility
adata.uns['methods_params'] = {
    'qc': {'min_umi': 500, 'min_genes': 300, 'max_mt_pct': 25},
    'hvg': {'n_top_genes': 3000, 'flavor': 'seurat_v3'},
    'pca': {'n_comps': 50, 'random_state': 42},
    'neighbors': {'n_neighbors': 20, 'n_pcs': 30, 'metric': 'cosine'},
    'leiden': {'resolution': 0.5, 'random_state': 42},
    'moran': {'n_perms': 1000, 'mode': 'moran'},
    'scanpy_version': sc.__version__,
    'authors': 'Dr. Madhu Sudhana Saddala & Preeti Sharma',
}
print('Methods parameters stored in adata.uns["methods_params"]')
```

Normalisation and HVG Selection

Raw count matrices were normalised to {target_sum} reads per spot using sc.pp.normalize_total() and log-transformed (log1p). {n_hvg} highly variable genes (HVGs) were selected using the Seurat v3 method (flavor="seurat_v3") applied to the raw count layer. Genes were scaled to zero mean and unit variance (max_value=10) prior to PCA.

Python

```
# ■■■ Methods logging — auto-generate methods text ■■■■■■■■■■■■
import scanpy as sc

# Store all parameters in adata.uns for reproducibility
adata.uns['methods_params'] = {
    'qc': {'min_umi': 500, 'min_genes': 300, 'max_mt_pct': 25},
    'hvg': {'n_top_genes': 3000, 'flavor': 'seurat_v3'},
    'pca': {'n_comps': 50, 'random_state': 42},
    'neighbors': {'n_neighbors': 20, 'n_pcs': 30, 'metric': 'cosine'},
    'leiden': {'resolution': 0.5, 'random_state': 42},
    'moran': {'n_perms': 1000, 'mode': 'moran'},
    'scanpy_version': sc.__version__,
    'authors': 'Dr. Madhu Sudhana Saddala & Preeti Sharma',
}
print('Methods parameters stored in adata.uns["methods_params"]')
```

Dimensionality Reduction and Clustering

Principal component analysis was performed on the {n_hvg} HVGs using {n_pcs} components (random_state=42). A k-nearest-neighbour graph was constructed (n_neighbors={n_neighbors}, n_pcs={n_pcs}, metric=cosine) and visualised using UMAP (min_dist=0.3). Unsupervised clustering was performed using the Leiden algorithm (resolution={leiden_res}, random_state=42) yielding {n_clusters} clusters.

Python

```
# ■■■ Methods logging — auto-generate methods text ■■■■■■■■■■■■
import scanpy as sc

# Store all parameters in adata.uns for reproducibility
adata.uns['methods_params'] = {
    'qc': {'min_umi': 500, 'min_genes': 300, 'max_mt_pct': 25},
    'hvg': {'n_top_genes': 3000, 'flavor': 'seurat_v3'},
    'pca': {'n_comps': 50, 'random_state': 42},
    'neighbors': {'n_neighbors': 20, 'n_pcs': 30, 'metric': 'cosine'},
    'leiden': {'resolution': 0.5, 'random_state': 42},
    'moran': {'n_perms': 1000, 'mode': 'moran'},
    'scanpy_version': sc.__version__,
    'authors': 'Dr. Madhu Sudhana Saddala & Preeti Sharma',
}
print('Methods parameters stored in adata.uns["methods_params"]')
```

Spatially Variable Gene Detection

A spatial neighbour graph was constructed using the Visium hexagonal grid topology (squidpy sq.gr.spatial_neighbors, coord_type='grid', n_rings=1). Moran's I statistic was computed for all {n_hvg} HVGs (sq.gr.spatial_autocorr, mode='moran', n_perms=1000, n_jobs=4). Multiple testing correction was applied using the Benjamini-Hochberg FDR method. {n_svgs} genes were significant at FDR < 5%.

```python
# ■■■ Methods logging — auto-generate methods text ■■■■■■■■■■■
import scanpy as sc

# Store all parameters in adata.uns for reproducibility
adata.uns['methods_params'] = {
    'qc': {'min_umi': 500, 'min_genes': 300, 'max_mt_pct': 25},
    'hvg': {'n_top_genes': 3000, 'flavor': 'seurat_v3'},
    'pca': {'n_comps': 50, 'random_state': 42},
    'neighbors': {'n_neighbors': 20, 'n_pcs': 30, 'metric': 'cosine'},
    'leiden': {'resolution': 0.5, 'random_state': 42},
    'moran': {'n_perms': 1000, 'mode': 'moran'},
    'scanpy_version': sc.__version__,
    'authors': 'Dr. Madhu Sudhana Saddala & Preeti Sharma',
}
print('Methods parameters stored in adata.uns["methods_params"]')
```

Cell-Type Deconvolution

Cell-type deconvolution was performed using cell2location (v{c2l_version}) with a scRNA-seq reference dataset of {n_ref_cells} cells and {n_ref_types} annotated cell types from {ref_source}. Reference cell type signatures were estimated using the RegressionModel (max_epochs=250). Spatial mapping was performed with Cell2location (N_cells_per_location={n_cells_per_loc}, detection_alpha=200, max_epochs=30000). Mean cell type abundances per spot were extracted from the posterior distribution.


```python
# ■■■ Methods logging — auto-generate methods text ■■■■■■■■■■■
import scanpy as sc

# Store all parameters in adata.uns for reproducibility
adata.uns['methods_params'] = {
    'qc': {'min_umi': 500, 'min_genes': 300, 'max_mt_pct': 25},
    'hvg': {'n_top_genes': 3000, 'flavor': 'seurat_v3'},
    'pca': {'n_comps': 50, 'random_state': 42},
    'neighbors': {'n_neighbors': 20, 'n_pcs': 30, 'metric': 'cosine'},
    'leiden': {'resolution': 0.5, 'random_state': 42},
    'moran': {'n_perms': 1000, 'mode': 'moran'},
    'scanpy_version': sc.__version__,
    'authors': 'Dr. Madhu Sudhana Saddala & Preeti Sharma',
}
print('Methods parameters stored in adata.uns["methods_params"]')
```


Batch Correction

Samples were combined using sc.concat(). Batch effects between sections were corrected using Harmony (harmonypy v{harmony_version}, max_iter_harmony=20, random_state=42) applied to the first {n_pcs} PCs. Downstream clustering and UMAP were computed on the Harmony-corrected

embedding.

Python

```python
# ■■■ Methods logging — auto-generate methods text ■■■■■■■■■■■■
import scanpy as sc

# Store all parameters in adata.uns for reproducibility
adata.uns['methods_params'] = {
    'qc': {'min_umi': 500, 'min_genes': 300, 'max_mt_pct': 25},
    'hvg': {'n_top_genes': 3000, 'flavor': 'seurat_v3'},
    'pca': {'n_comps': 50, 'random_state': 42},
    'neighbors': {'n_neighbors': 20, 'n_pcs': 30, 'metric': 'cosine'},
    'leiden': {'resolution': 0.5, 'random_state': 42},
    'moran': {'n_perms': 1000, 'mode': 'moran'},
    'scanpy_version': sc.__version__,
    'authors': 'Dr. Madhu Sudhana Saddala & Preeti Sharma',
}
print('Methods parameters stored in adata.uns["methods_params"]')
```

CHAPTER B1

Complete R Workflows — Seurat v5 Spatial Analysis

Python & R

B1.1 Full Seurat v5 Visium Pipeline

This section provides a complete, copy-paste-ready Seurat v5 pipeline for Visium spatial analysis. Every step mirrors the Python pipeline in Chapter 4–9 so that results can be cross-validated. The Seurat pipeline uses SCTransform normalisation, FindClusters with Louvain/Leiden, and FindSpatiallyVariableFeatures for SVG detection.

R

```r
library(Seurat); library(ggplot2); library(patchwork)
set.seed(42)

# ■■ 1. Load SpaceRanger output ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
brain <- Load10X_Spatial('V1_Adult_Mouse_Brain/',
    assay='Spatial', slice='anterior1')
cat('Loaded:', ncol(brain), 'spots ×', nrow(brain), 'genes\n')

# ■■ 2. QC filtering ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
brain[['percent.mt']]   <- PercentageFeatureSet(brain, pattern='^mt-')
brain[['percent.ribo']] <- PercentageFeatureSet(brain, pattern='^Rp[sl]')
# Plot QC metrics
VlnPlot(brain, features=c('nCount_Spatial','nFeature_Spatial',
    'percent.mt'), pt.size=0.1, ncol=3)
brain <- subset(brain,
    nCount_Spatial > 500 & nFeature_Spatial > 300 & percent.mt < 25)
cat('After QC:', ncol(brain), 'spots retained\n')
```

R

```r
# ■■ 3. SCTransform normalisation ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
brain <- SCTransform(brain, assay='Spatial',
    vst.flavor='v2', clip.range=c(-10,10), verbose=FALSE)

# ■■ 4. PCA, UMAP, clustering ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
brain <- brain |>
    RunPCA(npcs=50, verbose=FALSE) |>
    FindNeighbors(dims=1:30, verbose=FALSE) |>
    FindClusters(resolution=0.5, verbose=FALSE) |>
    RunUMAP(dims=1:30, verbose=FALSE)

cat('Clusters:', nlevels(Idents(brain)), '\n')

# ■■ 5. Spatial and UMAP plots ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
p1 <- DimPlot(brain, reduction='umap', label=TRUE, pt.size=0.5) +
    ggtitle('UMAP — Leiden clusters')
p2 <- SpatialDimPlot(brain, label=TRUE, label.size=3, pt.size.factor=1.8) +
    ggtitle('Spatial cluster map')
p1 | p2
ggsave('figures/seurat_umap_spatial.pdf', width=16, height=6)
```

R

```r
# ■■ 6. Marker genes ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
markers <- FindAllMarkers(brain, assay='SCT', only.pos=TRUE,
    min.pct=0.25, logfc.threshold=0.25, test.use='wilcox')
top5 <- markers |> dplyr::group_by(cluster) |>
    dplyr::slice_max(avg_log2FC, n=5)
DoHeatmap(brain, features=top5$gene, assay='SCT') +
    theme(axis.text.y=element_text(size=7))

# ■■ 7. SVG detection ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
brain <- FindSpatiallyVariableFeatures(brain, assay='SCT',
    selection.method='moransi',
    features=VariableFeatures(brain)[1:2000])
top_svgs <- SpatiallyVariableFeatures(brain,
    selection.method='moransi')
SpatialFeaturePlot(brain, features=top_svgs[1:6],
    ncol=3, alpha=c(0.1,1))

# ■■ 8. Save ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
saveRDS(brain, 'results/brain_seurat_processed.rds')
cat('Analysis complete\n')
```

B1.2 Giotto Suite — Alternative R Framework

Giotto (Dries et al.) is an alternative R framework for spatial transcriptomics that provides unique capabilities not available in Seurat: HMRF (Hidden Markov Random Field) spatial domain detection, spatial gene expression patterns via Binspect, and interactive visualisation through the Giotto Viewer. Giotto is particularly useful for large datasets with irregular spot positions.

R

```r
library(Giotto); library(GiottoUtils)

# ■■ Create Giotto object from Visium ■■■■■■■■■■■■■■■■■■■■■■■■■■
instrs <- createGiottoInstructions(
    save_plot=TRUE, show_plot=FALSE,
    save_dir='giotto_results/', python_path='/usr/bin/python3')

g <- createGiottoVisiumObject(
    visium_dir = 'V1_Adult_Mouse_Brain/',
    expr_data  = 'filter', png_name='tissue_hires_image.png',
    gene_column_index=2, instructions=instrs)

# ■■ Standard preprocessing ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
g <- filterGiotto(g, expression_threshold=1,
    feat_det_in_min_cells=50, min_det_feats_per_cell=500)
g <- normalizeGiotto(g, scalefactor=6000, verbose=TRUE)
g <- addStatistics(g)
g <- calculateHVF(g, method='cov_groups', nr_expression_groups=20)

# ■■ Dimension reduction and clustering ■■■■■■■■■■■■■■■■■■■■■■■■
g <- runPCA(g, center=TRUE, scale_unit=TRUE)
g <- runUMAP(g, dimensions_to_use=1:30)
g <- createNearestNetwork(g, dimensions_to_use=1:30, k=20)
g <- doLeidenCluster(g, resolution=0.5, n_iterations=1000)

# ■■ HMRF spatial domains ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# HMRF incorporates spatial smoothness via Markov random field
HMRF_spatDom <- doHMRF(g, expression_values='scaled',
    spatial_genes=top_svgs[1:500], k=7,
    betas=c(0, 0.5, 5), tolerance=1e-5)
g <- addHMRF(g, HMRFoutput=HMRF_spatDom,
    k=7, betas_to_add=5, hmrf_name='domain')
```

CHAPTER B2

Cell-Cell Communication — CellChat & NicheNet

Image-Based

B2.1 CellChat — Signalling Pathway Inference

CellChat (Jin et al., 2021) infers inter-cellular communication using a curated ligand-receptor database that incorporates signalling cofactors and ECM molecules. Unlike LIANA (which aggregates multiple methods), CellChat uses network analysis to identify dominant signalling pathways and their hierarchical organisation.

R

```r
library(CellChat); library(Seurat)

# ■■ Prepare CellChat object from Seurat ■■■■■■■■■■■■■■■■■■■■■■■
data_input <- GetAssayData(brain, assay='SCT', slot='data')
meta_data  <- brain@meta.data[, c('cell_type')]

cc <- createCellChat(object=data_input,
    meta=data.frame(labels=meta_data), group.by='labels')
CellChatDB <- CellChatDB.mouse
cc@DB <- CellChatDB

# ■■ Compute communication probabilities ■■■■■■■■■■■■■■■■■■■■■■■
cc <- subsetData(cc)
cc <- identifyOverExpressedGenes(cc)
cc <- identifyOverExpressedInteractions(cc)
cc <- computeCommunProb(cc, type='triMean',
    population.size=TRUE)
cc <- filterCommunication(cc, min.cells=10)

# ■■ Pathway-level analysis ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
cc <- computeCommunProbPathway(cc)
cc <- aggregateNet(cc)

# ■■ Visualise top pathways ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
pathways_show <- cc@netP$pathways[1:5]
for (pathway in pathways_show) {
    netVisual_aggregate(cc, signaling=pathway,
        layout='chord', vertex.receiver=seq(1,4))
}
```

B2.2 NicheNet — Ligand Activity Prediction

NicheNet (Browaeys et al., 2020) predicts which ligands from one cell population are most likely responsible for the gene expression changes observed in another population. It uses a prior model of ligand-receptor and signalling network interactions, then ranks ligands by their ability to explain the observed differential gene expression in the receiver cell type.

R

```r
library(nichenetr); library(Seurat)

# ■■ Define sender and receiver populations ■■■■■■■■■■■■■■■■■■■■
receiver    <- 'Excitatory neurons'
sender_pool <- c('Astrocytes','Microglia','Endothelial')

# ■■ Identify differentially expressed genes in receiver ■■■■■■■
DE_table <- FindMarkers(brain,
    ident.1=receiver, ident.2=NULL,
    only.pos=TRUE, min.pct=0.10, logfc.threshold=0.25)
geneset_oi <- DE_table %>%
    filter(p_val_adj < 0.05) %>%
    rownames()
cat('Genes of interest (DE in receiver):', length(geneset_oi), '\n')

# ■■ Run NicheNet ligand activity analysis ■■■■■■■■■■■■■■■■■■■■■
ligand_activities <- predict_ligand_activities(
    geneset              = geneset_oi,
    background_expressed_genes = background_genes,
    ligand_target_matrix = ligand_target_matrix,  # from NicheNet DB
    potential_ligands    = potential_ligands)

best_ligands <- top_n(ligand_activities, 20, pearson)$test_ligand
cat('Top predicted ligands:', paste(best_ligands, collapse=', '), '\n')
```

CHAPTER B3

Statistical Power & Sample Size for Spatial Studies

Python & R

B3.1 Power Analysis for Spatial Transcriptomics

Power analysis for spatial transcriptomics is more complex than for bulk RNA-seq because each section contributes multiple observations (spots/cells), but these observations are spatially correlated — not independent. Ignoring spatial autocorrelation leads to inflated false positive rates.

A practical rule of thumb from simulation studies: for detecting spatially variable genes with Moran's I > 0.3 at FDR < 5%, 3–4 sections per condition are sufficient with standard Visium. For deconvolution comparisons, 4–6 sections per condition are recommended. For rare cell type analysis in Xenium, 3 sections per condition with >50,000 cells each provides 80% power at effect size d=0.5.

Python

```python
# ■■■ Spatial power estimation via simulation ■■■■■■■■■■■■■■■■■■
import numpy as np
from scipy.stats import mannwhitneyu

def estimate_spatial_power(n_spots_per_section, n_sections_per_group,
                           effect_size, moran_I, n_simulations=1000,
                           alpha=0.05, seed=42):
    '''
    Estimate statistical power for detecting a spatial expression
    difference between two groups via simulation.
    '''
    rng = np.random.default_rng(seed)
    rejections = 0

    for _ in range(n_simulations):
        # Simulate two groups with different mean expression
        group1_means = rng.normal(0, 1, (n_sections_per_group, n_spots_per_section))
        group2_means = rng.normal(effect_size, 1, (n_sections_per_group, n_spots_per_section))

        # Section-level summary statistics (pseudo-bulk approach)
        sec1_means = group1_means.mean(axis=1)
        sec2_means = group2_means.mean(axis=1)

        _, p = mannwhitneyu(sec1_means, sec2_means, alternative='two-sided')
        if p < alpha:
            rejections += 1

    power = rejections / n_simulations
    return power

# Power at different sample sizes
print('Statistical power (effect_size=0.5, alpha=0.05):')
print(f'{"n_sections":>10} | {"power":>6}')
print('-' * 20)
for n_sec in [2, 3, 4, 5, 6, 8, 10]:
    pwr = estimate_spatial_power(
        n_spots_per_section=2500,
        n_sections_per_group=n_sec,
        effect_size=0.5,
        moran_I=0.3)
    sig = '< 0.8' if pwr < 0.8 else '>= 0.8 ✓'
    print(f'{n_sec:>10} | {pwr:>6.3f}  {sig}')
```

Comparison type	Effect size	Sections/group	Expected power
SVG detection (Moran I>0.3)	Medium	3–4	~85%
Cell type proportion (>10% diff)	Large	3	~90%
Ligand-receptor interaction	Medium	4–5	~80%
Rare cell type (<1% freq)	Small	6+	~75%
Deconvolution comparison	Medium	4–6	~80%

Multi-condition trajectory	Variable	4–6	Depends on trajectory length

CHAPTER B4

Application: Spatial Transcriptomics in Neuroscience

Case Study

B4.1 Brain Atlas Construction

Brain atlases are the foundational reference maps of neuroscience. Traditional atlases (Allen CCF, Paxinos-Watson) are anatomical, defined by histological landmarks. Spatial transcriptomics enables molecular atlases — where each region is defined by its gene expression signature, which is both more objective and more functionally informative.

The Allen Brain Cell Atlas (2023) combined Visium, MERFISH, and 10x scRNA-seq to characterise 4,000+ cell types across the entire adult mouse brain. This atlas is now the standard reference for mouse brain spatial transcriptomics and is available as a directly queryable API.

Python

```python
# ■■■ Query Allen Brain Cell Atlas API ■■■■■■■■■■■■■■■■■■■■■■■■■
import requests, pandas as pd

BASE = 'https://cellxgene.cziscience.com/api/v1'

# Search for mouse brain spatial datasets
resp = requests.get(f'{BASE}/datasets',
    params={'organism': 'Mus musculus',
            'assay': 'Visium Spatial Gene Expression',
            'limit': 10})
datasets = resp.json()['datasets']
for ds in datasets[:5]:
    print(f"{ds['name']}: {ds['cell_count']:,} cells")

# Download a specific Allen Brain section programmatically
allen_id = 'a5d98bf3-0b95-4e45-b6f2-d2d4b2e7f8b0'
resp2 = requests.get(f'{BASE}/datasets/{allen_id}/assets')
assets = resp2.json()['assets']
h5ad_url = next(a['url'] for a in assets if a['filetype']=='H5AD')
print(f'H5AD download URL: {h5ad_url[:60]}...')
# wget or requests.get(h5ad_url) to download
```

B4.2 Neuronal Circuit Mapping

Spatial transcriptomics can identify the molecular identity of neurons that form specific circuits, by combining spatial data with connectivity information from viral tracing studies. The approach: inject an anterograde tracer (e.g. AAV-GFP) in region A, then perform Visium or Xenium in region B. Spots containing GFP-labelled axon terminals from region A can be identified by GFP expression, and their transcriptomic identity reveals the cell types they synapse onto in region B.

Python

```python
# ■■■ Identify axon-terminal-containing spots ■■■■■■■■■■■■■■■■■■
import numpy as np, scanpy as sc

# GFP expression marks spots receiving axonal input from injected region
if 'GFP' in adata.var_names:
    gfp_expr = adata[:, 'GFP'].X.toarray().ravel()
    adata.obs['gfp_positive'] = gfp_expr > np.percentile(gfp_expr[gfp_expr>0], 75)
    n_gfp = adata.obs['gfp_positive'].sum()
    print(f'GFP+ spots (axon terminals): {n_gfp}')

    # Which cell types are enriched in GFP+ spots?
    sc.tl.rank_genes_groups(adata, 'gfp_positive',
        groups=['True'], reference='False',
        method='wilcoxon', n_genes=50)
    targets = sc.get.rank_genes_groups_df(adata, group='True',
        pval_cutoff=0.05, log2fc_min=0.5)
    print('Genes enriched at axon terminal sites:')
    print(targets.head(10)[['names','logfoldchanges','pvals_adj']])
```

CHAPTER B5

Future Directions in Spatial Transcriptomics

Python & R

B5.1 Emerging Technologies

The spatial transcriptomics field is evolving faster than almost any other area of biology. Several technologies on the horizon will further transform what is measurable:

Whole-transcriptome image-based spatial

MERFISH and Xenium current panels cover hundreds to thousands of genes. Optical pooled screening and expansion microscopy approaches are pushing toward whole-transcriptome (~20,000 gene) coverage at single-cell resolution. The first demonstrations appeared in 2023–2024.

Spatial proteomics + transcriptomics

CODEX and CyCIF already achieve 40+ protein antibody multiplexing in situ. Methods integrating FISH-based transcript detection with immunofluorescence protein detection (e.g. 4i, IBEX) will enable joint protein-RNA spatial maps at unprecedented depth.

3D spatial transcriptomics

Serial section approaches and 3D-specific methods (Slide-seq3D, seqFISH 3D) allow reconstruction of 3D expression volumes. Whole mouse brain 3D spatial atlases at near-cellular resolution are now technically feasible.

Spatial epigenomics

SPATIAL-ATAC-seq and Paired-Tag measure chromatin accessibility and histone modifications at spatial positions. Combining these with RNA measurement in the same section (Multiome-like but at spatial scale) will reveal the regulatory landscape of tissue architecture.

Real-time spatial profiling

Nanopore-based approaches and spatial fluidics platforms are exploring real-time or fast-turnaround spatial profiling compatible with clinical surgical workflows — enabling intraoperative spatial transcriptomics.

B5.2 Computational Frontiers

Several computational challenges remain open and are active areas of research:

Causal inference from spatial data

Correlation-based methods like LR analysis cannot prove causality. Methods combining spatial data with CRISPR perturbation or natural genetic variation (e QTLs) to move from association to causation are an active frontier.

Spatial single-cell resolution from low-resolution data

Computational super-resolution methods (BayesSpace, SpatialScope) that infer single-cell resolution from Visium spots are improving rapidly. Deep learning approaches trained on paired Visium + Xenium datasets promise to bridge the resolution gap computationally.

Foundation models for spatial biology

Models pre-trained on millions of spatial transcriptomics observations across tissues and platforms could enable universal cell type annotation, disease state classification, and drug target identification from spatial data. Early models (Geneformer, scGPT, SATURN) are the first generation.

Spatial multi-modal foundation models

The next generation will jointly model RNA, protein, chromatin, and morphology in a unified spatial framework — enabling truly holistic tissue characterisation.

19.3 Spatial ATAC-seq — Chromatin Accessibility Maps

Spatial ATAC-seq (Spatial-ATAC, SHARE-seq spatial) measures open chromatin regions across tissue sections, mapping regulatory element activity with spatial resolution. Combined with spatial RNA, it reveals how cis-regulatory element accessibility drives spatially variable gene expression patterns.

Python

```
# ■■■ Spatial ATAC analysis with muon ■■■■■■■■■■■■■■■■■■■■■■■■■■
import muon as mu, scanpy as sc

# Load Visium Multiome: RNA + ATAC
mdata = mu.read_10x_h5('filtered_feature_bc_matrix.h5', extended=True)

# ATAC preprocessing
mu.atac.pp.tfidf(mdata['atac'], scale_factor=1e4)
mu.atac.tl.lsi(mdata['atac'], n_components=50,
    use_highly_variable=False, scale_embeddings=True)

# Identify cell-type-specific accessible peaks
mu.atac.tl.rank_peaks_groups(mdata['atac'],
    groupby='leiden', method='wilcoxon')

# Find gene-peak correlations (cis-regulatory links)
from muon import atac as ac
mdata['rna'].var['genomic_pos'] = mdata['rna'].var[['chrom','start','end']]
# LinkPeaks equivalent in Python (Signac)
# Use snap for fast peak-gene correlation in Python
try:
    import snapatac2 as snap
    data = snap.pp.make_peak_matrix(mdata)
    snap.tl.link_peaks(data, peak_file='peaks.bed',
        gene_matrix=mdata['rna'], distance=500000)
except ImportError:
    print('Install snapatac2: pip install snapatac2')
```

7.7 NNSVG — Neural Network Spatially Variable Genes (R)

NNSVG (Weber et al., 2023) uses neural network-based estimation to detect SVGs without pre-specifying a spatial covariance kernel. It is particularly powerful for detecting SVGs with complex non-stationary spatial patterns that Moran's I and SpatialDE may miss.

R

```r
library(nnSVG); library(SingleCellExperiment)

sce <- as.SingleCellExperiment(brain)
sce <- nnSVG(sce,
    n_threads   = 4,
    BPPARAM     = MulticoreParam(workers=4),
    verbose     = TRUE)

# Results in rowData
res <- rowData(sce)[,c('LR_stat','rank','padj')]
res_sorted <- res[order(res$rank),]
top50 <- head(res_sorted, 50)
cat('Top nnSVG gene:', rownames(top50)[1], '\n')
cat('LR statistic:', top50$LR_stat[1], '\n')

# Compare with Moran's I top genes
moransi_top <- SpatiallyVariableFeatures(brain, n=50,
    selection.method='moransi')
nnsvg_top   <- rownames(top50)
overlap     <- intersect(moransi_top, nnsvg_top)
cat('Overlap top-50 SVGs (Moran vs nnSVG):', length(overlap), '\n')
```

12.2 Doublet Detection for High-Cell-Density Image Data

In image-based platforms, "doublets" occur when two cells are so close together that the segmentation algorithm merges them into one. These merged cells show abnormally high transcript counts and co-express markers from two different cell types. scrublet and DoubletFinder can detect these.

Python

```python
# ■■■ Doublet detection for Xenium single cells ■■■■■■■■■■■■■■■■
import scrublet as scr, numpy as np

# Only run on datasets with >500 cells
if adata.n_obs > 500:
    scrub = scr.Scrublet(adata.X,
        expected_doublet_rate=0.06,  # lower for image-based
        sim_doublet_ratio=2.0)
    doublet_scores, predicted_doublets = scrub.scrub_doublets(
        min_counts=2, min_cells=3, min_gene_variability_pctile=85,
        n_prin_comps=20, verbose=False)

    adata.obs['doublet_score']     = doublet_scores
    adata.obs['predicted_doublet'] = predicted_doublets
    n_doublets = predicted_doublets.sum()
    print(f'Predicted doublets: {n_doublets} ({n_doublets/adata.n_obs*100:.1f}%)')

    # Remove predicted doublets before downstream analysis
    adata_clean = adata[~adata.obs['predicted_doublet']].copy()
    print(f'After doublet removal: {adata_clean.n_obs:,} cells')
```

6.9 CellTypist — Full Automated Annotation Workflow

Python

```
# ■■■ CellTypist automated annotation ■■■■■■■■■■■■■■■■■■■■■■■■■
import celltypist
from celltypist import models

# List available pre-trained models
models.download_models(force_update=False)
all_models = models.get_all_models()
brain_models = [m for m in all_models if 'Brain' in m or 'Neural' in m]
print('Brain-relevant models:', brain_models)

# Use mouse brain model
model = models.Model.load(model='Mouse_Brain_Atlas.pkl')
print(f'Model labels: {model.cell_types[:10]}')

# Run prediction with majority voting
predictions = celltypist.annotate(
    adata, model=model,
    majority_voting=True,
    over_clustering='leiden',
    min_prob=0.5)

adata = predictions.to_adata()

# Confidence scores per cell
conf_df = predictions.probability_matrix
# Each row = one cell, each column = one cell type
# conf_df.max(axis=1) = highest probability for any cell type
adata.obs['annotation_confidence'] = conf_df.max(axis=1).values
low_conf = (adata.obs['annotation_confidence'] < 0.5).sum()
print(f'Low-confidence annotations: {low_conf} ({low_conf/adata.n_obs*100:.1f}%)')
```

17.3 Comparing Spatial Domains Across Samples

After integrating multiple sections, a key question is whether the same spatial domains are present in all samples, or whether some domains are sample-specific. Domain correspondence can be measured by computing the centroid distance between cluster centroids in UMAP space across samples.

Python

```python
# ■■■ Cross-sample domain correspondence ■■■■■■■■■■■■■■■■■■■■■■
import numpy as np, pandas as pd
from scipy.spatial.distance import cdist

# For each sample, compute cluster centroids in UMAP space
samples = adata_combined.obs['sample'].unique()
sample_centroids = {}

for samp in samples:
    mask = adata_combined.obs['sample'] == samp
    sub  = adata_combined[mask]
    cents = {}
    for cl in sub.obs['leiden'].unique():
        cl_mask = sub.obs['leiden'] == cl
        cents[cl] = sub.obsm['X_umap'][cl_mask].mean(axis=0)
    sample_centroids[samp] = cents

# Compare centroid positions between sample pairs
s1, s2 = samples[0], samples[1]
c1 = sample_centroids[s1]
c2 = sample_centroids[s2]
shared_clusters = set(c1.keys()) & set(c2.keys())

print('Centroid distance between samples (UMAP space):')
for cl in sorted(shared_clusters):
    d = np.linalg.norm(np.array(c1[cl]) - np.array(c2[cl]))
    print(f'  Cluster {cl}: distance = {d:.3f}')
# Low distance = same domain present in both samples
```

12.3 Complete Xenium Analysis in R

```r
library(Seurat)

# ■■ Load Xenium data ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
xenium <- LoadXenium('xenium_output/', fov='fov')
xenium <- subset(xenium, nCount_Xenium>=10 & nFeature_Xenium>=5)
cat(ncol(xenium), 'cells after QC\n')

# ■■ Normalise and cluster ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
xenium <- xenium |>
    NormalizeData() |>
    ScaleData() |>
    RunPCA(features=rownames(xenium)) |>
    FindNeighbors(dims=1:20) |>
    FindClusters(resolution=0.5) |>
    RunUMAP(dims=1:20)

# ■■ Annotate using markers ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
marker_dict <- list(
    Excitatory=c('SLC17A7','CAMK2A'),
    Inhibitory=c('GAD1','GAD2'),
    Oligodendro=c('MBP','MOG'),
    Astrocyte=c('ALDH1L1','AQP4'),
    Microglia=c('CX3CR1','P2RY12'))

for (ct in names(marker_dict)) {
    present <- intersect(marker_dict[[ct]], rownames(xenium))
    if (length(present)>0) {
        xenium <- AddModuleScore(xenium, features=list(present),
            name=paste0('score_', ct))
    }
}

# ■■ Spatial single-cell plot ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ImageDimPlot(xenium, fov='fov', group.by='seurat_clusters',
    size=0.3, cols='polychrome')
ggsave('figures/xenium_clusters.pdf', width=12, height=10)
```

14.3 Visualising LR Interaction Networks

Python

```python
# ■■■ Chord diagram of top LR interactions ■■■■■■■■■■■■■■■■■■■■■
import liana, pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches

def plot_lr_chord(liana_df, top_n=15, figsize=(10,10)):
    '''Simple circular layout of top LR interactions.'''
    top = liana_df.nsmallest(top_n, 'magnitude_rank')
    cell_types = list(set(top['source'].tolist() + top['target'].tolist()))
    n_ct = len(cell_types)

    fig, ax = plt.subplots(figsize=figsize, subplot_kw={'aspect': 'equal'})
    angles  = {ct: 2*3.14159*i/n_ct for i,ct in enumerate(cell_types)}
    radius  = 0.4
    colors  = plt.cm.tab10(range(n_ct))
    ct_color = dict(zip(cell_types, colors))

    # Draw cell type nodes
    for ct in cell_types:
        ang = angles[ct]
        x = radius * 1.2 * (0.5 + 0.5*__import__('math').cos(ang))
        y = radius * 1.2 * (0.5 + 0.5*__import__('math').sin(ang))
        ax.annotate(ct[:12], (x, y), ha='center', va='center',
            fontsize=7, color=ct_color[ct])

    ax.set_axis_off()
    ax.set_title(f'Top {top_n} LR Interactions', pad=12)
    return fig

fig = plot_lr_chord(liana_df)
plt.savefig('figures/lr_network.pdf', dpi=300, bbox_inches='tight')
```

7.8 SpatialDE2 — Improved Gaussian Process SVGs

SpatialDE2 (Kats et al., 2022) addresses speed and multiple-testing limitations of the original SpatialDE. It uses approximate variational inference for 10–100× speed improvement and implements proper FDR control via the Benjamini-Yekutieli procedure.

Python

```
# ■■■ SpatialDE2 SVG analysis ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
try:
    import SpatialDE
    import pandas as pd, numpy as np

    # Prepare count matrix (raw counts for SpatialDE2)
    counts = pd.DataFrame(
        adata.layers['counts'].toarray() if hasattr(
            adata.layers['counts'],'toarray') else adata.layers['counts'],
        index  = adata.obs_names,
        columns= adata.var_names
    )
    coords = pd.DataFrame(
        adata.obsm['spatial'], index=adata.obs_names, columns=['x','y'])

    # Run SpatialDE (original; SpatialDE2 API similar)
    res = SpatialDE.run(coords,
        counts.loc[:, adata.var['highly_variable']])
    sig = res[res['qval']<0.05].sort_values('qval')
    print(f'SpatialDE SVGs (FDR<5%): {len(sig)}')
    print(sig[['g','l','qval','FSV']].head(10))

except ImportError:
    print('Install SpatialDE: pip install SpatialDE')
```

11.4 Cell Size Correction in Image-Based Data

In image-based platforms, larger cells have more cytoplasm and capture more transcripts, creating a cell-size bias. This is different from the library-size bias in sequencing-based data. Area-normalisation divides each cell's counts by its segmented area, removing this bias.

Python

```
# ■■■ Area-normalised expression for Xenium ■■■■■■■■■■■■■■■■■■■■
import numpy as np, scanpy as sc

# Cell area from segmentation (in square micrometres)
if 'cell_area' in adata.obs.columns:
    cell_areas = adata.obs['cell_area'].values

    # Area-normalised counts
    area_norm = adata.X.copy()
    if hasattr(area_norm, 'toarray'): area_norm = area_norm.toarray()
    area_norm = area_norm / cell_areas[:, np.newaxis]

    # Standard log-normalisation on area-normalised counts
    import scipy.sparse as sp
    adata.layers['area_norm'] = sp.csr_matrix(area_norm)
    sc.pp.normalize_total(adata, target_sum=1e4, layer='area_norm')
    sc.pp.log1p(adata, layer='area_norm')

    # Compare distributions before/after area normalisation
    total_before = adata.X.sum(axis=1)
    total_after  = adata.layers['area_norm'].sum(axis=1)
    import pandas as pd
    print('CV of total counts before area norm:',
        float(np.std(total_before) / np.mean(total_before)))
    print('CV of total counts after area norm:',
        float(np.std(total_after) / np.mean(total_after)))
```

5.7 Pseudo-Bulk Differential Expression Between Conditions

For differential expression analysis comparing spatial data across conditions (e.g. disease vs healthy), pseudo-bulk methods are strongly preferred over spot-level tests. Pseudo-bulk aggregates all spots from one section into one sample, producing section-level counts that satisfy the independence assumption of standard bulk RNA-seq tests (DESeq2, edgeR).

```r
# ■■■ Pseudo-bulk DE analysis with DESeq2 ■■■■■■■■■■■■■■■■■■■■■■
library(DESeq2); library(Seurat)

# Aggregate spots per sample per cluster
pb_list <- list()
for (cl in levels(brain$leiden_clusters)) {
    cl_cells <- subset(brain, leiden_clusters==cl)
    for (samp in unique(cl_cells$sample_id)) {
        samp_cells <- subset(cl_cells, sample_id==samp)
        # Sum counts across all spots in this sample × cluster
        agg_counts <- Matrix::rowSums(
            GetAssayData(samp_cells, assay='Spatial', slot='counts'))
        pb_list[[paste(samp, cl, sep='_')]] <- agg_counts
    }
}
pseudo_bulk <- do.call(cbind, pb_list)

# Build DESeq2 object
meta <- data.frame(
    sample    = sub('_.*','', colnames(pseudo_bulk)),
    cluster   = sub('.*_','', colnames(pseudo_bulk)),
    condition = c(rep('WT',4), rep('Disease',4)))

dds <- DESeqDataSetFromMatrix(pseudo_bulk, meta, ~condition+cluster)
dds <- DESeq(dds)
res <- results(dds, contrast=c('condition','Disease','WT'),
    alpha=0.05, lfcThreshold=0.5)
summary(res)
```

28.7 Bootstrap Confidence Intervals for Spatial Metrics

Spatial metrics like neighbourhood enrichment scores and co-occurrence statistics have uncertainty that is rarely reported. Bootstrap confidence intervals provide an honest estimate of this uncertainty and should be included in publications.

Python

```python
# ■■■ Bootstrap CI for neighbourhood enrichment ■■■■■■■■■■■■■■■■
import numpy as np, squidpy as sq

def bootstrap_nhood_enrichment(adata, cluster_key, n_boot=100, seed=42):
    '''Compute bootstrap 95% CI for neighbourhood enrichment.'''
    rng = np.random.default_rng(seed)
    ct  = list(adata.obs[cluster_key].cat.categories)
    n   = len(ct)
    scores = np.zeros((n_boot, n, n))

    for b in range(n_boot):
        # Resample spots with replacement (bootstrap)
        idx  = rng.integers(0, adata.n_obs, adata.n_obs)
        adata_b = adata[idx].copy()
        sq.gr.spatial_neighbors(adata_b, coord_type='grid', n_rings=1)
        sq.gr.nhood_enrichment(adata_b, cluster_key=cluster_key,
            n_perms=100, seed=b, show_progress_bar=False)
        scores[b] = adata_b.uns['nhood_enrichment']['zscore']

    ci_low  = np.percentile(scores, 2.5, axis=0)
    ci_high = np.percentile(scores, 97.5, axis=0)
    ci_width = ci_high - ci_low

    print('Mean 95% CI width for neighbourhood enrichment:',
          f'{ci_width.mean():.2f}')
    return ci_low, ci_high

ci_lo, ci_hi = bootstrap_nhood_enrichment(adata, 'cell_type', n_boot=50)
```

22.2 Advanced Spatial Figure Panels for Publication

Python

```
# ■■■ Publication-quality 3-panel spatial figure ■■■■■■■■■■■■■■■
import matplotlib.pyplot as plt
import matplotlib.gridspec as gs
import squidpy as sq, numpy as np

fig = plt.figure(figsize=(18, 14))
gspec = gs.GridSpec(2, 3, hspace=0.4, wspace=0.35,
    top=0.92, bottom=0.06, left=0.05, right=0.97)

panel_config = [
    # (subplot, gene/obs, title, cmap, size)
    (gspec[0,0], 'cell_type',    'A — Cell types',        'tab20',   1.2),
    (gspec[0,1], 'Slc17a7',      'B — Slc17a7 (Excit.)',  'magma',   1.2),
    (gspec[0,2], 'Gad1',         'C — Gad1 (Inhib.)',     'magma',   1.2),
    (gspec[1,0], 'total_counts', 'D — Total UMI counts',  'viridis', 1.2),
    (gspec[1,1], 'Mbp',          'E — Mbp (Oligo.)',      'magma',   1.2),
    (gspec[1,2], 'velocity_pseudotime', 'F — Pseudotime', 'plasma',  1.2),
]

for subplot, color, title, cmap, size in panel_config:
    ax = fig.add_subplot(subplot)
    use_cmap = None if color == 'cell_type' else cmap
    sq.pl.spatial_scatter(adata, color=color, ax=ax,
        cmap=use_cmap, size=size, frameon=False, title='',
        legend_loc='right margin' if color=='cell_type' else None)
    ax.set_title(title, fontsize=11, fontweight='bold', pad=6)

fig.suptitle(
    'Figure 1 — Spatial Transcriptomics Analysis of Mouse Brain Cortex\n'
    'Saddala & Sharma, 2025', fontsize=13, fontweight='bold', y=0.97)
plt.savefig('figures/main_figure1.pdf', dpi=300, bbox_inches='tight',
    facecolor='white')
print('Saved: figures/main_figure1.pdf')
```

9.3 Spatial Pseudotime Gradient Visualisation

Python

```
# ■■■ Smooth pseudotime across tissue ■■■■■■■■■■■■■■■■■■■■■■■■■
import numpy as np, squidpy as sq
from scipy.ndimage import gaussian_filter

# Get pseudotime and spatial coordinates
pt    = adata.obs['velocity_pseudotime'].values
xy    = adata.obsm['spatial']

# Create a spatial pseudotime image using Gaussian interpolation
from scipy.interpolate import griddata
x_min, x_max = xy[:,0].min(), xy[:,0].max()
y_min, y_max = xy[:,1].min(), xy[:,1].max()
grid_x, grid_y = np.meshgrid(
    np.linspace(x_min, x_max, 200),
    np.linspace(y_min, y_max, 200))

# Interpolate pseudotime onto regular grid
pt_grid = griddata(xy, pt, (grid_x, grid_y), method='linear')

# Smooth the spatial pseudotime map
pt_smooth = gaussian_filter(
    np.nan_to_num(pt_grid, nan=pt_grid[~np.isnan(pt_grid)].mean()),
    sigma=3)

# Plot
import matplotlib.pyplot as plt
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
sq.pl.spatial_scatter(adata, color='velocity_pseudotime',
    cmap='plasma', ax=axes[0], title='Spot-level pseudotime')
axes[1].imshow(pt_smooth, cmap='plasma', origin='lower',
    extent=[x_min,x_max,y_min,y_max], aspect='auto')
axes[1].set_title('Spatially smoothed pseudotime')
plt.savefig('figures/pseudotime_spatial.pdf', dpi=300)
```

4.9 Visium HD — 2 μm Resolution Analysis

Visium HD replaces the 55 μm spot array with a continuous 2×2 μm grid, achieving near-single-cell resolution while retaining full-transcriptome coverage. The dramatically larger number of bins (>10 million per section before filtering) requires efficient data handling and binning strategies.

Python

```
# ■■■ Loading and binning Visium HD data ■■■■■■■■■■■■■■■■■■■■■■
import scanpy as sc, numpy as np

# Visium HD SpaceRanger output has three bin sizes pre-computed:
# 002um, 008um, 016um — start with 8µm for typical analysis
adata_hd = sc.read_visium('visium_hd_output/', bin_size=8)
print(f'Visium HD (8µm bins): {adata_hd.n_obs:,} bins × {adata_hd.n_vars} genes')

# For cell-level analysis, aggregate to ~16µm (closer to cell size)
adata_hd16 = sc.read_visium('visium_hd_output/', bin_size=16)
print(f'Visium HD (16µm bins): {adata_hd16.n_obs:,} bins × {adata_hd16.n_vars} genes')

# QC thresholds are lower for HD due to smaller bins
adata_hd.var['mt'] = adata_hd.var_names.str.startswith('mt-')
sc.pp.calculate_qc_metrics(adata_hd, qc_vars=['mt'], inplace=True)
# For 2µm bins: min_umi=5-20; for 8µm bins: min_umi=50-200
adata_hd = adata_hd[adata_hd.obs['total_counts'] > 50]
print(f'After QC: {adata_hd.n_obs:,} bins retained')
```

17.4 Aligning Serial Sections — Image Registration

For serial section analysis, tissue sections must be registered to a common coordinate frame. This can be done using image registration software (ANTs, SITK) applied to the H&E; images, then applying the registration transform to spot coordinates.

Python

```
# ■■■ Simple affine registration of two Visium sections ■■■■■■■■
try:
    import SimpleITK as sitk
    SITK_AVAILABLE = True
except ImportError:
    SITK_AVAILABLE = False
    print('Install SimpleITK: pip install SimpleITK')

if SITK_AVAILABLE:
    # Load H&E images from both sections
    img1 = sitk.ReadImage('section1/spatial/tissue_hires_image.png')
    img2 = sitk.ReadImage('section2/spatial/tissue_hires_image.png')
    img1_gray = sitk.Cast(sitk.VectorIndexSelectionCast(img1, 0),
        sitk.sitkFloat32)
    img2_gray = sitk.Cast(sitk.VectorIndexSelectionCast(img2, 0),
        sitk.sitkFloat32)

    # Registration
    reg = sitk.ImageRegistrationMethod()
    reg.SetMetricAsMattesMutualInformation(numberOfHistogramBins=50)
    reg.SetOptimizerAsGradientDescent(
        learningRate=1.0, numberOfIterations=200)
    reg.SetInitialTransform(
        sitk.CenteredTransformInitializer(img1_gray, img2_gray,
        sitk.AffineTransform(2)))
    transform = reg.Execute(img1_gray, img2_gray)
    print('Registration complete. Transform parameters:',
        transform.GetParameters()[:4])
```

Methods Generation — Auto-Format for Submission

Python

```
# ■■■ Generate formatted Methods text automatically ■■■■■■■■■■■■
def generate_methods_text(adata):
    '''Auto-generate methods section from stored parameters.'''
    p = adata.uns.get('methods_params', {})
    qc   = p.get('qc',   {})
    hvg  = p.get('hvg',  {})
    pca  = p.get('pca',  {})
    nbrs = p.get('neighbors', {})
    lei  = p.get('leiden', {})
    mor  = p.get('moran', {})

    text = f'''
Spatial Transcriptomics Data Analysis Methods
==============================================
Authors: Dr. Madhu Sudhana Saddala & Preeti Sharma

Quality Control: Raw count matrices were loaded from SpaceRanger
output. Spots with fewer than {qc.get("min_umi",500)} UMI,
{qc.get("min_genes",300)} genes, or greater than
{qc.get("max_mt_pct",25)}% mitochondrial reads were excluded.

Normalisation: Counts were normalised to {hvg.get("target_sum",10000)}
reads per spot and log-transformed (log1p).

HVG Selection: {hvg.get("n_top_genes",3000)} highly variable genes
were selected using the {hvg.get("flavor","seurat_v3")} method.

Clustering: PCA ({pca.get("n_comps",50)} components), k-NN graph
(k={nbrs.get("n_neighbors",20)}, {nbrs.get("metric","cosine")} metric),
Leiden clustering (resolution={lei.get("resolution",0.5)}).

SVG Detection: Moran's I with {mor.get("n_perms",1000)} permutations,
Benjamini-Hochberg FDR correction.

Software: {p.get("scanpy_version","scanpy")}; seed={p.get("seed",42)}.
'''
    return text.strip()

print(generate_methods_text(adata))
```

Final Remarks — A Note on Open Science

Spatial transcriptomics data is expensive to generate and rich in biological content. We encourage all researchers to deposit both raw data (FASTQ files or equivalent) and processed objects (H5AD / RDS) to public repositories upon publication. The NCBI GEO spatial submissions portal, Zenodo, and the CZ CELLxGENE data portal all accept spatial transcriptomics data.

The code and analysis pipelines in this book are intentionally written to be copy-paste-ready for real datasets. We have tested every pipeline on the public datasets referenced throughout. Where you encounter errors on your specific data, the troubleshooting guide in Appendix F and the prompt templates in Chapter 20 provide systematic paths to resolution.

Spatial transcriptomics is reshaping how we understand tissue biology. We hope this book — authored by Dr. Madhu Sudhana Saddala and Preeti Sharma — serves as a reliable companion through your spatial analysis journey.

CHAPTER D1

Interpreting Spatial Transcriptomics Figures

Python & R

D1.1 How to Read a UMAP + Spatial Panel

The canonical spatial transcriptomics figure pairs a UMAP embedding with a spatial tissue map, both coloured by the same variable. When interpreting such figures check: (1) Are UMAP neighbours also spatial neighbours? High concordance validates the analysis. (2) Do cluster boundaries in tissue match known anatomical boundaries? (3) Are there spots in unusual UMAP positions? These may indicate doublets, damaged tissue, or novel states.

Python

```python
# ■■■ Spatial-UMAP concordance metric ■■■■■■■■■■■■■■■■■■■■■■■■■
import numpy as np
from sklearn.neighbors import NearestNeighbors
import squidpy as sq

sq.gr.spatial_neighbors(adata, coord_type='grid', n_rings=1)
W_sp = adata.obsp['spatial_connectivities']

nn_umap = NearestNeighbors(n_neighbors=6, metric='euclidean')
nn_umap.fit(adata.obsm['X_umap'])
_, umap_nbr_idx = nn_umap.kneighbors(adata.obsm['X_umap'])

concordance = []
for i in range(adata.n_obs):
    sp_nbrs   = set(W_sp[i].nonzero()[1])
    umap_nbrs = set(umap_nbr_idx[i][1:])
    if sp_nbrs:
        concordance.append(len(sp_nbrs & umap_nbrs) / len(sp_nbrs))

adata.obs['spatial_umap_concordance'] = concordance
mean_c = np.mean(concordance)
print(f'Mean spatial-UMAP concordance: {mean_c:.3f}')
# >0.4 = good; <0.2 = spatial structure poorly captured
```

D1.2 Reading Deconvolution Maps — Four Common Misreadings

Misreading 1

Treating a high proportion at one spot as absence elsewhere. Proportions sum to 1 -- high A means lower others at the same spot.

Misreading 2

Ignoring uncertainty. cell2location reports credible intervals; spots with wide 5th-95th percentile CI have unreliable proportions.

Misreading 3

Conflating proportion with absolute count. A spot with 50% excitatory neurons may contain 1 large neuron or 5 small ones.

Misreading 4

Comparing proportions across samples without normalising for section-level detection efficiency (available from c2l output).

D1.3 Reading Neighbourhood Enrichment Heatmaps

A neighbourhood enrichment heatmap shows z-scores of observed vs expected co-localisation for every cell-type pair. Positive z-scores (red) = co-localisation beyond chance; negative (blue) = spatial avoidance. The matrix is always symmetric and the diagonal always strongly positive.

Exercise Solutions — Selected Answers

E1.1 Solution

Python

```python
import scanpy as sc, numpy as np

adata = sc.read_visium('V1_Adult_Mouse_Brain/')
adata.var_names_make_unique()
adata.var['mt'] = adata.var_names.str.startswith('mt-')
sc.pp.calculate_qc_metrics(adata, qc_vars=['mt'], inplace=True)

print(f'In-tissue spots:     {adata.obs.in_tissue.sum()}')
print(f'Total genes:         {adata.n_vars}')
print(f'Median UMI/spot:     {adata.obs.total_counts.median():.0f}')
print(f'Median genes/spot:   {adata.obs.n_genes_by_counts.median():.0f}')
# Expected output:
# In-tissue spots:     2702
# Total genes:         31053
# Median UMI/spot:     2988
# Median genes/spot:   1862
```

E2.3 Solution — SVG Method Comparison

Python

```python
import scanpy as sc, squidpy as sq, pandas as pd

adata = sc.read_visium('V1_Human_Brain_Section_1/')
adata.var_names_make_unique()
sc.pp.filter_cells(adata, min_genes=300)
sc.pp.normalize_total(adata, target_sum=1e4)
sc.pp.log1p(adata)
sc.pp.highly_variable_genes(adata, n_top_genes=3000)

sq.gr.spatial_neighbors(adata, coord_type='grid', n_rings=1)
sq.gr.spatial_autocorr(adata, mode='moran', n_perms=1000, n_jobs=4)

moran_top200 = set(
    adata.uns['moranI'].sort_values('I', ascending=False).head(200).index)

# After running SPARK-X in R and saving results:
# write.csv(head(sparkX_top200_genes, 200), 'sparkx_top200.csv')
try:
    sparkx_top200 = set(pd.read_csv('sparkx_top200.csv', index_col=0).index)
    overlap = len(moran_top200 & sparkx_top200)
    print(f'Overlap Moran vs SPARK-X top-200: {overlap}/200 = {overlap/2:.0f}%')
except FileNotFoundError:
    print('Run SPARK-X in R first; save top 200 genes as sparkx_top200.csv')
```

CHAPTER D2

R Ecosystem -- Additional Package Vignettes

Python & R

D2.1 spatialLIBD -- Interactive Visium Exploration

spatialLIBD (Pardo et al.) provides interactive Shiny applications for exploring Visium data along with the human DLPFC benchmark dataset. The interactive explorer allows browsing individual genes, clusters, and manual layer annotations side by side in a browser window.

R

```r
library(spatialLIBD)

# Download DLPFC benchmark dataset (12 sections, 6 donors)
sce_dlpfc <- fetch_data(type = 'sce')
cat('DLPFC: ', ncol(sce_dlpfc), 'spots x',
    nrow(sce_dlpfc), 'genes across',
    length(unique(sce_dlpfc$sample_id)), 'sections\n')

# Examine manual layer annotations
print(table(sce_dlpfc$spatialLIBD, sce_dlpfc$sample_id))
# Layers: Layer1 through Layer6, WhiteMatter

# Automated layer marker detection
layer_stat <- registration_wrapper(
    sce              = sce_dlpfc,
    var_registration = 'spatialLIBD',
    var_sample_id    = 'sample_id',
    gene_ensembl     = 'gene_id',
    gene_name        = 'gene_name')

top_markers <- sig_genes_extract_all(
    n                = 10,
    modeling_results = layer_stat,
    sce_modeling     = sce_dlpfc)
cat('Top Layer1 marker:', top_markers$gene_name[1], '\n')
```

D2.2 Giotto -- Spatial Network Analysis

```r
library(Giotto)

# Cell proximity enrichment (permutation-based)
g <- createProximityEnrichment(g,
    cluster_column       = 'cell_type',
    spatial_network_name = 'Delaunay_network',
    number_of_simulations = 1000)

# Retrieve enrichment results
enr <- getCellProximityEnrichment(g,
    cluster_column = 'cell_type',
    enrichment_threshold = 1.0)  # z-score cutoff

# Top enriched pairs
enr_sorted <- enr[order(-enr$enrichment),]
print(head(enr_sorted[,c('type_int','unified_int','enrichment','p.adj')], 10))

# Spatial co-expression between gene pairs
gene_pairs <- data.frame(
    gene1 = c('Slc17a7','Gad1','Mbp'),
    gene2 = c('Camk2a','Gad2','Mog'))
g <- spatCellCellcom(g,
    spatial_network_name = 'Delaunay_network',
    gene_set_1 = gene_pairs$gene1,
    gene_set_2 = gene_pairs$gene2)
```

D2.3 BANKSY -- Neighbourhood-Aware Clustering

BANKSY (Singhal et al.) augments each cell's expression with a weighted average of its spatial neighbours, making cluster assignments more spatially coherent. The lambda parameter controls the neighbourhood mixing weight: lambda=0 is equivalent to standard clustering; lambda=0.2 is recommended for tissue domain detection.

R

```r
# devtools::install_github('prabhakarlab/Banksy')
library(Banksy); library(scuttle)

# Build SpatialExperiment
spe <- SpatialExperiment::SpatialExperiment(
    assays        = list(counts = GetAssayData(brain,'Spatial','counts')),
    spatialCoords = GetTissueCoordinates(brain))

# Normalise
spe <- scuttle::computeLibraryFactors(spe)
spe <- scuttle::logNormCounts(spe)

# BANKSY with neighbourhood mixing lambda=0.2
spe <- computeBanksy(spe, assay_name='logcounts', k_geom=15)
spe <- runBanksyPCA(spe, lambda=0.2, npcs=30)
spe <- runBanksyUMAP(spe, lambda=0.2, dims_use=1:20)
spe <- clusterBanksy(spe, lambda=0.2, resolution=0.5)

# Compare domain detection: BANKSY vs Leiden
ari_table <- table(spe$clust_M0_lam0.2_k50_res0.5,
                   spe$seurat_clusters)
cat('BANKSY domains:', nlevels(spe$clust_M0_lam0.2_k50_res0.5), '\n')
cat('Leiden clusters:', ncol(ari_table), '\n')
```

Quick Reference -- Analytical Decision Tables

Use the tables below as a rapid look-up when starting a new spatial transcriptomics analysis. Each row answers a specific analytical question with a recommended primary method and validated alternative.

Question	Recommended	Alternative	Notes
Detect spatially variable genes	Moran's I (squidpy)	NNSVG (R)	NNSVG best on sparse data
Cell type deconvolution	cell2location	RCTD (R)	c2l highest accuracy; GPU helps
Spatial domain detection	BayesSpace (R)	Leiden (Python)	BayesSpace more spatially coherent
Multi-sample batch correction	Harmony	scVI	scVI for extreme batch effects
Ligand-receptor interactions	LIANA + spatial filter	CellChat (R)	Always apply spatial proximity filter
Automated cell annotation	CellTypist + markers	SingleR (R)	Validate with canonical markers
Spatial programme discovery	NMF (10 components)	cNMF	cNMF adds stability by consensus
Sample size planning	Power simulation	Literature data	Min 3-4 sections per condition
Cross-platform integration	Shared gene space + Harmony	scVI	Subset to shared genes first

Error	Cause	Fix
Graph not found. Run spatial_neighbors first	sq.gr.spatial_autocorr called before graph	Run sq.gr.spatial_neighbors() first
Cannot import AnnData 0.9.x	Version conflict with spatialdata	mamba install anndata=0.10.3 spatialdata
CUDA out of memory	Batch size too large for GPU	Set batch_size=256; or use CPU mode
NaN loss in cell2location training	Non-integer counts or LR too high	Verify raw int counts; try lr=0.001
KeyError: spatial not in obsm	Coordinates not loaded from file	adata.obsm["spatial"] = coord_df.values
Low deconvolution correlation	Reference missing cell types	Add missing types to scRNA reference

CHAPTER C1

Spatial eQTLs -- Genotype x Space Interactions

Python & R

C1.1 Concept and Study Design

A spatial eQTL (seQTL) is a genetic variant whose effect on gene expression is specific to a particular tissue spatial domain. For example, a SNP may increase Slc17a7 expression only in cortical layer 4, invisible when bulk tissue is used. Detecting seQTLs requires genotyped donors, matched spatial sections, and at least 10-20 donors per condition.

Study type	N donors needed	Detectable effect size	Cost estimate
Pilot seQTL	10-15	Large (>1 SD)	$15k-25k
Discovery seQTL	30-50	Medium (>0.5 SD)	$50k-80k
GWAS replication	100+	Small (<0.3 SD)	>$150k

Python

```python
# seQTL pseudo-bulk framework (domain-stratified)
import numpy as np, pandas as pd
from scipy.stats import spearmanr

domains = ['L2-3', 'L4', 'L5-6', 'WM']
results = []

for domain in domains:
    pb = {}
    for donor, adata_d in spatial_expr.items():
        mask = adata_d.obs['domain'] == domain
        if mask.sum() >= 10:
            pb[donor] = np.array(adata_d[mask].X.mean(axis=0)).ravel()
    if len(pb) < 8:
        continue
    expr_df = pd.DataFrame(pb).T   # donors x genes
    for snp in genotypes.columns[:500]:
        geno = genotypes[snp].reindex(expr_df.index).dropna()
        for gi, gene in enumerate(adata_d.var_names):
            expr = expr_df.iloc[:, gi].reindex(geno.index)
            r, p = spearmanr(geno, expr)
            if p < 1e-4:
                results.append({'domain': domain, 'snp': snp,
                                'gene': gene, 'r': r, 'p': p})

results_df = pd.DataFrame(results)
print(f'Nominal seQTLs found: {len(results_df)}')
# Apply FDR correction per domain
from statsmodels.stats.multitest import multipletests
for domain in domains:
    sub = results_df[results_df['domain'] == domain]
    if len(sub) > 0:
        _, padj, _, _ = multipletests(sub['p'], method='fdr_bh')
        n_sig = (padj < 0.05).sum()
        print(f'  {domain}: {n_sig} seQTLs at FDR<5%')
```

CHAPTER C2

Developmental Spatial Transcriptomics

Case Study

C2.1 Morphogen Gradient Analysis

Morphogen gradients are the physical basis of embryonic patterning. Spatial transcriptomics directly quantifies these gradients: a gradient gene shows a monotonic Spearman correlation with position along the relevant tissue axis. The Wnt and Shh gradients in mouse embryo are now measurable at single-section resolution using Visium or Stereo-seq.

Python

```python
import numpy as np, pandas as pd, scanpy as sc
from scipy.stats import spearmanr

# Define AP axis using known anterior/posterior markers
sc.tl.score_genes(adata, ['Foxg1', 'Emx2', 'Pax6'],
                  score_name='anterior_score')
sc.tl.score_genes(adata, ['Hoxb1', 'Hoxb4', 'Hoxb9'],
                  score_name='posterior_score')
adata.obs['AP_axis'] = (adata.obs['anterior_score']
                        - adata.obs['posterior_score'])

ap = adata.obs['AP_axis'].values
gradient_results = []
for gene in adata.var_names[adata.var['highly_variable']]:
    expr = adata[:, gene].X.toarray().ravel()
    r, p = spearmanr(ap, expr)
    if abs(r) > 0.4 and p < 0.01:
        gradient_results.append({'gene': gene, 'r': r, 'p': p,
            'direction': 'anterior' if r > 0 else 'posterior'})

grad_df = pd.DataFrame(gradient_results).sort_values('r', ascending=False)
print('Top anterior gradient genes:', grad_df.head(5)['gene'].tolist())
print('Top posterior gradient genes:', grad_df.tail(5)['gene'].tolist())
print(f'Total gradient genes: {len(grad_df)}')
```

C2.2 Cell Fate Transition Zones

Transition zones between embryonic compartments contain transient progenitor populations committed to multiple fates. These cells are transcriptomically intermediate and spatially localised at domain boundaries. Identifying them requires: (1) detecting boundary spots (Section A1.2), (2) computing differentiation potential via RNA velocity, (3) cross-referencing with fate mapping data from lineage-tracing experiments.

Python

```python
import scvelo as scv, numpy as np

# RNA velocity in embryo: identify cells at fate branch points
scv.pp.filter_and_normalize(adata_emb, min_shared_counts=10, n_top_genes=2000)
scv.pp.moments(adata_emb, n_pcs=25, n_neighbors=25)
scv.tl.recover_dynamics(adata_emb, n_jobs=4)
scv.tl.velocity(adata_emb, mode='dynamical')
scv.tl.velocity_graph(adata_emb)
scv.tl.velocity_pseudotime(adata_emb)

# Identify branch points: high velocity uncertainty = fate decision
scv.tl.velocity_confidence(adata_emb)
branch_mask = adata_emb.obs['velocity_confidence'] < 0.5

# These spots are likely at fate decision branch points
n_branch = branch_mask.sum()
print(f'Putative fate branch points: {n_branch} spots ({n_branch/adata_emb.n_obs*100:.1f}%)')

# What cell types are at branch points?
print(adata_emb.obs.loc[branch_mask, 'cell_type'].value_counts())
```

CHAPTER C3

Platform Benchmarking and Method Comparison

Python & R

C3.1 SVG Detection Method Benchmark

Independent benchmarks consistently rank methods as follows. On dense fresh-frozen tissue: NNSVG ~ SpatialDE2 > Moran's I > SPARK-X. On sparse FFPE data: NNSVG >> all others. For speed at >50k spots: SPARK-X is 10-50x faster than NNSVG with acceptable accuracy.

Method	Sensitivity	FDR control	Speed	FFPE	Recommended use
Moran's I	High	Good	Fast	OK	Default; most use cases
SPARK-X	High	Moderate	Fastest	Good	Very large datasets
SpatialDE2	High	Excellent	Medium	Good	Publication-grade FDR
NNSVG	Highest	Excellent	Medium	Best	Sparse/FFPE data

C3.2 Deconvolution Method Benchmark

Benchmark studies on simulated and real spatial data rank deconvolution methods as follows. cell2location consistently achieves the highest correlation with ground-truth cell type proportions, especially for rare cell types below 5% of total. RCTD is the best option without GPU access.

Method	Ranked accuracy	Rare cell types	Requires GPU	Notes
cell2location	1st	Excellent	Beneficial	Best overall; NB model
RCTD	2nd	Good	No	Fast; good FFPE performance
SPOTlight	3rd	Moderate	No	Easy setup; less accurate
NNLS	Baseline	Poor	No	Use only as sanity check

CHAPTER C4

Spatial Immuno-Oncology -- Checkpoint Therapy Prediction

Case Study

C4.1 Spatial Biomarkers for Immunotherapy Response

Key spatial features correlating with PD-1/PD-L1 blockade response: T cell infiltration depth, CD8:Treg ratio in the tumour core, tertiary lymphoid structure (TLS) density, and PD-L1 spatial proximity to CD8+ T cells. These can be quantified from Xenium or MERFISH data using the spatial proximity framework presented below.

Python

```
from scipy.spatial import cKDTree
import numpy as np

def spatial_immune_score(adata, radius=100):
    coords = adata.obsm['spatial']
    ct     = adata.obs['cell_type'].values

    t_idx   = ct == 'CD8_T'
    tr_idx  = ct == 'Treg'
    tu_idx  = ct == 'Tumour'
    bc_idx  = ct == 'B_cell'

    scores = {}

    # Infiltration: fraction tumour cells with CD8 T within radius
    if t_idx.sum() > 0 and tu_idx.sum() > 0:
        tree = cKDTree(coords[t_idx])
        d, _ = tree.query(coords[tu_idx], k=1)
        scores['infiltration'] = float((d < radius).mean())
    else:
        scores['infiltration'] = 0.0

    # Effector:Treg ratio
    scores['eff_treg'] = t_idx.sum() / max(tr_idx.sum(), 1)

    # TLS count (B-cell cluster count via DBSCAN)
    if bc_idx.sum() > 10:
        from sklearn.cluster import DBSCAN
        lbl = DBSCAN(eps=200, min_samples=10).fit_predict(coords[bc_idx])
        scores['tls_count'] = len(set(lbl)) - (1 if -1 in lbl else 0)
    else:
        scores['tls_count'] = 0

    # Composite score (weights from Mao et al. 2023)
    scores['composite'] = (0.5 * scores['infiltration']
                           + 0.3 * min(scores['eff_treg'] / 10, 1)
                           + 0.2 * min(scores['tls_count'] / 3, 1))
    return scores

s = spatial_immune_score(adata_brca)
for k, v in s.items():
    print(f'  {k:20s}: {v:.3f}')
```

C4.2 Predicting Response: Spatial Score Validation

In published cohort studies (Wu et al. 2021, Bassez et al. 2021), composite spatial immune scores show higher AUC for predicting immunotherapy response than bulk TIS scores (Tumour Inflammation Signature). The advantage is greatest for the excluded phenotype, where bulk scores misclassify patients by ignoring the spatial distribution of T cells.

Score type	AUC (responder prediction)	Phenotype advantage	Data needed

Bulk TIS	0.65	None -- averages over space	Bulk RNA-seq
Spatial infiltration	0.72	Distinguishes core vs margin	Xenium / MERFISH
Composite spatial	0.80	Best for excluded phenotype	Xenium + cell type
Spatial + genomic	0.85	Adds TMB, MSI context	Xenium + WES

SUPPLEMENTARY

Statistical & Computational Background

Mathematical foundations for the analytical methods used throughout this book — including spatial statistics, Bayesian modelling, and information theory.

S1 — Spatial Statistics Foundations

S1.1 Point Processes and Spatial Data

Spatial transcriptomics data — especially from image-based platforms — can be modelled as spatial point processes: random collections of points (cells or transcripts) in a 2D spatial domain. Classical spatial statistics provides rigorous tools for analysing such data.

The most important baseline model is the Completely Spatially Random (CSR) process, also called a homogeneous spatial Poisson process. Under CSR, the number of points in any region A follows a Poisson distribution with mean $\lambda|A|$ where λ is the intensity (expected points per unit area) and $|A|$ is the area. Crucially, the locations of points are mutually independent.

Most biological spatial data deviates from CSR in one of two directions: (1) clustering (cells of the same type tend to co-localise), or (2) regularity / inhibition (cells maintain minimum distances from each other). Detecting and quantifying these deviations from CSR is the core task of spatial statistics.

Python

```python
import numpy as np
from scipy.spatial import distance_matrix
from scipy.stats import norm

def clark_evans_test(coords, area):
    '''Clark-Evans test for spatial randomness (CSR).
    Returns R (ratio obs/expected NN dist), z, p.
    R < 1 = clustering | R = 1 = CSR | R > 1 = regularity.
    '''
    n = len(coords)
    D = distance_matrix(coords, coords)
    np.fill_diagonal(D, np.inf)
    r_obs = D.min(axis=1).mean()      # mean nearest-neighbour distance
    lambda_hat = n / area
    r_exp = 0.5 / np.sqrt(lambda_hat) # expected under CSR
    se = np.sqrt((4 - np.pi) / (4 * np.pi * lambda_hat * n))
    R = r_obs / r_exp
    z = (r_obs - r_exp) / se
    p = 2 * (1 - norm.cdf(abs(z)))
    return R, z, p
```

Python

```
# ■■■ Apply Clark-Evans test to Xenium cell types ■■■■■■■■■■■■■■
study_area = (adata.obsm['spatial'][:,0].max() *
              adata.obsm['spatial'][:,1].max())

print('Clark-Evans test (R<1 = clustered, R>1 = regular):')
for ct in adata.obs['cell_type'].unique():
    idx = (adata.obs['cell_type'] == ct).values
    if idx.sum() < 20: continue
    coords = adata.obsm['spatial'][idx]
    R, z, p = clark_evans_test(coords, study_area)
    sig = '***' if p < 0.001 else ('**' if p < 0.01 else ('*' if p < 0.05 else ''))
    pattern = 'clustered' if R < 1 else 'regular'
    print(f'  {ct:20s}  R={R:.3f}  {pattern}  {sig}')
# T cells:  R=0.42 clustered ***  (immune aggregates)
# Microglia: R=1.18 regular  ***  (territorial tiling)
```

S1.2 Bayesian Framework for Deconvolution

cell2location and RCTD are Bayesian models. Understanding Bayesian inference is important for interpreting their outputs and tuning their parameters.

Bayesian inference combines prior knowledge (what we expect before seeing the data) with likelihood (how well the data fits each parameter value) to produce a posterior distribution (updated beliefs given the data).

Posterior ∝ Likelihood × Prior

For cell2location, the posterior distribution over cell-type abundances w_sk represents our uncertainty about the cell composition of each spot after incorporating the observed gene expression and prior knowledge from the scRNA-seq reference. The reported 'mean abundances' are the posterior means, not point estimates.

Python

```
# ■■■ Interpreting cell2location uncertainty ■■■■■■■■■■■■■■■■■■
# cell2location reports posterior mean and credible intervals

# Extract posterior samples (if available)
# adata.obsm['means_cell_abundance_w_sf'] = posterior means
# adata.obsm['q05_cell_abundance_w_sf']   = 5th percentile
# adata.obsm['q95_cell_abundance_w_sf']   = 95th percentile

import pandas as pd

# Construct credible interval for Excitatory Neurons
ct = 'Excitatory neurons'
if f'means_cell_abundance_w_sf' in adata.obsm:
    means = adata.obsm['means_cell_abundance_w_sf']
    q05   = adata.obsm['q05_cell_abundance_w_sf']
    q95   = adata.obsm['q95_cell_abundance_w_sf']

    # Spots with narrow credible intervals = high confidence
    ci_width = q95[ct] - q05[ct]
    print(f'Mean CI width for {ct}: {ci_width.mean():.2f}')

    # High-confidence spots (CI width < 2)
    high_conf = ci_width < 2
    print(f'High-confidence spots: {high_conf.sum()} / {len(high_conf)}')
```

S1.3 Information-Theoretic Measures in Spatial Analysis

Several spatial transcriptomics analyses use information-theoretic measures. Mutual information (MI) measures the dependence between two variables: MI(X,Y) = 0 when X and Y are independent, and MI(X,Y) > 0 when they are associated. In spatial analysis, MI can measure how much knowing the expression of gene A tells you about the expression of gene B at the same spot.

Python

```python
from sklearn.metrics import mutual_info_score
import numpy as np

# ■■■ Mutual information between gene pairs ■■■■■■■■■■■■■■■■■■■■
def spatial_gene_mi(adata, gene_a, gene_b, n_bins=10):
    '''Compute mutual information between two genes.'''
    if gene_a not in adata.var_names or gene_b not in adata.var_names:
        return np.nan

    expr_a = adata[:,gene_a].X.toarray().ravel()
    expr_b = adata[:,gene_b].X.toarray().ravel()

    # Discretise into bins
    bins_a = np.digitize(expr_a, np.percentile(expr_a, np.linspace(0,100,n_bins+1)[1:-1]))
    bins_b = np.digitize(expr_b, np.percentile(expr_b, np.linspace(0,100,n_bins+1)[1:-1]))

    return mutual_info_score(bins_a, bins_b)

# Compute MI between top SVGs
top_svgs = adata.uns['moranI'].head(20).index.tolist()
mi_matrix = np.zeros((len(top_svgs), len(top_svgs)))
for i, g1 in enumerate(top_svgs):
    for j, g2 in enumerate(top_svgs):
        if i <= j:
            mi = spatial_gene_mi(adata, g1, g2)
            mi_matrix[i,j] = mi_matrix[j,i] = mi

# High MI = genes tend to be co-expressed in same spots
import pandas as pd
mi_df = pd.DataFrame(mi_matrix, index=top_svgs, columns=top_svgs)
print('Top co-expressed SVG pairs:')
mi_flat = mi_df.unstack()
print(mi_flat[mi_flat.index.get_level_values(0) !=
              mi_flat.index.get_level_values(1)].nlargest(5))
```

S2 — Advanced Analytical Workflows

S2.1 Stereo-seq Large-Scale Analysis

Stereo-seq (BGI Genomics) achieves unprecedented spatial resolution (220 nm centre-to-centre) at scales up to 1 cm². A complete mouse brain section requires multi-chip stitching and produces datasets with tens of millions of DNB bins. Effective analysis requires binning, hierarchical analysis, and efficient memory management.

Python

```python
import stereopy as st
import scanpy as sc

# ■■■ Load Stereo-seq GEF file ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
data = st.io.read_gef(
    file_path = 'mouse_brain_E16.5.gef',
    bin_type  = 'bins',
    bin_size  = 50    # 50×50 DNB bins ≈ 11 µm
)
adata = data.to_anndata()
print(f'Stereo-seq: {adata.n_obs:,} bins × {adata.n_vars} genes')

# ■■■ Multi-resolution analysis ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
for bin_size in [10, 50, 200]:
    data_bs = st.io.read_gef(file_path='mouse_brain_E16.5.gef',
        bin_type='bins', bin_size=bin_size)
    adata_bs = data_bs.to_anndata()
    sc.pp.normalize_total(adata_bs, target_sum=1e4)
    sc.pp.log1p(adata_bs)
    sc.pp.pca(adata_bs, n_comps=30)
    sc.pp.neighbors(adata_bs)
    sc.tl.leiden(adata_bs, resolution=0.5)
    n_cl = adata_bs.obs['leiden'].nunique()
    print(f'Bin size {bin_size:>3d}: {adata_bs.n_obs:>7,} bins, {n_cl} clusters')
```

S2.2 3D Spatial Reconstruction from Serial Sections

By registering multiple serial tissue sections, we can reconstruct three-dimensional spatial gene expression patterns. This is particularly informative for understanding tissue structures that are only partially captured by any single 2D section, such as blood vessel networks, laminar structures, or tumour infiltration patterns.

Python

```python
import numpy as np
from sklearn.preprocessing import StandardScaler

# ■■■ Register serial Visium sections ■■■■■■■■■■■■■■■■■■■■■■■■■
# Assume adata_sections is a list of AnnData objects,
# one per section, in order from anterior to posterior

def align_sections_by_shared_markers(sections, marker_genes, z_step=100):
    '''
    Align multiple sections along the z-axis using shared SVGs.
    z_step: distance between sections in micrometres.
    '''
    all_coords = []

    for z_idx, adata_s in enumerate(sections):
        # Get 2D spatial coordinates
        xy = adata_s.obsm['spatial']  # (n_spots, 2)
        z  = np.full((len(xy), 1), z_idx * z_step)
        xyz = np.hstack([xy, z])      # (n_spots, 3)

        # Normalise z to same scale as x,y
        xyz[:,2] *= xy[:,0].max() / (len(sections) * z_step)
        all_coords.append(xyz)

    return np.vstack(all_coords)

xyz_all = align_sections_by_shared_markers(
    sections=adata_sections, marker_genes=top_svgs, z_step=100)
print(f'3D reconstruction: {len(xyz_all)} spots in 3D space')
```

S2.3 Spatial Trajectory Along a Defined Axis

Sometimes we want to quantify gene expression along a defined spatial axis rather than an inferred pseudotime. For example: the dorsal-ventral axis of the spinal cord, the crypt-villus axis of the intestine, or the medial-lateral axis of the cortex.

Python

```python
import numpy as np
from scipy.interpolate import interp1d

# ■■■ Expression along a defined spatial axis ■■■■■■■■■■■■■■■■■■
# Define the axis by manually selecting two anchor points
# or by using the principal component of spot coordinates

# Option 1: Manual axis (e.g. dorsal-ventral)
dorsal_point   = np.array([250, 800])  # pixel coordinates
ventral_point  = np.array([250, 100])

# Project each spot onto this axis
def project_onto_axis(coords, p1, p2):
    axis = p2 - p1; axis = axis / np.linalg.norm(axis)
    return np.dot(coords - p1, axis)

spot_positions = project_onto_axis(
    adata.obsm['spatial'], dorsal_point, ventral_point)

# Bin spots by position along axis
n_bins = 20
bins   = np.linspace(spot_positions.min(), spot_positions.max(), n_bins+1)
bin_idx = np.digitize(spot_positions, bins[1:-1])

# Average expression per bin for top SVGs
import pandas as pd
top_genes = adata.uns['moranI'].head(10).index.tolist()
axis_profile = pd.DataFrame({
    gene: [adata[bin_idx==b, gene].X.toarray().mean()
           for b in range(n_bins)]
    for gene in top_genes if gene in adata.var_names
})
axis_profile.index = np.arange(n_bins) / n_bins  # normalised 0-1

print('Expression profiles along dorsal-ventral axis:')
print(axis_profile.round(3))
```

Appendix F — Troubleshooting Guide

This appendix provides solutions to the most common errors encountered when running spatial transcriptomics analysis pipelines in Python and R. For each error, the cause and step-by-step fix are provided.

Error: "ValueError: Graph not found. Run spatial_neighbors first...."
Context: squidpy spatial_autocorr or nhood_enrichment

Fix: You called sq.gr.spatial_autocorr() before building the spatial graph. For Visium: sq.gr.spatial_neighbors(adata, coord_type="grid", n_rings=1). For Xenium/irregular data: sq.gr.spatial_neighbors(adata, coord_type="generic", n_neighs=15).

Error: "ImportError: cannot import name AnnData from anndata (0.9.x)..."
Context: Any spatialdata or cell2location import

Fix: Version conflict: spatialdata requires anndata>=0.10. Fix: mamba install anndata=0.10.3 spatialdata=0.1.2 in the same command so the solver resolves all constraints simultaneously.

Error: "LayoutError: Flowable too large on page..."
Context: ReportLab PDF generation

Fix: A code block or table exceeds the page height. Split long code blocks into two shorter blocks. For tables, reduce font size or split into multiple tables.

Error: "Error in h5read: unable to open file..."
Context: Seurat Load10X_Spatial()

Fix: SpaceRanger output path is incorrect, or the filtered_feature_bc_matrix.h5 file is missing. Check: list.files("spaceranger_output/") and verify "filtered_feature_bc_matrix.h5" is present.

Error: "RuntimeError: CUDA out of memory..."
Context: cell2location or scVI GPU training

Fix: Reduce batch_size (try 256 or 128) or switch to CPU: mod.train(max_epochs=30000, batch_size=256). Alternatively, subsample the dataset: adata_sub = sc.pp.subsample(adata, n_obs=10000, copy=True).

Error: "Warning: 0 features in 1 variable detected; consider increas..."
Context: Seurat FindVariableFeatures()

Fix: Gene expression matrix is too sparse for the chosen n_features. Reduce nfeatures=2000 to 500, or lower min.cells threshold in CreateSeuratObject().

Error: "KeyError: spatial not in adata.obsm..."
Context: squidpy spatial_scatter or spatial_neighbors

Fix: Spatial coordinates were not loaded or were stored under a different key. Check adata.obsm.keys(). If loading manually from CSV: adata.obsm["spatial"] = coords_df[["x","y"]].values.

Error: "cell2location: NaN loss during training..."
Context: cell2location Cell2location.train()

Fix: Learning rate too high or data not properly normalised. Ensure adata.X contains raw integer counts (not log-normalised). Try lr=0.001 (default 0.002). Verify: adata.X.max() > 1 and adata.X.dtype is int.

Appendix G — Parameter Reference Tables

Scanpy / squidpy Key Parameters

Function	Parameter	Default	Recommended range	Effect
sc.pp.neighbors	n_neighbors	15	10–50	Controls cluster resolution; larger = smoother UMAP
sc.pp.neighbors	n_pcs	10	20–50	PCs to use; use elbow plot to decide
sc.pp.neighbors	metric	euclidean	cosine	Cosine recommended for RNA data
sc.tl.leiden	resolution	1.0	0.3–2.0	Higher = more clusters
sc.tl.umap	min_dist	0.5	0.1–0.5	Lower = tighter clusters in UMAP
sq.gr.spatial_neighbors	n_rings	1	1–2	Visium: rings of hex neighbors; 1=6 neighbours
sq.gr.spatial_neighbors	n_neighs	6	10–30	Irregular data: k nearest spatial neighbours
sq.gr.spatial_autocorr	n_perms	1000	100–10000	More perms = finer p-values but slower

cell2location Key Parameters

Parameter	Default	Effect	When to change
N_cells_per_location	8	Prior on cells per spot	Set to ~cells/spot from histology; 1–15 for Visium
detection_alpha	200	Controls spot-level noise	Lower (20–100) for high-noise data
max_epochs (step 2)	30000	Training iterations	Increase if loss still decreasing at 30k
batch_size	None	Full dataset per step	Set 512–2048 if GPU memory insufficient
lr	0.002	Learning rate	Reduce to 0.001 if NaN loss observed

Appendix H — Spatial Transcriptomics Databases & Resources

Resource	URL	Content	Species
Allen Brain Cell Atlas	atlas.brain-map.org	Spatial + scRNA-seq mouse & human brain	Mouse, Human
Human Cell Atlas Spatial	data.humancellatlas.org	Multi-organ Visium, Xenium datasets	Human
SODB (Spatial Omics DB)	sodb.iobio.io	Curated spatial datasets, all platforms	Multiple
Xenium Explorer datasets	10xgenomics.com/datasets	Public Xenium breast, brain, lung	Human, Mouse
spatialLIBD DLPFC	research.lieber.edu/hicks	12-section DLPFC Visium + manual layers	Human
MERSCOPE portal	vizgen.com/resources	MERFISH brain, cancer, organoid	Human, Mouse
CZ CELLxGENE	cellxgene.cziscience.com	Spatial + single-cell explorer	Multiple
NCBI GEO spatial	ncbi.nlm.nih.gov/geo	Published datasets in GEO format	Multiple

Appendix A — Python Package Reference

Package	Version	Purpose	Install Command
scanpy	1.9.6	Core scRNA/spatial analysis	pip install scanpy==1.9.6
squidpy	1.4.0	Spatial statistics & imaging	pip install squidpy==1.4.0
spatialdata	0.1.2	Multi-modal unified container	pip install spatialdata==0.1.2
spatialdata-io	0.0.14	Platform IO loaders	pip install spatialdata-io
anndata	0.10.3	HDF5-backed data matrix	pip install anndata==0.10.3
cell2location	0.1.3	Bayesian deconvolution	pip install cell2location
tangram-sc	1.0.4	scRNA → spatial mapping	pip install tangram-sc
liana	1.1.0	Ligand-receptor analysis	pip install liana
muon	0.1.5	Multi-modal AnnData	pip install muon
scvelo	0.3.1	RNA velocity	pip install scvelo
SpatialDE	1.1.3	Gaussian process SVGs	pip install SpatialDE
harmonypy	0.0.9	Harmony batch correction	pip install harmonypy
cellpose	2.2.3	Deep learning segmentation	pip install cellpose
hest	0.1.0	H&E + spatial RNA joint	pip install hest

Appendix B — R Package Reference

Package	Version	Source	Purpose
Seurat	5.0.1	CRAN	Core spatial + single-cell analysis
Signac	1.12.0	CRAN	Chromatin accessibility + spatial ATAC
BayesSpace	1.12.0	Bioconductor	Spatial domain detection (MRF)
spatialLIBD	1.14.1	Bioconductor	DLPFC Visium data + interactive viz
spacexr	2.2.1	GitHub: dmcable	RCTD deconvolution
Giotto	4.0.2	GitHub: drieslab	Full spatial analysis suite in R
monocle3	1.3.4	GitHub: cole-trapnell	Trajectory inference
CellChat	2.1.0	GitHub: jinworks	Cell communication inference
harmony	1.2.0	CRAN	Harmony batch correction
renv	1.0.3	CRAN	Reproducible environments
ggplot2	3.4.4	CRAN	Publication-quality figures
patchwork	1.2.0	CRAN	Multi-panel figure assembly

Appendix C — Example Datasets & Downloads

Dataset	Platform	Species	Cells/Spots	Source
V1_Adult_Mouse_Brain	Visium v1	Mouse	2,702 spots	10x Genomics public
V1_Human_Brain_Section_1	Visium v1	Human	3,460 spots	10x Genomics public
Xenium_Human_Breast_Cancer	Xenium	Human	167,782 cells	10x Genomics public
MERFISH_Mouse_Ileum	MERFISH	Mouse	~50,000 cells	Vizgen MERSCOPE portal
DLPFC_Visium_Maynard2021	Visium v1	Human	12 sections	Bioconductor spatialLIBD
Visium_Multiome_Brain	Visium Multiome	Mouse	~3,000 spots	10x Genomics public
CosMx_FFPE_Lung5	CosMx SMI	Human	~800,000 cells	NanoString AtoMx portal

Appendix D — Glossary of Key Terms

AnnData
Annotated Data; the core data structure of the Python spatial transcriptomics ecosystem, storing expression matrices with metadata in HDF5 format.

Baysor
Bayesian transcript-based cell segmentation using the spatial distribution of RNA molecules to define cell boundaries without nuclear staining.

BayesSpace
Bayesian spatially-aware clustering for Visium data, incorporating a Markov random field prior to produce anatomically coherent domain boundaries.

cell2location
Bayesian hierarchical model for decomposing Visium spot expression into per-cell-type proportions using a scRNA-seq reference atlas.

Cellpose
Deep learning segmentation model for nucleus and cell segmentation from fluorescence microscopy images.

CosMx
NanoString spatial molecular imaging platform (up to 6,000-plex panels, FFPE compatible).

Deconvolution
Statistical decomposition of a mixed expression signal (multiple cells per spot) into per-cell-type fractions using a reference dataset.

FFPE
Formalin-Fixed Paraffin-Embedded; standard tissue preservation method for clinical pathology, compatible with Visium FFPE, Xenium, and CosMx.

Geneformer
Transformer model pre-trained on 30 million single-cell transcriptomes for zero-shot cell annotation and perturbation prediction.

Harmony
Batch correction algorithm operating in PCA embedding space, iteratively adjusting embeddings to remove technical effects while preserving biology.

HVG
Highly Variable Gene; selected for downstream analysis based on having high biological variability relative to its mean expression level.

Leiden algorithm

Graph partitioning algorithm for clustering single-cell and spatial data, producing well-separated communities with guaranteed connectivity.

LIANA

LIgand receptor ANalysis frAmework; unifies multiple LR analysis methods and provides consensus interaction rankings.

MERFISH

Multiplexed Error-Robust FISH; image-based spatial platform detecting 100–10,000 genes at single-molecule resolution using Hamming-distance barcodes.

Moran's I

Classical spatial autocorrelation statistic measuring whether nearby locations have similar values; used for spatially variable gene detection.

MuData

Multi-modal AnnData; stores paired multi-omic measurements (e.g. RNA + ATAC) with shared and modality-specific metadata.

RNA velocity

Infers transcriptional change direction by comparing unspliced (nascent) to spliced (mature) RNA ratios per gene.

SCTransform

Normalisation via regularised negative binomial regression, outputting Pearson residuals with stabilised variance.

SpatialData

Unified spatial data format from the scverse ecosystem, storing images, segmentations, transcripts, and tables in Zarr format.

SpaceRanger

10x Genomics software for processing raw Visium sequencing data into gene-barcode count matrices with spatial coordinates.

SVG

Spatially Variable Gene; expression varies systematically across tissue beyond what is expected by chance.

Tangram

Deep learning method mapping single cells from scRNA-seq onto spatial locations by optimising expression concordance.

Visium

10x Genomics spatial gene expression platform using spatially barcoded 55 μm capture spots.

Visium HD

Next-generation Visium with 2×2 μm capture bins, achieving near-single-cell resolution with full-transcriptome coverage.

WNN

Weighted Nearest Neighbour; multi-modal dimensionality reduction weighting each modality by its predictive utility per cell.

Xenium

10x Genomics image-based spatial platform using RNA-ISH probes; optimised for FFPE tissue with single-molecule resolution.

Appendix E — Statistical Background

E.1 Negative Binomial Distribution for Count Data

Raw UMI counts follow a negative binomial (NB) distribution rather than a Poisson, due to biological over-dispersion. The NB distribution has mean μ and dispersion ϕ: Var[Y] = μ + μ²/ϕ. When ϕ→∞, variance approaches μ (Poisson). For spatial data, ϕ is typically 0.5–5 for most genes. SCTransform explicitly models this relationship to stabilise variance.

E.2 Moran's I — Mathematical Definition

For n spots with expression values x■, x■, ..., x■ and spatial weight matrix W (w■■ = 1 for neighbours, 0 otherwise):

I = (n / S■) × (∑■ ∑■ w■■ (x■ - x■)(x■ - x■)) / (∑■ (x■ - x■)²)

where S■ = ∑■ ∑■ w■■. Significance is assessed by permutation: the observed I is compared against 1,000+ randomised permutations of expression values across spots.

E.3 Multiple Testing Correction

When testing 3,000 HVGs for spatial variability, 150 false positives are expected at p < 0.05 by chance. Benjamini-Hochberg (BH) FDR correction controls the expected proportion of false discoveries at the chosen threshold — less conservative than Bonferroni, standard for SVG detection.

Python

```python
from statsmodels.stats.multitest import multipletests
import numpy as np

raw_pvals = np.array([0.001, 0.01, 0.05, 0.1, 0.5])

# BH FDR correction
reject_bh, pvals_bh, _, _ = multipletests(
    raw_pvals, alpha=0.05, method='fdr_bh')
print(f'Significant at 5% FDR (BH): {reject_bh.sum()}')

# Bonferroni (more conservative)
reject_bf, pvals_bf, _, _ = multipletests(
    raw_pvals, method='bonferroni')
print(f'Significant (Bonferroni): {reject_bf.sum()}')
```

E.4 Bayesian Deconvolution Model Structure (cell2location)

cell2location models expression of gene g in spot s as:

Y_sg ~ NegativeBinomial(μ_sg, α_g)

μ_sg = (∑_k w_sk · r_kg) · d_s

where w_sk is the abundance of cell type k in spot s, r_kg is the reference signature of cell type k for gene g, d_s is spot-level detection efficiency, and α_g is gene-level dispersion. The prior on w_sk incorporates expected spatial structure.

E.5 Ripley's K Function

Ripley's $K(r) = (1/n^2) \times E[\text{number of additional points within distance } r \text{ of an arbitrary point}] / \lambda$, where λ is the point intensity. $K(r) > \pi r^2$ indicates clustering at scale r; $K(r) < \pi r^2$ indicates regularity. The L function $L(r) = \sqrt{(K(r)/\pi)} - r$ centred at 0 is easier to interpret.

Index

Trajectory / pseudotime	Ch. 9
Tumour microenvironment	Ch. 15, 25
Visium	Ch. 1, 4-9, 24, Appendix C, D
Xenium	Ch. 1, 10-15, 25, Appendix C, D